Hybrid Adaptive Consensus: A Sovereign Finite-State-Machine Fallback for Modular Blockchains

Tien Cuong Dang[1] Binh Minh Nguyen[1,*]

[1]Hanoi University of Science and Technology, Hanoi, Vietnam

cuong.dt220020@sis.hust.edu.vn, minh.nguyenbinh@hust.edu.vn

*Corresponding author

Abstract

Modular blockchains obtain settlement and data availability (DA) from independently operated layers. A chain whose block validity requires Bitcoin settlement receipts and DA receipts inherits the liveness of those layers: when either stalls, no proposal satisfies the validity predicate and the chain halts, although its validators are connected and correct. Dropping the requirement restores progress but suspends the guarantees it provided. We present *Engram Hybrid Adaptive Consensus*, a protocol layer on a CometBFT agreement core that makes this suspension an agreed, deterministic state transition. Engram adds (i) a *health certificate*, a deterministic aggregate of the height-bound, signed peripheral observations of a quorum, carried as vote extensions and bounded by the range of honest observations (Lemma 1), and (ii) a four-state finite state machine (ANCHORED, SUSPICIOUS, SOVEREIGN, RECOVERING) evaluated deterministically on that certificate. When Bitcoin, DA, or the peripheral P2P view degrades, the chain keeps producing blocks under the validator quorum, relaxes the receipt requirement of the unhealthy layer, and deterministically excludes withdrawal-marked transactions while it is not anchored; the exclusion takes effect only once the FSM locks. It returns to ANCHORED after recovery hysteresis, DA backfill of the degraded interval, and a validator-verified zero-knowledge recovery proof (Noir/UltraHonk, 256 headers per proof).

A layered TLA+ specification, checked layer by layer with bounded model checking, has zero violations at $N \leq 4$ (the largest run explored 4.54 million distinct states); a four-mechanism ablation reproduces a distinct predicted failure for each removed mechanism. The cross-layer refinement obligation remains open. On a 7-validator Cosmos SDK testnet with live Bitcoin regtest and Celestia networks, a layered composite fault, repeated five times, leaves the chain committing blocks through a full SOVEREIGN→RECOVERING→ANCHORED cycle with 100% AppHash agreement. The health certificate in each proposal is about 0.9 KB at seven validators (a fixed 70 bytes plus 118 per validator) and takes 263 μs to validate. Against seven vanilla CometBFT validators on the same hosts, the block interval is 1.262 s against 1.233 s (a few percent at most: point estimate 2%, 95% interval −0.6 to 5.0% over five deployments, which includes zero), with every height committing in the first round. Against our strict receipt-gated baseline, which enforces the same receipts at every height, Engram keeps 96–102% of its healthy block rate under a checkpoint pause, a DA outage and both, whereas the baseline falls to 56%, 16% and 17% and commits no block in the 120 s after the layer returns in all nine trials (three runs per cell, one host). Engram's cost is the detection delay: 18–56 withdrawal-marked transactions were committed before it locked (41–117 s after the fault began). From the lock until the fault ended it committed none, which checks the circuit breaker against Engram's own transition rather than measuring exclusion independently; a baseline that keeps Engram's liveness but never locks lets 31–72 through in the same window.

Keywords: blockchain consensus, Byzantine fault tolerance, modular blockchain, finite state machine, formal verification, TLA+, zero-knowledge proofs

1. Introduction

Modular blockchains separate execution from services such as settlement and data availability (DA), which independently operated networks provide. Consider a chain whose validity predicate requires fresh evidence from such layers: a block is valid only if it passes the BFT checks and carries both a Bitcoin settlement receipt and a receipt from a DA network such as Celestia [1], each attesting a height within a tolerance of the current Bitcoin tip (τ_{BTC}) or of the block (τ_{DA}). Formally,

$$\mathit{valid}(B) = \mathit{BFTvalid}(B) \wedge \mathit{BTCreceipt}(B) \wedge \mathit{DAreceipt}(B).$$

If Bitcoin settlement is congested or censored, or the DA network is unavailable, no proposal satisfies the predicate. The BFT consensus layer [2, 3] built on top stalls although the validator set is fully connected and correct. The chain inherits the liveness of the layers it depends on.

Dropping the requirement restores progress but suspends the guarantees that motivated the modular design: without a settlement checkpoint the chain cannot claim externally anchored settlement, and without DA confirmation it cannot claim that its recent data is externally available. The problem is therefore not only to detect an outage. The protocol must make all validators agree on when an external guarantee may be suspended, restrict the operations whose safety depended on it, and define the evidence required before the guarantee is restored.

The gap. Existing work addresses parts of this problem under other failure models. Protocols that trade finality for availability adapt to validator participation or consensus-network conditions, not to external infrastructure [4,5]. Rollup mechanisms provide alternative paths when sequencing or data publication fails, but the decision rests with users, a sequencer, a committee, or a contract on the parent chain [6–8]. Oracle designs and ABCI++ vote extensions let validators bring externally observed values into replicated state [9,10], but the value goes to the application. None of them covers a native chain whose validity depends on independently operated external layers that different validators observe differently. There, validators must agree on the observation and on the change in safety policy that follows from it (Section 2).

Why the decision belongs in consensus. The relaxation of receipt requirements and the withdrawal circuit breaker change two things that every validator must agree on: whether a block is valid, and whether a withdrawal executes. Bitcoin, DA, and peripheral-network status are observed locally and asynchronously, so two honest validators may see different conditions at the same instant. Suppose each validator applied a local policy to its own view. Validators would disagree on validity, so proposals would collect nil votes and rounds would repeat, or they would execute withdrawals differently and produce diverging `AppHash` values. A local, non-deterministic observation cannot drive replicated state directly. It must first become a deterministic input that every validator evaluates identically.

Design approach. Engram keeps CometBFT's proposer selection, prevote/precommit protocol, quorum intersection, and deterministic finality unchanged, and modifies the object and the validity policy that this pipeline carries. At height $h-1$, each validator signs its sensor snapshot as a vote extension; the sensors are read only there. The proposer of height h includes the signed snapshots of a quorum (more than two thirds of the voting power) in a *health certificate* C^h. Every validator verifies C^h, recomputes a deterministic weighted-median/majority aggregate $\mathsf{Agg}(C^h)$, and accepts a proposal only if its target state satisfies $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$, where δ is the transition function of a four-state finite-state machine (FSM) with states Anchored, Suspicious, Sovereign, and Recovering. FSM state is written only from a certificate that CometBFT has already agreed on (Section 4.3). "Hybrid Adaptive Consensus" denotes this adaptive layer inside the consensus pipeline, not a replacement for the agreement protocol.

The FSM makes the chain's security mode explicit. In Anchored, blocks require the Bitcoin and DA receipts. Sustained degradation moves the chain through Suspicious into Sovereign, where

the quorum keeps ordering blocks, the receipt requirement of each unhealthy layer is relaxed, and cross-chain withdrawals are locked. Return to ANCHORED requires recovery hysteresis, DA backfill of the SOVEREIGN interval, and a validator-verified zero-knowledge proof over that interval (Sections 4.4 and 4.7; the prototype's proof gate is qualified in Section 8.3).

Scope. Engram handles degradation of Bitcoin, Celestia, and the peripheral P2P interfaces while the CometBFT quorum remains live. It does not address loss of the core quorum: like any BFT protocol, it needs at least a quorum (more than two thirds of the voting power) to communicate after global stabilization time. "P2P degradation" in this paper always means partial, peripheral degradation (Section 3.1).

To our knowledge, no published design combines the following for a modular chain whose validity depends on external settlement or DA layers: (i) a quorum-backed set of validator observations from which every validator deterministically derives external-layer health; (ii) deterministic adaptation of both block-validity requirements and withdrawal admissibility from that health while the native quorum keeps ordering blocks; and (iii) a recovery condition that requires a proof over the degraded interval and its DA backfill. Engram targets this combination.

Contributions.

- **Quorum-backed external-health agreement.** A height-bound certificate of signed observations from a quorum, carried in vote extensions, and a deterministic weighted-median/majority aggregation rule. When Byzantine validators hold less than one third of the voting power, every numeric aggregate stays within the range of the honest observations in a well-formed certificate, whichever observations a Byzantine proposer includes or omits (Section 4.3).
- **Dependency-aware adaptive execution.** A four-state FSM on the certificate, with asymmetric hysteresis and exponential backoff on repeated failed recovery, that changes the Bitcoin and DA receipt requirements without modifying CometBFT's agreement core. A withdrawal circuit breaker [11] excludes withdrawal-marked transactions while the chain runs without external corroboration (Sections 4.4–4.5; Withdrawal Admission Safety theorem in Section 5.1).
- **Proof- and backfill-gated recovery.** A Noir/UltraHonk circuit that proves, for up to 256 SOVEREIGN-interval headers at constant proving and verification cost, header-chain continuity, that every recorded state is SOVEREIGN or RECOVERING with the lock flag set, the binding of the interval to the last anchored root and to the new root, and the quorum signatures of the two boundary blocks. It does not check transitions against the transition function or show that no withdrawal executed. Longer intervals chain several proofs (Appendix C). The exit condition also requires that the interval's data was published to the DA layer. Validators verify each proof inside block execution (Section 4.7).
- **Layered formal specification.** A four-layer TLA+ specification. Bounded model checking of each layer's safety and liveness properties found zero violations at $N \leq 4$; the largest single run explored 4.54 million distinct states. In a four-mechanism ablation study, removing each mechanism reproduces a distinct predicted counterexample. The cross-layer refinement obligation is specified but open (Section 5).
- **Live evaluation.** A 7-validator Cosmos SDK testnet against live Bitcoin regtest and Celestia networks with network- and application-level fault injection, comparing Engram against a strict receipt-gated baseline and a vanilla CometBFT control (Sections 6–7).

The remainder of this paper is organized as follows. Section 2 situates Engram in the literature. Section 3 states the system model, the threat model, and the security goals. Section 4 defines the protocol: sensors, the health certificate, the FSM, proposal validity, and recovery. Section 5

Table 1: Research questions

RQ	Question
RQ1	**Safety.** Under Byzantine voting power below one third (two of the seven testbed validators), do honest validators stay in agreement on the FSM state and the certified health, and is no withdrawal-marked transaction committed while the FSM is locked? Safety here is consensus safety; asset-level safety is outside the prototype's scope (Section 8).
RQ2	**Liveness.** When Bitcoin settlement or DA becomes unavailable, does Engram keep committing blocks where a strict receipt-gated baseline stalls, and what does it give up: how many withdrawals commit before its circuit breaker locks, compared with a baseline that never locks? A degraded peripheral P2P view does not affect a baseline gated on receipts, so it is examined under RQ1.
RQ3	**Recovery.** Once peripherals recover, do the DA backfill and the recovery proof return the chain to ANCHORED? And how much does the FSM's hysteresis reduce flapping, at what cost in detection delay?
RQ4	**Cost.** What overhead do the extended consensus object, certificate validation, and recovery proof add, and where does it come from: proposal size, validation time, or additional consensus rounds?

states the security properties and reports the formal specification and bounded model checking. Section 6 describes the implementation, testbed, baselines, and methodology, and Section 7 answers RQ1–RQ4. Section 8 discusses the security envelope, limitations, and future work, and Section 9 concludes.

2. Related Work

A chain whose block validity depends on external layers raises five questions, and each has been studied by a separate community. Fig. 1 organizes the literature by these questions: how consensus behaves under degraded conditions (Section 2.1), how local observations of an external layer become data that all validators see and agree on (Section 2.2), how modular designs cope with the failure of Bitcoin, DA, or settlement layers (Section 2.3), how a chain returns to the anchored state (Section 2.4), and how such designs are validated (Section 2.5). Several building blocks of Engram exist in prior work: vote extensions that carry signed external data, weighted-median aggregation, and protocol-defined fallback for DA and sequencing failures. Engram differs in what the aggregated observation controls. Table 3 compares representative systems by trigger, decision authority, behavior under failure, recovery, assumption, and reported assurance; Section 2.6 states the combination left open.

2.1. Consensus Under Degraded Conditions

Agreement core. PBFT [50] and the partial-synchrony model of Dwork, Lynch, and Stockmeyer [51] underlie Tendermint and its CometBFT successor [2, 3], which provide deterministic finality with a rotating proposer. HotStuff [52] achieves responsiveness and linear communication under a correct leader within the same leader-based lineage. Narwhal/Tusk [53] separate transaction dissemination from ordering, and Bullshark [54] orders transactions on top of the Narwhal DAG. These protocols agree on the content of a proposal; whether a proposal is acceptable is decided by an application predicate that every correct validator must evaluate identically. Tendermint's consensus problem includes such a `valid()` predicate [3], and Cachin et al. [55] introduced the *external validity* condition for multi-valued Byzantine agreement. Engram relies on the same structure: its proposal-validation predicate depends on the health certificate, so the predicate stays deterministic only if the certificate is itself agreed on. BFT forensics [56] studies how many malicious replicas can be identified after a safety violation. AdoB [57] mechanizes

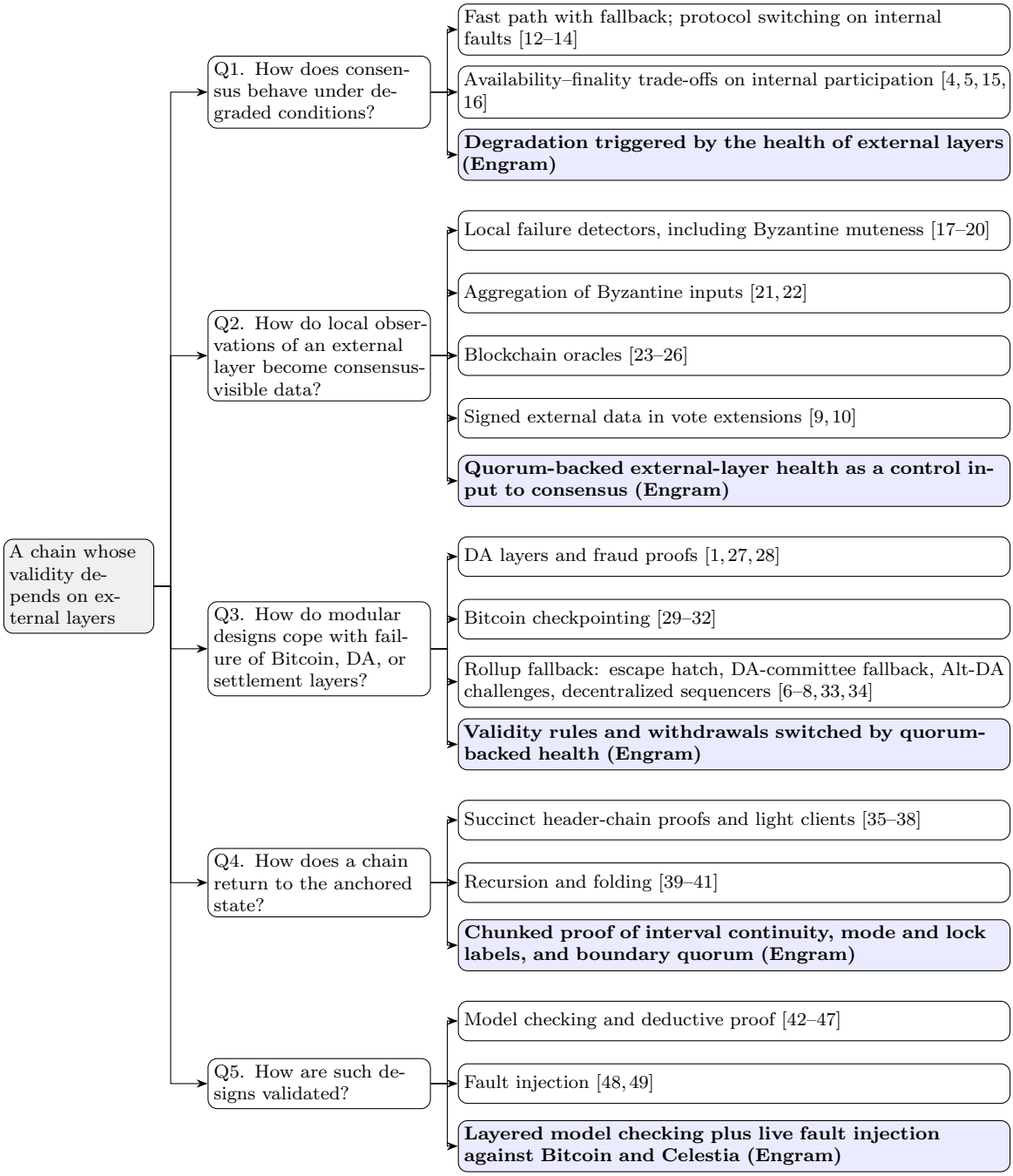


Figure 1: Taxonomy of related work, organized by the question each line of work answers. Shaded leaves mark where Engram sits. Vote-extension oracles supply signed external data to consensus (Q2) but attach no fallback semantics to it.

benign and Byzantine consensus in one model, and Adore [58] adds certified reconfiguration. Engram inherits CometBFT's evidence pool for accountability and leaves the agreement core unchanged.

Fast paths and protocol switching. A second group changes the protocol's behavior in response to conditions that the replicas themselves observe. Abstract [13] lets a replicated state machine abort a client request when something goes wrong, which supports composing and reconfiguring BFT protocols. Thunderella [12] combines an optimistic fast path with a synchronous fallback, and Autobahn [14] targets seamless recovery from network blips. The trigger in each case is a fault of the replicas or of the network among them.

Availability–finality trade-offs. The closest conceptual relatives give up finality to keep the chain growing. The sleepy model [15] lets players be online or offline and gives a consensus protocol that is resilient as long as a majority of the online players are honest. Ebb-and-flow protocols [4] expose both a dynamically available ledger and a finalized prefix, so the chain keeps growing when finality stalls and the finalized ledger catches up on recovery. Casper FFG [16], and Gasper [5], which combines it with the LMD GHOST fork-choice rule, prove accountable safety and plausible liveness. For the case in which more than a third of the validators crash, Casper FFG proposes an inactivity leak that drains the deposits of non-voting validators until the voting validators again form a supermajority, and leaves the exact formula open. These designs share the "degrade, do not halt" goal and the tension described by the CAP theorem [59]. They differ from Engram in two respects: the trigger is *internal* participation, and the response changes how finality is reached, not which blocks and withdrawals are valid.

2.2. From Local Observations to Consensus-Visible External Data

Failure detectors. Unreliable failure detectors [17], their QoS metrics [18], and accrual detectors [19] give each process a local, eventually accurate suspicion of other processes. Muteness failure detectors [20] extend the abstraction to Byzantine processes that stop sending the messages an algorithm expects while possibly continuing to send others. Engram's sensors play the same role for peripheral layers, with two differences: a local suspicion has no effect until a quorum of signed observations backs it, and the adversary controls up to f of the reporters.

Aggregating Byzantine observations. Approximate agreement [21] requires non-faulty processes to decide on values that are close together rather than identical. Median validity [22] requires the decision to be close to the median of the non-Byzantine inputs and, at maximal fault tolerance with $n \geq 3t + 1$, within the range of those inputs. The Engram health certificate satisfies an analogous range-validity property for each weighted-median metric. It does not run a second approximate-agreement protocol: validators sign individual height-bound sensor snapshots carried as vote extensions, CometBFT agrees on the proposal that carries them, and every validator then recomputes the aggregate deterministically (Section 4.3).

Blockchain oracles. Oracle research addresses how replicated applications import external information. Town Crier authenticates web data with trusted hardware [23]. A systematization of oracle designs classifies their trust assumptions and manipulation risks [24]. DecenTruth combines multi-source truth discovery with decentralized consensus to tolerate dishonest reports [25], and PSCBO analyzes attacks on consensus-based oracle designs [26]. In all of these, the oracle supplies a value to the application layer.

Vote extensions. The closest mechanism to Engram's certificate is the ABCI++ vote extension. Each validator extends its precommit with application data, which may be obtained non-deterministically, and the next proposer includes the signed extensions in its proposal. The Cosmos SDK design record requires signatures from at least two thirds of the voting power and uses an oracle price fetcher as its example [9]; Slinky applies the pattern to off-chain

price data and aggregates the extensions with a stake-weighted median [10]. Engram uses the same transport: sensor snapshots taken at height H reach the proposer at $H+1$, where their deterministic aggregation forms the health certificate. The difference lies in what the aggregate controls. A price oracle supplies a value to the application; the health certificate describes the infrastructure on which block validity depends and determines the receipt requirements, the withdrawal policy, and the entry into and exit from a degraded mode.

Peripheral P2P attacks. Sybil attacks [60], in which one entity presents multiple identities, and eclipse attacks on Bitcoin's P2P layer [61, 62], which monopolize a node's connections, let an adversary control what a node sees of an external network. They motivate the multi-signal peer-health profile that Engram's P2P sensor uses and are one reason a single local reading is not trusted.

2.3. Fallback for Modular-Chain Dependencies

Data availability. LazyLedger/Celestia [1] decouples DA from execution. Fraud and data availability proofs [27] let light clients detect withheld data, and coded distributed arrays target the peer-to-peer layer that distributes data availability sampling fragments [28]. This line of work makes DA *verifiable*, but a chain that requires a DA inclusion proof before accepting a block still stops when DA stops. Engram adds a consensus-level response to DA unavailability and a backfill requirement before the chain returns to ANCHORED (Section 4.7).

Bitcoin anchoring. Pegged sidechains [63] introduced two-way pegs to Bitcoin. Karakostas and Kiayias [29] study checkpointing as a defense of proof-of-work ledgers against 51% attacks and report a liveness attack, "block lead", against existing checkpointing designs. For proof-of-stake chains, Babylon [30, 32] and Pikachu [31] checkpoint onto Bitcoin to prevent long-range attacks. Tas et al. [32] show that slashable safety is impossible without an external source of trust and state their guarantees under the assumption that Bitcoin is safe and live. Their design also defines a fallback in which Bitcoin orders the PoS blocks after censorship, which strengthens the PoS chain's dependence on Bitcoin. To our knowledge, none of these publications specifies how the chain operates while the anchor itself is unreachable. Engram treats unreachability as an input backed by a quorum of signed observations, keeps ordering blocks, and locks withdrawals while it persists. Accountability comes from CometBFT's equivocation evidence; Engram implements no Bitcoin-enforced slashing.

Rollup fallback and its decision locus. When a rollup operator stops building blocks or proposing state roots, escape hatches [6] let users withdraw on the L1 by proving their assets against an L2 state root. Arbitrum [64] resolves disputes about contract behavior with a challenge-based protocol on the L1. Surveys of layer-two protocols [65] treat the ability of honest users to withdraw as a core security property, and list among the open weaknesses that layer-two designs assume the underlying blockchain eventually accepts transactions. Capretto et al. [33] give fraud-proof games, mechanized in Lean 4, with which an honest agent can prove on the L1 that a sequencer-and-DAC "arranger" violated a safety property even if all of its replicas are compromised; a liveness violation that preserves safety cannot be proved on the L1. A follow-up decentralizes the sequencer and the DA committee with set consensus [34].

Two designs define a protocol-level response to a DA failure. AnyTrust normally relies on a data-availability committee. If the sequencer cannot collect enough committee signatures within a few minutes, it posts the full data to the parent chain as a non-AnyTrust rollup would do [7]. Withdrawals then incur the delay of a standard rollup, and the chain switches back to the cheaper mode when the committee resumes [66]. The sequencer makes this decision, and the failure it reacts to is a committee failure. OP Stack Alt-DA keeps data with an external DA provider and lets anyone challenge a commitment on the L1. If the data is not posted before the resolve window elapses, derivation reorgs the L2 blocks that depended on it [8]. This design answers

the same question as Engram, what happens when an external DA input fails, by resolving the failure on the L1 and discarding the affected blocks.

These designs show that protocol-defined fallback for DA and sequencing failures exists. They differ from Engram in three respects:

- *Who decides.* A sequencer, a committee, users, or an L1 contract; Engram decides inside the validator set's own consensus instance.
- *What is observed.* A committee or data-posting failure, not the health of Bitcoin settlement or of an external DA network; Engram uses signed observations of all three peripheral layers.
- *What the response changes.* It publishes more data or reorganizes the chain; Engram changes the validity predicate and the withdrawal policy and commits the FSM state in the same block.

Withdrawal risk. Lee et al. [67] break a bridge into its components, identify the risk each introduces, and analyze past exploits that targeted bridges. They report that over USD 1 billion in digital assets was stolen, incorrectly minted, or locked in bridges in the year before their study. Withdrawals are the path through which a chain without external corroboration can lose assets, which is why Engram locks them while the chain runs in SOVEREIGN mode.

2.4. Recovery and Re-Anchoring

Succinct proofs are the standard tool for convincing a verifier of a chain's history without re-executing it. They differ in how a long history is covered. Mina [37] keeps a constant-size proof of the whole chain by recursively composing SNARKs, which Plumo's authors report imposes a large overhead on the protocol being proved [36]. zkBridge [35] verifies source-chain consensus inside a two-layer recursive proof system. Plumo instead syncs a client through a chain of sequential SNARK transition proofs, each covering four months of blockchain history [36]. Recursive composition [39, 40] and folding [41] are the general techniques behind the recursive designs. FlyClient [38] takes a different route for proof-of-work chains: a light client downloads only a logarithmic number of block headers, selected by probabilistic block sampling. All of these prove that a header chain, or a validator-set transition, is valid. A chain that has run without external corroboration must also show that the interval followed the protocol. The long-range attack literature [68] reviews how an attacker can rewrite the history of a proof-of-stake chain, which is the exposure of an interval secured only by the chain's own validators. Engram's recovery proof is a fixed-capacity PLONK-family [69] proof over one interval chunk, and chunk proofs are chained through an on-chain root, as Plumo chains its transition proofs. The statement differs: besides header continuity it covers the recorded mode and lock-flag label of every header and the quorum signatures of the two boundary blocks, and it does not check transitions against the transition function (Section 4.7).

2.5. Validation Methodology

TLA+ [70] with TLC and Apalache [42] has been used to specify and model-check Tendermint's blockchain synchronization protocol [43]. Deductive approaches establish properties by machine-checked proof instead of by exploring a bounded instance. Ivy [44] verifies safety of infinite-state systems interactively, and Lotan and Shoham [71] automate liveness proofs in first-order logic over finite but unbounded domains. IronFleet [45] and Verdi [46] verify distributed-system implementations, the latter in Coq, and LiDO [47] gives mechanized safety and liveness proofs in Coq for unpipelined and pipelined Jolteon. A recent preprint places these approaches on a maturity scale [72]. Engram sits in the model-checking family, with a per-layer result and an open

Table 2: What the failure of an external dependency stops, by design (entries from the sources of Table 3, except the baseline and Engram).

Gate	What stops	Design
Block validity	Every proposal, while the validity predicate cannot be satisfied	Strict receipt-gated baseline (this work)
Finality	The finalized prefix lags; the ledger keeps growing	Ebb-and-flow
Client acceptance	A client stops adding blocks whose data behind a signed checkpoint is unavailable	Babylon emergency break
L2 derivation	Derivation stalls until a challenge resolves the data or the blocks are reorged	OP Alt-DA
Oracle input	Nothing: an empty vote extension is valid and the chain keeps running	Vote-extension oracle
An operation	Withdrawals, and the return to the anchored mode; block production continues	Engram

refinement obligation (Section 5). On the empirical side, Twins [48] tests BFT implementations by duplicating validator identities, and chaos engineering [49] injects faults into production systems. Engram's evaluation combines network-level and application-level fault injection against live Bitcoin and Celestia networks.

2.6. Research Gap

Prior work supplies consensus that adapts to internal participation faults, oracle systems that aggregate external observations, vote extensions that deliver signed external data to the validator pipeline, rollup fallbacks for sequencing and DA failures, and succinct proofs of chain histories. When an external layer fails, a chain that gates validity on it halts, and a chain that drops the dependency gives up the guarantees that motivated it. The fallbacks surveyed above place the decision with users, watchers, a sequencer, a committee, or an L1 contract.

What a response gates. The designs differ in what the failed dependency stops (Table 2). Only the strict receipt-gated baseline stops block commitment. The other designs stop finality, client acceptance, L2 derivation or nothing, or leave the chain running and gate an operation.

To the best of our knowledge, no published design combines (i) a quorum-backed set of height-bound validator observations from which every validator deterministically derives the health of external layers, (ii) use of that health inside the validator consensus pipeline to change both proposal-validity requirements and withdrawal admissibility while the native validator quorum keeps ordering blocks, and (iii) a recovery condition that requires a proof over the degraded interval and the backfill of that interval to the DA layer before the chain returns to the externally anchored mode. Engram provides these three (with the proof-gate limitation of Section 8.3), retains CometBFT as the agreement protocol, and Sections 3–7 evaluate the combination.

3. System Model and Security Goals

This section fixes who participates, what may fail, what is trusted, and which properties the protocol is meant to give in each FSM state.

3.1. Validators, Network, and Assumptions

Validators and network. We assume partial synchrony: message delay is unbounded before an unknown global stabilization time (GST) and bounded after it [51]. Let V be the validator set, $w_i > 0$ the voting power of $i \in V$, $W = \sum_{i \in V} w_i$ the total voting power, and $\mathrm{wt}(S) = \sum_{i \in S} w_i$

Table 3: Failure handling in representative systems, from each system's primary publication or specification, except the baseline and Engram rows, which are the two designs evaluated in this paper; "n/a": not stated by the source. The assurance column does not rank the designs.

System	Trigger and observation	Decided by	Behavior under failure and recovery	Assumption	Assurance reported
Reference point					
Strict receipt-gated baseline (this work)[a]	Missing Bitcoin or DA receipt	Validity predicate	No proposal is valid until the receipt returns	External layers live	n/a
Degradation inside consensus					
Ebb-and-flow [4]	Dynamic participation or network partition	Validator protocol	Ledger keeps growing and the finalized prefix lags; the prefix catches up	Partial synchrony (finalized ledger); synchrony (available ledger)[b]	Security proof; simulations
Casper FFG leak [16]	More than one third of validators stop voting	Validator protocol	Inactivity leak drains non-voting deposits; formula left open	PoW overlay; under 1/3 slashable stake (safety); 2/3 voting (liveness)	Accountable-safety and plausible-liveness proofs
External data in consensus					
Vote-extension oracle [9, 10]	External data update; signed vote extensions	Validator consensus pipeline	Stake-weighted median goes to the application; an unreachable oracle yields an empty vote extension that counts as valid, so the chain keeps running; the ADR prescribes no fallback	Signatures from at least two thirds of voting power	Design record; open-source code
Bitcoin anchoring					
Babylon [32]	Transaction censorship; a signed checkpoint whose block is unavailable	Clients and Bitcoin[c]	Bitcoin orders PoS block hashes (rollup mode); the chain later exits rollup mode; clients stop adding blocks that lack available data (emergency break)	Bitcoin safe and live	Security proofs; measured Bitcoin checkpoint latency
Rollup fallback					
Escape hatch [6]	No new state root within a time limit	Users	Users bypass the bridge and withdraw with a Merkle proof; a nullifier blocks a second escape	L1 live; users submit proofs	Design and use cases
AnyTrust [7, 66]	Too few committee signatures	Sequencer	Full batch posted to the L1 with standard rollup delay; the committee path resumes	L1 live	Public documentation
OP Alt-DA [8]	External DA input unavailable; L1 challenge	L1 contract	Derivation stalls on missing data; the data is resolved on the L1, or the affected L2 blocks are reorged	L1 live; some party challenges	Experimental specification
Secure arranger [33]	Safety violation by the arranger	Honest agent and	Fraud proof settles on the L1; liveness	L1 safe and live; one honest	Fraud-proof games

the weight of $S \subseteq V$ (Table 4). The adversary controls a set $B \subseteq V$ of Byzantine validators, which may deviate arbitrarily and collude; the other validators $H = V \setminus B$ are honest. As in CometBFT, a *quorum* is a set whose weight exceeds two thirds of the total:

$$\mathrm{wt}(B) < \frac{W}{3}, \qquad \mathrm{wt}(S) > \frac{2W}{3} \ \text{ for every quorum } S. \tag{1}$$

The seven-validator testbed gives every validator the same power, so Eq. (1) allows f=2 Byzantine validators and a quorum is any five validators. The adversary cannot forge signatures, find hash collisions, or subvert the UltraHonk backend's KZG setup. After GST, the *core consensus network* connects a quorum, as CometBFT itself requires.

Table 4: Notation used in Sections 3–5.

Symbol	Meaning
V, w_i, W	validator set, voting power of validator i, total voting power
$\mathrm{wt}(S)$	total voting power of the validators in S
B, H	Byzantine and honest validators
h, r	consensus height and round
m_i, o_i	local metric vector and signed observation of validator i
C^h, Agg	health certificate of height h, deterministic aggregation
s^h, δ	FSM state committed at height h, transition function
τ_{BTC}, τ_{DA}	receipt tolerance of the Bitcoin receipt (Bitcoin blocks) and of the DA receipt (Engram blocks)

Validator set. The validator set is the genesis set and stays fixed throughout a SOVEREIGN or RECOVERING interval. Certificate signatures verify against the genesis validator keys, and the compiled recovery circuit fixes the set size at seven. Validator-set changes inside a degraded interval would require proving the set transitions and are outside this paper.

Peripheral layers. Bitcoin, Celestia, and the peripheral P2P interfaces may be slow, unavailable, partitioned from some validators, or adversarially degraded (delayed, censored, eclipsed) for arbitrarily long. They are not assumed to be Byzantine-fault-tolerant from the validator set's perspective, but Bitcoin reorganizations deeper than a configurable depth $K_{\mathrm{DEEP_FINALITY}}$ are excluded, as in every Bitcoin-anchoring design (Section 2). Honest validators' peripheral observations may differ at any instant, and identical views are not assumed. The health certificate (Section 4.3) gives every validator the same aggregate under arbitrary honest disagreement; whether a sensor reading matches the external layer is a separate question (Section 4.2).

Parameters. FSM thresholds, hysteresis limits, and $K_{\mathrm{DEEP_FINALITY}}$ are genesis parameters shared by every validator, like the block time or quorum size. A validator running different parameters simply fails to validate honest proposals; this affects only its own participation, not safety.

Layered adversary. The adversary may degrade the peripheral connectivity of *every* honest validator at once (delay Bitcoin headers, cut the DA bridge, eclipse the peripheral P2P interface), but it cannot partition the core consensus network below a quorum. Degrading the periphery changes the FSM state; it does not by itself stop block production.

Byzantine power of one third or more. Beyond $W/3$, no BFT protocol retains agreement. In ANCHORED and SUSPICIOUS, a checkpoint that carries the CometBFT block hash lets any Bitcoin observer detect a fork, because two conflicting histories cannot both match a Bitcoin-timestamped hash; this is accountability, not prevention. In SOVEREIGN nothing outside the validator set constrains the chain, which is why withdrawals are locked in that state (Section 3.2).

3.2. Security Goals by Mode

We state what each FSM state guarantees, using four properties of the committed chain $B_1, \ldots, B_h$. A *prefix* $B_1, \ldots, B_c$ is the chain up to height $c \leq h$.

- P_{agree}: no two conflicting blocks are committed at the same height. Assumption: $\text{wt}(B) < W/3$.
- $P_{\text{settle}}(c)$: the prefix up to c cannot be rewritten without a Bitcoin reorganization deeper than $K_{\text{DEEP_FINALITY}}$. Assumptions: such reorganizations are excluded (Section 3.1), and the Bitcoin checkpoint that buries height c commits the chain through c (assumption M2 of Theorem 1, Section 5.1).
- $P_{\text{avail}}(c)$: the block data of every height up to c is retrievable from the DA layer. Assumption: the DA layer is honest and the data was published.
- P_{exit}: a withdrawal commits only against a chain whose settled and available prefix is the one the withdrawal depends on, up to the receipt tolerance defined below. It is enforced by the circuit breaker.

Table 5 lists which properties hold in each state.

Table 5: Properties by FSM state. P_{agree} holds in every state under $\text{wt}(B) < W/3$. † holds only if the checkpoint commits the chain through its height and the recovery exit gate includes DA backfill (Section 4.7). The prototype implements both; the proof condition of the exit gate is qualified in Section 8.3.

State	P_{agree}	P_{settle}	P_{avail}	**Withdrawals**
ANCHORED	Yes	Yes†	Yes†	Permitted
SUSPICIOUS	Yes	Yes†, delayed	Yes†	Permitted
SOVEREIGN	Yes	No	No (validator block stores only)	Locked
RECOVERING	Yes	Pending recovery proof	Backfilling	Locked

Receipt tolerance. The two receipts are measured on different clocks, so each has its own tolerance, fixed by the round in which the block commits (Section 4.6). τ_{BTC} bounds, in Bitcoin blocks, how far the checkpoint's Bitcoin height may lag the certified Bitcoin tip b_h. τ_{DA} bounds, in Engram blocks, how far the DA receipt's height may lag h. In ANCHORED and SUSPICIOUS, a valid block at height h whose layers are reported healthy carries a Bitcoin receipt that settles Engram height c_s through a checkpoint at Bitcoin height at least $b_h - \tau_{\text{BTC}}$, and a DA receipt that attests availability up to Engram height $c_a \geq h - \tau_{\text{DA}}$.

Theorem 1 (Section 5.1) states the conditions under which a committed withdrawal satisfies P_{exit}. Section 4 defines the mechanisms, and Section 5 states and checks the properties.

4. Engram Protocol

The mechanisms follow the order of the data: the architecture, the sensors that produce observations, the certificate that aggregates them, the state machine that consumes the aggregate, the validity predicate that enforces it, and the recovery path out of SOVEREIGN.

4.1. Architecture Overview

Engram is a Cosmos SDK application [73] running on a CometBFT core. Three peripheral layers are monitored: (i) Bitcoin, used as a settlement and finality anchor via periodic checkpoints; (ii) Celestia [1], used as the DA layer; and (iii) the peer-to-peer interfaces through which validators

reach the Engram, Bitcoin, and Celestia networks. Engram follows one rule throughout: *sensors observe, the quorum certifies, the state machine decides.* Each validator's sensors read the peripheral layers locally and asynchronously, so two honest validators may observe different values at the same instant. Those readings never enter replicated state directly. Instead, each validator signs its reading as a CometBFT vote extension; the next proposer aggregates a quorum of them into a *health certificate* (Section 4.3); and the FSM transition is a deterministic function of that certificate and the previously committed FSM state (Fig. 2).

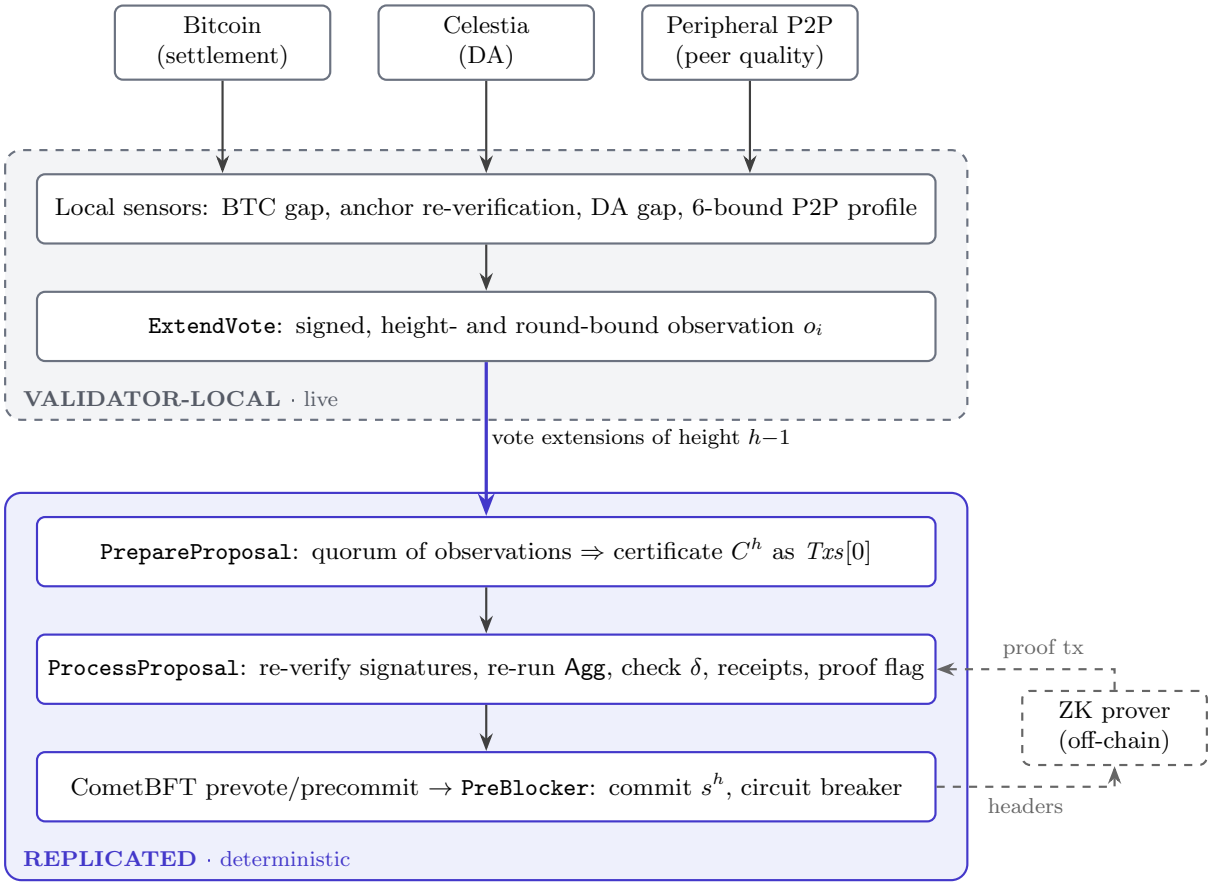


Figure 2: Engram architecture. Live sensor reads stay in the validator-local zone and enter consensus only as signed vote extensions (bold arrow, the single crossing). The replicated zone, from certificate aggregation downward, is a pure function of data every validator holds, so the committed FSM state s^h is deterministic. The dashed path is the recovery proof (Section 4.7).

4.2. Network Sensors

Sensors are the only non-deterministic component; their outputs enter consensus only through the certificate (Section 4.3).

Bitcoin finality gap. $\Delta H_{\text{BTC}} = H_{\text{current}} - \min(H_{\text{submitted}}, H_{\text{anchored}})$ measures how far the latest checkpoint lags Bitcoin's tip; it grows under congestion or checkpoint censorship. Because a delay metric cannot detect a checkpoint that was reorganized away, each validator also re-verifies, against its own Bitcoin full node, that the checkpoint transaction is still included at depth $\geq K_{\text{DEEP_FINALITY}}$. A failed re-verification is a critical condition.

DA gap. $\Delta H_{\text{DA}} = H_{\text{local}} - H_{\text{verified}}$ measures the lag between the chain head and the highest height whose DA submission the validator has confirmed as retrievable from its own Celestia bridge node. This is bridge-level retrievability, not a data-availability-sampling proof; Section 6.10

describes what the prototype publishes.

P2P health. Peer count alone cannot distinguish a long-lived honest peer from a fresh Sybil, and eclipse attacks combine vectors no single metric captures [61, 62]. The profiler combines six bounds – subnet diversity, active anchor peers, clean peers, churn rate, mean peer tenure, and latency – applied to the Engram P2P interface, the Bitcoin client, and the Celestia client. These bounds characterize peripheral and partial P2P degradation; a partition of the core consensus network below a quorum is outside what any FSM can handle (Section 1).

Agreement versus correctness of health. Lemma 1 (Section 4.3) bounds what Byzantine validators can do to the aggregate; it does not make an honest sensor accurate. The six P2P bounds are a heuristic detector: an eclipse that keeps peer count, subnet diversity, tenure, and latency within bounds while controlling a validator's view is not detected by them. The TLA+ model assumes that an eclipsed honest validator reports a degraded reading and never a healthier one (Section 5); that is a sensor-soundness assumption of the model, not a property derived from the six bounds. A detector that is evaded at every honest validator yields a healthy aggregate and no transition; it does not break agreement.

4.3. Health Certificates

The health certificate turns asynchronous, possibly inconsistent local observations into a single value that every validator can recompute.

Observation. At height $h-1$ and round r, validator i produces an observation $o_i = (i, m_i, \sigma_i)$. Here m_i is its local metric vector (Section 4.2) – Bitcoin gap, anchored and current Bitcoin heights, anchor re-verification result, DA gap and verified height, DA sampling and attestation flags, and the six P2P-quality bounds – and σ_i is i's signature over CometBFT's vote-extension sign bytes $(\mathit{chainID}, h-1, r, m_i)$. The extension is produced in `ExtendVote`, the only ABCI++ hook permitted to read live sensors.

Certificate. Let $r^\star$ be the round in which height $h-1$ committed. The proposer of height h receives the extended commit of height $h-1$ and places

$$C^h = \big(h-1,\ r^\star,\ \{o_i\}_{i\in S}\big)$$

as the first transaction of its proposal. C^h is *well-formed* iff (a) every σ_i verifies against the genesis key of validator i for $(h-1, r^\star)$; (b) each validator contributes at most one observation; (c) every $i \in S$ has a commit precommit in round $r^\star$ of height $h-1$; and (d) $\mathrm{wt}(S) > 2W/3$, so S is a quorum.

Aggregation. For a numeric metric j, the aggregate is the smallest value whose cumulative weight exceeds half of the certificate weight; a Boolean metric is true iff the weight backing true exceeds half of it:

$$\begin{aligned} \widehat{m}_j &= \min\big\{x : \mathrm{wt}(\{i \in S : m_{i,j} \le x\}) > \tfrac{1}{2}\,\mathrm{wt}(S)\big\}, \\ \widehat{b}_j &= \mathbf{1}\big[\mathrm{wt}(\{i \in S : b_{i,j} = 1\}) > \tfrac{1}{2}\,\mathrm{wt}(S)\big]. \end{aligned} \tag{2}$$

$\mathsf{Agg}(C^h)$ collects these values. The Boolean metrics are failure flags (failed anchor re-verification, failed DA sampling, failed DA attestation), so a tie leaves the flag unset. Agg is a pure function: every validator that receives C^h computes the same value without contacting any peripheral network. The TLA+ model gives every validator the same voting power (Section 5).

Lemma 1 (honest-bounded aggregate). *Let C^h be well-formed and $\mathrm{wt}(B) < W/3$, where B is the set of Byzantine validators and H the set of the others. For every numeric metric j,*

$$\min_{i\in S\cap H} m_{i,j} \ \le\ \widehat{m}_j \ \le\ \max_{i\in S\cap H} m_{i,j},$$

and every Boolean metric on which all members of $S \cap H$ agree takes that value.

Proof. Write $w = \mathrm{wt}(S)$. By (d), $w > 2W/3$, and $\mathrm{wt}(S \cap B) \le \mathrm{wt}(B) < W/3 < w/2$. Hence $\mathrm{wt}(S \cap H) = w - \mathrm{wt}(S \cap B) > w/2$, and $S \cap H$ is not empty. Let a and b be the smallest and the largest value of metric j among the members of $S \cap H$, and let $F(x) = \mathrm{wt}(\{i \in S : m_{i,j} \le x\})$, so that $\widehat{m}_j$ is the least x with $F(x) > w/2$ (Eq. (2)); such an x exists because F reaches w at the largest reported value. Every member of $S \cap H$ reports at most b, so $F(b) \ge \mathrm{wt}(S \cap H) > w/2$ and $\widehat{m}_j \le b$. For $x < a$ no member of $S \cap H$ reports at most x, so $F(x) \le \mathrm{wt}(S \cap B) < w/2$; no $x < a$ satisfies the condition and $\widehat{m}_j \ge a$. For a Boolean metric on which all members of $S \cap H$ report v: if $v = 1$, the weight backing true is at least $\mathrm{wt}(S \cap H) > w/2$; if $v = 0$, it is at most $\mathrm{wt}(S \cap B) < w/2$. In both cases $\widehat{b}_j = v$. □

This is the range form of median validity [22] for weighted observations. The bound on $\mathrm{wt}(B)$ is the largest the quorum rule supports: the proposer chooses S, and once it omits enough honest observations that w is just above $2W/3$, a coalition of weight just above $W/3$ exceeds $w/2$ and the aggregate can leave the honest range. The unequal-power test of Section 7.1 found such coalitions at 34 or 35 out of 100.

Corollary 1 (threshold decisions). *Under the hypotheses of Lemma 1, let θ be a threshold on metric j. If every member of $S \cap H$ reports $m_{i,j} > \theta$, then $\widehat{m}_j > \theta$; if every member reports $m_{i,j} \le \theta$, then $\widehat{m}_j \le \theta$.* Both follow from $a \le \widehat{m}_j \le b$. A degradation that every honest validator in S observes therefore moves the certified aggregate across the threshold, and a weight below $W/3$ cannot move it across when no honest validator in S observes the degradation.

What the lemma does not give. It bounds each metric separately. The aggregate vector need not equal the observation of any honest validator, so a condition on several metrics can hold of $\mathsf{Agg}(C^h)$ when it holds of no honest observation. For example, with three equally weighted honest validators and three metrics, each validator seeing a different metric beyond its threshold, every aggregated metric is within its threshold although every honest validator sees a degradation. The lemma also does not say which honest value the aggregate takes when honest values differ (see the proposer discussion below).

Lemma 1 does not require honest observations to agree; it bounds what the adversary can do when they disagree. Each adversarial case reduces to it:

- *Stale or replayed observation.* A signature binds the height and round; an observation signed at another height or round fails (a) and is discarded.
- *Conflicting observations.* Two different signed observations by one validator count once by (b); Byzantine weight is below $W/3$ either way.
- *False observations.* Byzantine validators below $W/3$ reporting extreme values cannot move the aggregate outside the honest range (Lemma 1).
- *Proposer omission.* The proposer chooses S, and may omit honest observations. If S still meets (d), Lemma 1 still holds. The proposer's residual influence is choosing *which* honest value in the honest range is selected when honest validators genuinely disagree. With n equal-power validators, all honest, and a quorum of $q = \lfloor 2n/3 \rfloor + 1$ observations, the proposer can reach exactly the values from the $(\lfloor q/2 \rfloor + 1)$-th to the $(n - q + \lfloor q/2 \rfloor + 1)$-th smallest observation: a subset of $s \ge q$ observations has its median at rank $\lfloor s/2 \rfloor + 1$ within it, which is at least $\lfloor s/2 \rfloor + 1$ and at most $n - s + \lfloor s/2 \rfloor + 1$ among all n, and both extremes are reached at $s = q$. For n=7 and q=5 that is the third to the fifth smallest, the median of all seven being the fourth. Only when the honest observations straddle a threshold does this choice change a decision, and a transition needs the hysteresis count over consecutive blocks while proposers rotate in proportion to voting power, so a Byzantine proposer influences the blocks it proposes. The same freedom exists one level

down: CometBFT accepts any `LastCommit` that carries more than two thirds of the voting power and ignores absent signatures, so a proposer also chooses which precommits the block records. Requiring one observation for every commit vote of `LastCommit` would tie the two choices together and make an omission visible in the block, but it would not remove the choice (Section 8.5). If omission drops S to weight at most $2W/3$, the certificate is not well-formed and honest validators reject the proposal. The only exception is the first height after vote extensions are enabled, whose predecessor carries no extensions; there the certificate is replaced by a maximally degraded metric vector, the conservative choice. In the live collusion suite (Section 7, case B3), honest validators accepted a proposer that kept exactly a quorum of certificates in 3 of 3 repeats and rejected one that kept fewer, 210 times in total across 3 repeats.

- *Missing or malformed observations.* CometBFT accepts a precommit's extension only if its signature verifies under the validator's consensus key, and the application's `VerifyVoteExtension` accepts only extensions that decode; signatures are checked again against the genesis keys when the certificate is aggregated. The extended commit for $h-1$ carries weight above $2W/3$, so an honest proposer can always build a well-formed certificate, and a Byzantine validator cannot reduce the usable weight by sending a decodable extension with a bad signature, because CometBFT would not have accepted that vote. This relies on CometBFT and the aggregation using the same key and sign bytes.
- *Divergent honest views.* When honest views straddle a threshold, different proposers may select certificates on different sides of it. Each resulting transition is legal and deterministic given its certificate; the FSM's hysteresis absorbs short alternations (Section 4.5).

Height and round binding. The certificate committed at height h describes observations signed at height $h-1$, one block of lag. A signature binds $(\mathit{chainID}, h-1, r, m_i)$, and a well-formed certificate takes observations only from the commit round $r^\star$ and only from validators whose precommit is in that commit. This prevents replay of an observation from another height or round into C^h. The binding is a property of the consensus instance, not of the measurement: a validator can still sign a cached Bitcoin tip or stale peer metrics (Section 4.2).

4.4. Finite State Machine

The FSM has four states (Table 6). ANCHORED is normal operation: blocks are valid only with a Bitcoin receipt and a DA receipt within tolerance, and withdrawals are permitted. SUSPICIOUS is an early-warning state reached after a sustained warning; withdrawals remain permitted, but admission of forced transactions is rate-limited by a bounded queue. SOVEREIGN is the fallback: the Bitcoin and DA receipt checks are relaxed for each layer that the certificate does not report healthy, blocks continue under the validator quorum, and the circuit breaker locks all cross-chain withdrawals. RECOVERING is entered once the certificate reports healthy peripherals; withdrawals stay locked until the exit gate of Section 4.7 is satisfied.

Fig. 3 shows the transition function δ. A proposal carries a target state s^h; honest validators prevote for it only if

$$s^h = \delta\big(s^{h-1}, \mathsf{Agg}(C^h)\big), \tag{3}$$

where δ also reads the hysteresis counters committed with s^{h-1} (Section 4.5). Since Agg and δ are deterministic and the state and counters are committed, every honest validator reaches the same verdict on a given proposal, and the committed state inherits CometBFT's agreement guarantee. δ evaluates the conditions in a fixed priority: a critical condition first, then a warning (in RECOVERING, any non-healthy reading), then a healthy reading. The first condition that applies decides, so a critical condition overrides every hysteresis counter.

Table 6: FSM states and their validity requirements. The exit gate is defined in Section 4.7.

State	Validity requires	Withdrawals	Forced-tx admission	BTC settlement
ANCHORED	Quorum; BTC and DA receipts	Permitted	Full	Yes (K_{DEEP} conf.)
SUSPICIOUS	Quorum; receipt of each layer reported healthy	Permitted	Rate-limited	Yes, delayed
SOVEREIGN	Quorum; receipt of each layer reported healthy	Locked	Full	No
RECOVERING	Quorum; BTC and DA receipts; exit gate	Locked	Full	Pending proof

CometBFT's deterministic finality is unaffected by the FSM state; the measured commit interval on our testbed is about 1–2 s in every state. Bitcoin settlement is probabilistic and follows Bitcoin's own confirmation time (K_{DEEP} blocks, ∼10 min per block on mainnet); it is not measured in this paper.

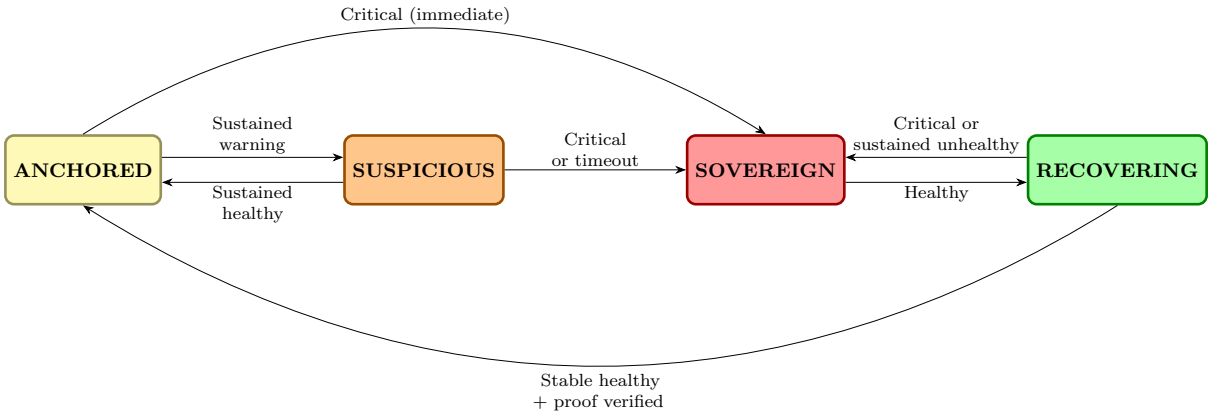


Figure 3: FSM transition function δ. *Sustained* means the condition accumulates over several blocks (hysteresis; up-counters tolerate isolated non-healthy blocks); critical conditions skip the wait and act immediately. The waits are given in Table 7.

4.5. Transitions and Hysteresis

A *warning* (suspicious Bitcoin gap, unhealthy DA, or unhealthy P2P quality) demotes ANCHORED to SUSPICIOUS only after $k_{\downarrow}$ consecutive non-healthy blocks. A *critical* condition (excessive Bitcoin gap, failed anchor re-verification, complete anchor-peer isolation, or a SUSPICIOUS timeout of T_{sus} blocks) moves the chain to SOVEREIGN immediately and bypasses every counter. Recovery is deliberately slower than degradation: SUSPICIOUS→ANCHORED requires $k_{\uparrow}$ consecutive healthy blocks, and RECOVERING→ANCHORED requires the safe-block counter to reach H plus the exit gate of Section 4.7. SOVEREIGN→RECOVERING needs only a healthy certificate.

The down threshold $k_{\downarrow}$ has an optional backoff against an adversary who repeatedly toggles a peripheral. It is an extension that we evaluate in Section 7.3 and do not claim as a contribution: it did not outperform a static threshold of 4 in our live runs. With the cap equal to the initial value $k_{\downarrow}$ is static. With backoff, it starts at a genesis value and doubles per flap level, up to a genesis cap, and both demotion edges (ANCHORED→SUSPICIOUS and the non-critical RECOVERING→SOVEREIGN) use it. The flap level rises by one for each such demotion until the cap is reached, falls by one after T_{stable} healthy ANCHORED blocks, and is not reset by a successful recovery.

The counters follow the committed state. The unhealthy streak counts non-healthy blocks absorbed in ANCHORED or RECOVERING; it resets on any transition and on a healthy block. The safe-block counter in RECOVERING rises on healthy blocks up to H and falls by one on an

Table 7: Hysteresis waits of the FSM (defaults of the prototype; Section 4.5).

Wait	Applies to	Default	Adaptive
Down wait $k_{\downarrow}$	ANCHORED→SUSPICIOUS, RECOVERING→SOVEREIGN	2; with the optional backoff, doubles per flap level up to a cap (8 on the testbed)	Optional
Suspicious exit $k_{\uparrow}$	SUSPICIOUS→ANCHORED	2	No
Recovery H	RECOVERING→ANCHORED	2	No

absorbed non-healthy block; the corresponding SUSPICIOUS counter behaves the same way up to $k_{\uparrow}$. Any other transition resets both. Section 7.3 evaluates all three hysteresis-gated edges.

4.6. Extended Proposal and Validity Predicate

A proposal P_h at height h carries, in $Txs[0]$, the certificate C^h, the target state s^h, a health flag, a DA receipt (last DA-verified height and attestation flag), a Bitcoin receipt (checkpoint height and block hash), and, when applicable, a recovery-proof reference.

Definition 1 (Engram proposal validity). `ProcessProposal` accepts a proposal P_h iff

$$\mathsf{Valid}_h(P_h) \iff \mathsf{WellFormed}(C^h) \wedge \mathsf{StateOK} \wedge \mathsf{ReceiptsOK} \wedge \mathsf{WithdrawalsOK} \wedge \mathsf{RecoveryOK}, \quad (4)$$

where

- $\mathsf{WellFormed}(C^h)$: C^h satisfies conditions (a)–(d) of Section 4.3;
- $\mathsf{StateOK}$: $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$ and the health flag matches $\mathsf{Agg}(C^h)$;
- $\mathsf{ReceiptsOK}$: in ANCHORED and RECOVERING, and for any layer the certificate reports healthy, the DA and Bitcoin receipts are consistent with $\mathsf{Agg}(C^h)$ within the round-dependent tolerances τ_{BTC} and τ_{DA} defined below;
- $\mathsf{WithdrawalsOK}$: no withdrawal is included while s^h is SOVEREIGN or RECOVERING;
- $\mathsf{RecoveryOK}$: the proposal carries a recovery-proof reference with a fresh DA attestation exactly when it targets RECOVERING with the safe-block counter at H, and the exit RECOVERING→ANCHORED is taken only on a proof the chain has verified (Section 4.7).

`ProcessProposal` never reads a live sensor. The FSM state is written to replicated state only by `PreBlocker` after commit, from the certificate already agreed upon.

The tolerances are deterministic functions of the round r and genesis parameters, so every validator computes the same bounds. Each is a base threshold plus a widening w_r that is capped, so no number of round skips enlarges it beyond its maximum. A DA receipt may lag the Engram height by $\tau_{\mathrm{DA}} = \Theta_{\mathrm{DA}} + w_r^{\mathrm{DA}}$ Engram blocks, where Θ_{DA} is the DA threshold and $w_r^{\mathrm{DA}} = 0$ for $r \le 1$, 2 for $r = 2$, and 4 for $r \ge 3$. A Bitcoin receipt's checkpoint height may lag the Bitcoin tip by $\tau_{\mathrm{BTC}} = K_{\mathrm{DEEP_FINALITY}} + 2 + w_r^{\mathrm{BTC}}$ Bitcoin blocks, where $w_r^{\mathrm{BTC}} = 0$ for $r \le 2$ and 1 for $r \ge 3$. The base $K_{\mathrm{DEEP_FINALITY}} + 2$ is the depth a confirmed checkpoint already lags the tip plus two blocks of propagation slack. Theorem 1 takes both tolerances at the round in which B_h committed.

$\mathsf{ReceiptsOK}$ is exactly what the strict receipt-gated baseline (Section 6.4) enforces unconditionally. Engram's only liveness-relevant change is to make it conditional on the certified state: when the certificate reports a layer unhealthy outside ANCHORED and RECOVERING, that layer's receipt is no longer required, and the circuit breaker ($\mathsf{WithdrawalsOK}$) removes the operations whose safety depended on it.

4.7. Recovery Proof and Re-anchoring

Normative protocol. The exit defined here assumes proof-only recovery: it requires a recovery proof that the chain has verified. The prototype also contains an optional engineering heuristic that can satisfy the proof condition; Section 8.3 identifies it.

Statement. On leaving SOVEREIGN, a prover builds a Noir circuit proof (Barretenberg UltraHonk backend [69,74]) over a contiguous range of $n \leq N_{\max} = 256$ headers starting from the last anchored root rt_{last}. A header is the tuple (*prev_hash*, *fsm_state*, *withdrawal_locked*, *state_root*), where *state_root* is CometBFT's application hash for that block. The circuit checks that (a) each header's *prev_hash* equals the Poseidon2 hash of its predecessor, the first links to rt_{last}, and the hash of the last header equals the claimed rt_{new}; (b) every header has *fsm_state* in {SOVEREIGN, RECOVERING} and *withdrawal_locked* set; and (c) the first and last blocks of the range carry ed25519 precommit signatures (round 0) from at least two thirds of the voting power of a validator set whose hash is a public input (the compiled circuit fixes the set size at seven). Unused slots are padding, so proving time, verification time, and proof size are constant in n (Appendix B). In short, the circuit proves header-chain continuity and root binding (a), mode-label and lock-flag consistency (b), and boundary quorum certification (c). It does not check each transition against δ, and the lock flag is the one the chain recorded in the header.

Intended verifier. A CometBFT quorum certificate shows that a quorum signed a block. The recovery proof adds a statement about the interval between two signed blocks: every recorded state in it is SOVEREIGN or RECOVERING with the lock flag set. This matters to a verifier that does not replay the chain, such as a bridge or light client on another chain, a node syncing from genesis, or a Bitcoin observer (Section 3.2). For validators, which already hold the headers, it is a consistency check. The target verifier is an external light client that accepts a re-anchoring checkpoint only with a valid proof; the prototype does not include such a client. As a check that the proof can be verified without the node, a separate process on the same host read a committed block over RPC, took the proof (14,656 bytes) and the 224-byte public inputs from its recovery-proof transaction, and verified them with `bb` against the verification key shipped with the repository; this was one proof on one chain.

Longer intervals. An interval longer than 256 headers is covered by a sequence of proofs: each accepted proof advances the on-chain rt_{last} to its rt_{new}, and the next proof must start from that root. This costs $\lceil L/256 \rceil$ proofs for L blocks. An aggregator circuit can fold up to four chunk proofs into one proof of the same size, which makes verification cost independent of L up to 1024 headers at the price of one extra proof; proving time still grows with L (Appendix C).

Who verifies what. The recovery proof is submitted as a message and verified by *every Engram validator* during block execution, using an embedded, deterministic WebAssembly build of the UltraHonk verifier. The message handler binds the proof's public inputs to chain state:

- rt_{last} to the committed last anchored root, and rt_{new} to the accumulator of a header the chain recorded;
- the first height to the last anchored height plus one, and the header count to the distance between the two checkpoints;
- the first and last block hashes to the CometBFT hashes the chain recorded at those heights, and the validator-set hash to that of the genesis set.

A verified height is latched so a proof cannot be replayed. Interior headers are bound only through the accumulator, not through per-block signatures. Bitcoin only timestamps the checkpoint; it does not verify the proof. After recovery, validators resume writing `OP_RETURN` checkpoints of 44 bytes (a 4-byte tag, the Engram height, and the CometBFT block hash at that height), and the Bitcoin gap sensor requires them to reach depth $K_{\text{DEEP_FINALITY}}$. Anchor re-verification

compares the hash with the local block hash. The proof's rt_{new} is a Poseidon2 accumulator, not a block hash, and is not written to Bitcoin.

Proof timing. The proof message executes in block h and latches the height of the new checkpoint. The `PreBlocker` of block $h{+}1$ recomputes the proof-validity flag from that height, and `ProcessProposal` reads this committed flag when it evaluates the exit edge, so RECOVERING→ANCHORED cannot occur before block $h{+}2$. `ProcessProposal` does not verify the proof itself.

Resulting guarantee. The recovery proof guarantees, to anyone who verifies it, that (i) the recorded headers of the SOVEREIGN interval form a single hash chain between two boundary blocks signed by at least two thirds of the voting power, and (ii) every recorded state is SOVEREIGN or RECOVERING with the lock flag set. Because the lock flag is the chain's own record, (ii) does not prove that no withdrawal executed. The proof does not make the interval's *ordering* secure against a Byzantine supermajority: during SOVEREIGN, safety rests on the validator quorum alone, which is precisely why withdrawals are locked. Re-anchoring restores Bitcoin timestamping from the recovery point forward.

Data availability of the Sovereign interval. A recovery proof shows the interval is recorded consistently, not that its data is *available*. The exit condition is therefore stronger than the proof: RECOVERING→ANCHORED requires (1) a recovery proof that the chain has verified and (2) a DA backfill certificate – a statement, backed by the quorum of signed observations in the health certificate, that the block data for every height of the SOVEREIGN interval has been published to the DA layer and is retrievable. During SOVEREIGN, each validator keeps the interval's blocks in its own store (they are ordinary committed CometBFT blocks) and queues them for publication; on recovery the queue is drained in height order. Each validator reports a contiguous height: the highest height such that every height up to it has been confirmed retrievable. The certificate takes the weighted median $\widehat{d}$ of these reports. Let e be the height at which RECOVERING was entered, and let safe^{h-1} and proof^{h-1} be the committed safe-block counter and proof-validity flag. The exit edge is taken at height h iff

$$\text{healthy}\big(\mathsf{Agg}(C^h)\big) \;\wedge\; \text{safe}^{h-1} = H \;\wedge\; \text{proof}^{h-1} \;\wedge\; \widehat{d} \geq e - 1. \tag{5}$$

The DA gap sensor keeps measuring head liveness, so the DA layer stays healthy while the backfill drains. The specification abstracts the backfill condition as a Boolean, `da_backfill_valid`. The prototype implements the queue and the gate; Section 6.10 describes the remaining live validation.

5. Security and Formal Analysis

This section states the properties that follow from the mechanisms of Section 4 under the assumptions of Section 3, and reports what bounded model checking establishes about them.

5.1. Analytical Properties

Security of mode switching. A transition happens only when a well-formed health certificate implies it (Lemma 1, Section 4.3). Byzantine validators below $W/3$ cannot move a numeric aggregate outside the range of the honest observations in a well-formed certificate, and cannot overturn a Boolean value on which all represented honest validators agree. When honest observations straddle an FSM threshold, the proposer's choice of quorum subset can still influence which side of the threshold the aggregate falls on; hysteresis limits this influence but does not remove it. The certificate therefore guarantees honest-range validity. It is not an unbiased estimate of the state of the peripheral layer, and it carries no information about that

state beyond what the honest validators observed. The second attack surface is griefing by an adversary that degrades the peripheral view of honest validators (Section 8).

Theorem 1 (Withdrawal Admission Safety). *Assume* $\mathrm{wt}(B) < W/3$*, no Bitcoin reorganization deeper than* $K_{DEEP_FINALITY}$*, and an honest DA layer. Assume further that (M1) honest validators reject a block that contains a withdrawal while the state is* Sovereign *or* Recovering*; (M2) each Bitcoin checkpoint commits the chain through its height, and with it the replicated state; and (M3)* Recovering→Anchored *requires a recovery proof that the chain has verified and a DA backfill certificate for the whole* Sovereign *interval. If a withdrawal commits in block* B_h*, then* P_{agree} *holds, the state at* h *is* Anchored *or* Suspicious*, and there are Engram heights* c_s *and* $c_a \geq h - \tau_{DA}$ *such that* $P_{settle}(c_s)$ *and* $P_{avail}(c_a)$ *hold, where the checkpoint that settles* c_s *lags the certified Bitcoin tip by at most* τ_{BTC} *Bitcoin blocks. Every height up to* $\min(c_s, c_a)$ *is settled and available, including every* Sovereign *interval that precedes* B_h*.*

Proof. P_{agree} follows from $\mathrm{wt}(B) < W/3$. By (M1), the state at h is Anchored or Suspicious. In the transition function δ, Suspicious is entered only from Anchored, and Sovereign is left only through Recovering; so every Sovereign interval before h ended with a Recovering→Anchored transition, which by (M3) required a verified recovery proof and a DA backfill certificate. Hence the interval is valid, hash-linked, and available. In the Anchored and Suspicious blocks since then, every block carries receipts within τ_{BTC} and τ_{DA}. The Bitcoin receipt in B_h attests a checkpoint at depth at least $K_{\mathrm{DEEP_FINALITY}}$ for some c_s; by (M2) it commits the chain through c_s, so rewriting the prefix up to c_s requires a deeper reorganization, which is excluded. The DA receipt in B_h attests data availability up to some $c_a \geq h - \tau_{\mathrm{DA}}$, and the DA layer is honest. □

Scope of Theorem 1. The theorem is a safety property of the committed history: it constrains which blocks may contain a withdrawal. It does not bound what a withdrawal depends on. Settlement is guaranteed only through c_s and availability only through c_a, whereas a withdrawal may depend on state created inside the exposure window after them, and the prototype records no dependency height for a withdrawal. Asset-level safety would add the condition that every withdrawal depends only on state at heights up to $\min(c_s, c_a)$; the theorem does not establish it. Liveness comes from relaxing the receipt checks in Sovereign; the property a bridge would need comes from tying withdrawals to the settled and available prefix.

The prototype realizes the three assumptions as follows:

- (M1) is enforced in the validity predicate and the ante handler.
- (M2) holds because each checkpoint payload carries the CometBFT block hash at its height c. That hash commits the header of B_c, hence the chain of blocks through c by the hash links, and the header carries the application hash of the state before B_c. The checkpoint therefore commits that state and the transactions of B_c, and, since execution is deterministic, the state after B_c.
- (M3) is enforced on the exit edge by the backfill condition, which the FSM safety run checks as an action property, and by the proof condition (qualified in Section 8.3).

The theorem is a pen-and-paper argument over these assumptions, not a mechanized statement about the implementation.

5.2. Formal Model

Model-checking consensus, FSM, and sensors together is intractable at useful scale. We therefore specify the protocol in TLA+ [70] as four layers – an abstract consensus object, a Tendermint-style consensus layer, the FSM layer, and a server layer composing them – and check each layer

with TLC and, for bounded safety searches, Apalache [42]. The target property is: under partial synchrony and an f-bounded adversary, no two honest validators commit conflicting blocks or conflicting FSM states, and any such conflict implies double-signing evidence.

Safety versus liveness. We follow the separation of concerns of LiDO [47], which verifies Jolteon in Coq; our tools and our liveness encoding differ (see below). In LiDO, safety is a property of an abstract object, a tree of election, method, and commit nodes, and it transfers to a protocol through a refinement mapping plus a set of certificate messages per node, so that whatever Byzantine validators send acts as if it went through the abstract interface. The premise that makes the abstract object safe is *QuorumOverlap* – any two valid quorums share at least one honest validator. In this paper we call it the linking invariant: together with honest validators' refusal to sign conflicting votes, it prevents two conflicting decisions, and with equal voting power it follows from $n \geq 3f+1$. It is a *safety* premise and does not establish liveness. That the composed server layer actually refines the abstract object, i.e. that the mapping preserves the invariant at every step, is the open obligation C1 below.

Liveness is argued separately. LiDO obtains it by adding a pacemaker to the abstract object and stating the liveness requirements as properties of bounded time segments, under a single GST. Our specification expresses the same kinds of requirement as TLA+ fairness hypotheses that TLC takes as given. It models GST as in partial synchrony: a predicate `AfterGST` on the model's real time, which never decreases, so that once GST has passed it stays passed. The predicate concerns the network only; it does not mention the FSM state or the peripheral layers, so the liveness claim covers every FSM state, SOVEREIGN included. The liveness argument has two levels. The consensus layer relies on the first three hypotheses below, which correspond to LiDO's pacemaker and leader requirements; the fourth, FSM progress, concerns the FSM layer and has no counterpart in LiDO. The four hypotheses are:

- *Partial synchrony:* liveness is claimed for behaviours in which GST is reached (`(<>AfterGST) ~> decided`), and messages between honest validators are delivered (weak fairness on each node's `Push` and `Pull`). Weak fairness constrains only infinite behaviours, so any finite delay before GST is allowed.
- *Timeout progress:* weak fairness on round-timeout expiry and on the $f+1$ round-skip (fast-forward) action.
- *Proposer progress:* weak fairness on each honest node's invocation step, so every honest proposer whose round is reached after GST eventually proposes.
- *FSM progress:* weak fairness on the FSM transition and on environment updates, so a persistently healthy (or unhealthy) certificate eventually drives the corresponding transition.

Under these hypotheses TLC checks the FSM liveness properties directly. A TLC ablation that removes the $f+1$ fast-forward, which would show the timeout-progress assumption is necessary, has not completed: its best run reached depth 12 (69,408,583 states generated, 828,673 distinct) with no violation. The necessity of that assumption is therefore not established here.

5.3. Verification Scope

What is checked. The evidence has two levels.

- *Per-layer bounded model checking (exhaustive within each stated bounded configuration).* Each layer's own safety invariants and liveness properties are checked in isolation at its own bounded configuration (Table 8), together with the ablation study (Table 9).
- *Cross-layer refinement (open).* The obligation that the composed server layer refines the abstract consensus object – which is what would transfer the target property to the

composed system – is specified at three scales (C1: $N{=}4, f{=}1$; C2: deeper rounds; C3: $N{=}7, f{=}2$) but has not converged at any of them. The composed system is therefore *not* formally verified end-to-end; its whole-system evidence in this paper is empirical (Section 7).

To make C1 tractable we split it by which actions may fire and check each part with TLC under symmetry reduction over the three honest validators. With the consensus core frozen, the environment part (sensor updates, clock ticks, DA backfill) completed: 17,500 distinct states, depth 10, 10 m 35 s, no violation. With the environment frozen, the consensus-core part did not complete: it was stopped after 1 h 41 m at 19,848,141 distinct states (depth 15, 13,542,792 states still queued) with no violation. Neither part covers the interleaving of environment and consensus steps, and no composition argument is established, so C1 is not claimed.

The server layer carries two guards that stop it from certifying a timeout for a round an honest validator has already left, and one added abstract action for a round ended by a quorum of nil precommits. These are changes to the checked model, and the last weakens the target specification. An inductive check of every server action with Apalache would avoid the composition gap; it has not been attempted.

Health certificates in the model. The server layer models the certificate. Each validator's observation holds three Booleans (healthy, warning, critical) and the Bitcoin and DA gaps clipped to 0..3, written to `sensor_view` by `ServerExtendVote`. A Byzantine validator may sign any observation; an honest validator with an eclipsed P2P layer may report a degraded one, never a healthier one, which is a sensor-soundness assumption of the model. Every validator has the same voting power. The proposer may choose any member set that forms a quorum. The aggregate (`AggregateHealthCertificate`) is a per-field majority for the Booleans and a weighted median for the gaps, and `IsValidProposal` rejects a certificate below a quorum.

Lemma 1 is checked as `AggWithinHonestRange` with Apalache, with the observation vector left symbolic so that one check covers every vector. For $N{=}4$ and $f{=}1$ it reported no violation (22 s). With two Byzantine validators in the same network it reports a violation (25 s), so the bound on Byzantine members is necessary. Signatures, height and round binding, and the individual P2P counters are not modelled. The check covers the aggregate as a function; the certificate's role inside `IsValidProposal` is covered only by the open refinement obligation.

5.4. Safety Invariants and Attack Resilience Lemmas

Six state invariants are specified and model-checked.

- *Circuit Breaker Isolation:* withdrawals are locked if and only if the state is SOVEREIGN or RECOVERING.
- *Hysteresis Integrity:* RECOVERING→ANCHORED requires the full hysteresis wait and a valid recovery proof.
- *Suspicious-Exit Hysteresis Integrity:* the corresponding guard on SUSPICIOUS→ANCHORED.
- *Strict FSM Transition Safety:* only transitions drawn in Fig. 3 occur.
- *Monotonicity Safety:* heights and time never decrease.
- *Sovereign-Data Publication:* every RECOVERING→ANCHORED transition occurs with the DA backfill condition true. This is an action property, checked in the FSM safety run; the Server-layer drivers have not been re-run with it.

Five attack-specific properties are enforced by the validity predicate: resistance to data withholding; resistance to forged or reorganized anchors through anchor re-verification; a deterministic

bound on Byzantine timeout flooding; resistance to eclipse-driven recovery through the six-bound P2P gate; and accountability, i.e. any safety violation implies double-signing evidence. Accountability is inherited from CometBFT's equivocation evidence pool rather than proved anew. All of these properties are conditional on the Bitcoin reorg-depth bound of Section 3.1.

5.5. Bounded Results and Mechanism Ablation

Table 8: Per-layer, non-refinement verification results (TLC)

Layer / property	States gen.	Distinct	Depth	Time	Workers	Viol.
Tendermint (safety)	3,772,809	537,536	25	1 m 01 s	10	0
FSM (safety)	1,371,377,696	4,542,271	23	10 m 19 s	10	0
FSM (liveness)	17,986,105	31,105	23	3 m 45 s	10	0

Each row is one completed TLC run of that layer's driver. Depth is the number of breadth-first levels at which the search terminated with every state satisfying the state constraint explored, so it is the diameter of the constrained state graph and not a cap. Hardware: Apple M1 Pro, 10 cores, 32 GB RAM.

Table 8 reports the per-layer results, each obtained by running that layer's own driver standalone at the configuration of Table 10. All three runs found zero violations in their bounded state spaces. They cover the Tendermint-style and FSM layers; the abstract and server layers are exercised only through the open refinement obligation. The ablation study spans two layers: the hysteresis and P2P health gate belong to the FSM layer, the circuit breaker and DA consistency to the Tendermint-layer driver, so each result holds within its own layer's configuration.

Table 9: Ablation study: removing one mechanism at a time (Apalache)

Removed	Targeted threat	Depth	Time	Violated property
Circuit breaker	Withdrawal while locked	2	66.7 s	`Sanity_NeverAttempt WithdrawalWhileLocked`
Hysteresis	Premature re-anchoring	6	117.0 s	`Sanity_NoIllegal HysteresisExit`
P2P health gate	False recovery under eclipse	3	24.2 s	`Sanity_NoIllegal P2PRecovery`
DA consistency	Data withholding	2	63.3 s	`Sanity_Never ProposeWithheldData`

Table 9 reports the ablation study. Each row removes one mechanism and re-runs verification against the sanity property that mechanism protects. Apalache's bounded SMT search found each counterexample in 24–117 s at depth 2–6. Each ablation reproduced the violation its mechanism is designed to prevent: without the circuit breaker a withdrawal can be proposed in a locked state; without hysteresis the chain re-anchors prematurely; without the P2P gate an eclipsed validator forces an illegal recovery; and without DA consistency a withholding proposal is accepted. Single-factor ablation does not test joint removal, so it does not establish that the set of mechanisms is sufficient.

6. Implementation and Evaluation Methodology

6.1. Implementation

The reference implementation is a Cosmos SDK application, `engramd`, on a forked CometBFT core. The application mounts no `x/auth`, `x/bank`, or `x/staking`: transactions are unsigned envelopes and the validator set comes from the genesis file. Three modules hold the protocol logic: `x/sovereignty` (the FSM, the sensors, the circuit breaker, and the ABCI++ hooks), `x/da` (the Celestia receipt and its verification), and `x/anchor` (the Bitcoin settlement receipt

Table 10: TLC run configurations

Run	Constants and bounds
Tendermint (safety)	N=4, f=1 (three honest, one Byzantine node); MaxRound 2; MaxBTCHeight 10; MaxEngramHeight 10; MaxTimestamp 2; symmetry reduction over the three honest nodes
FSM (safety, liveness)	SuspiciousThreshold 2; SovereignThreshold 4; DAThreshold 2; RecoveryHysteresisWait 2; SuspiciousHysteresisWait 2; MaxSuspiciousTime 2; DownHysteresisThreshold 2; MaxDownHysteresisThreshold 8; MinPeers 3; MinSubnetDiversity 2; MinAnchorPeers 1; MaxChurnRate 10; MinAvgTenure 100; MaxPeerLatency 50; the Bitcoin and Engram heights stay below 10 (safety run); no symmetry reduction

and its re-verification). The ABCI++ hooks follow the pipeline of Section 4.1: only `ExtendVote` reads live sensors, and `PreBlocker` commits the FSM transition from the agreed certificate.

The fork differs from upstream CometBFT in four ways: an $f+1$ timeout round-skip, per-peer round-trip-time measurement and a peer health monitor that feed the P2P sensor, per-peer rate limiting and bounded round acceptance for timeout messages, and the proposal round passed to the application through `PrepareProposal` and `ProcessProposal`.

Bitcoin and Celestia are live dependencies, deployed as described in Section 6.2. The recovery proof is a Noir circuit proved with Barretenberg (Section 4.7); a prover process watches for a SOVEREIGN backlog, builds the proof from committed headers, and submits it as a transaction that every validator verifies during block execution. Section 6.3 lists the versions.

6.2. Testbed

The testbed runs the reference implementation of Section 6.1 as seven validators with equal voting power (n=7, f=2), each in its own container. Validator gossip uses a dedicated full mesh of pairwise links, separate from the network that carries Bitcoin, Celestia, and RPC traffic; this separation lets us degrade peripheral connectivity without partitioning the core consensus network. The peripheral layers are real software, not simulators: a two-node Bitcoin regtest network mined continuously at roughly one block per 20 s (thirty times faster than Bitcoin's 600 s average; the thresholds are in Bitcoin blocks, so detection times measured here convert to mainnet as in Section 8), and a Celestia network with seven bridge nodes, one per validator, so that each validator forms its own DA view. The single-actor attacks of Appendix A and the f=2 collusion suite of Section 7.1 ran on the seven-validator cluster.

6.3. Environments and Software Versions

All results come from one environment, Testbed A: the seven-validator Docker deployment above on a single host. Every table and figure in Section 7 is a Testbed A measurement. Table 11 lists the configuration.

6.4. Baselines

Strict receipt-gated baseline (primary, RQ2). The same binary with the FSM pinned to ANCHORED (the command-line mode is called `hard-gated`). Condition (3) of the validity predicate (Section 4.6) is enforced at every height, so a proposal is valid only if it satisfies the agreement checks, carries a Bitcoin receipt within tolerance, and carries a DA receipt within tolerance: $V_h(B) = V_{\mathrm{BFT}}(B) \wedge R_{\mathrm{BTC}}(B,h) \wedge R_{\mathrm{DA}}(B,h)$, with no other validity path while a receipt predicate cannot be satisfied. It is the behavior of a chain that makes settlement and DA inclusion part of block validity, it is a construct of this paper and not a published system, and

Table 11: Testbed configuration and software versions

Component	Testbed A
Host	Apple M1 Pro (10 cores: 8P + 2E), 32 GB RAM, 1 TB SSD; macOS 27.0
Container runtime	Docker 29.0.1; VM allocation 10 CPUs, 23.4 GiB RAM
Validators (7)	No CPU or memory limit per container
Consensus core	CometBFT v0.39.3 module path, replaced by the fork `engram-consensus-core` at commit `493a20121`
Application	Cosmos SDK v0.54.3; this repository at [PENDING: results commit]
Bitcoin	Bitcoin Core v25.0, regtest, two nodes (0.3 CPU, 512 MB each)
Celestia	celestia-app v1.11.0 (0.5 CPU, 1 GB); seven celestia-node v0.14.1 bridges (0.3 CPU, 512 MB each)
Prover	nargo 1.0.0-beta.22, `bb` 5.0.0-nightly.20260522 (6 CPU, 8 GB limit)
Fault injection	Pumba (image tag `latest`) driving kernel `tc netem`
Formal tools	TLC 2026.04.18 (rev. c45c125); Apalache v0.58.3 on JDK 17

CPU and memory figures are container limits. The fork's differences from upstream CometBFT are listed in Section 6.1.

it isolates the effect of Engram's adaptive relaxation. It is a runtime mode of the same binary (`ENGRAM_MODE=hard-gated`); Section 7.2 runs the RQ2 fault schedule against it.

What the predicate implies, and what RQ2 measures. If, over an interval, some receipt predicate holds for no proposal at any height, then no value satisfying V_h can be decided in that interval. This is the external-validity property of Byzantine agreement [55], and in CometBFT an application rejection in `ProcessProposal` yields a nil prevote. It is a consequence of the predicate, not a result of this paper. RQ2 therefore asks different questions: how many blocks the implementation commits while the predicate is nearly unsatisfiable (the receipt tolerance widens with the round), whether it resumes after the fault, and what Engram commits instead. The first two depend on the implementation and are reported separately.

Soft baseline (RQ2). The real FSM runs, but withdrawals are never locked in SOVEREIGN or RECOVERING (`ENGRAM_MODE=soft`). It keeps liveness and drops the circuit breaker, which isolates what the withdrawal lock contributes.

Vanilla CometBFT (control, RQ4). The same application with the health-aware ABCI++ hooks disabled. It does not depend on Bitcoin or Celestia and therefore never stalls when they fail; it serves as the cost reference for RQ4 and as a control showing that measured stalls or fallbacks are not artifacts of the harness.

Ablation baselines. A peer-count-only P2P detector isolates the contribution of the six-bound profiler (RQ1), and each hysteresis edge with its threshold set to its floor isolates the contribution of hysteresis (RQ3).

6.5. Fault Injection

Network-level faults use kernel traffic control (`tc netem` via Pumba): uniform delay and jitter, packet loss, single-node isolation, and a heterogeneous per-validator wide-area profile approximating seven regions. Appendix E lists the parameters of every profile. The *WAN latency baseline* is the per-validator delay and jitter profile of Table 36, held for the whole run unless a step in Table 12 suspends it.

Application-level faults reproduce mechanisms that delay alone cannot, and each simulates the consequence of a failure rather than its cause.

- *Bitcoin congestion* suspends the submission of new checkpoints (a pending one still confirms), so the finality gap grows directly. It does not congest the Bitcoin mempool.

- *DA outage* stops one or all seven Celestia bridge containers. It does not emulate a data-availability-sampling failure.
- *Byzantine behavior* is injected through attacker processes or through a validator configured to forge proposal fields, omit transactions, double-sign, or flood timeout messages.

Every fault targets a peripheral layer or a minority of validators; none partitions the core consensus network below a quorum, consistent with the scope defined in Section 1.

6.6. Fault Schedules

Table 12 gives the onset, mechanism, and duration of each fault in the two scripted runs of Section 7.2. Times are relative to the start of each run, durations are the length of the polling window the script holds the fault for, and each of S2–S5 is followed by a wait of up to 90 s for the validators to return to ANCHORED before the next scenario starts.

Table 12: Fault schedules of the scripted runs

Step	Fault and mechanism	Duration
Composite run (standing WAN latency baseline)		
C1	Baseline, no fault	15 s
C2	Bitcoin fault: 500 ± 100 ms RPC delay on one `bitcoind` (profile length 2 min)	60 s
C3	All seven Celestia bridges stopped, added to C2	60 s
C4	Three cycles of 5% packet loss on validators 1 and 2 (WAN baseline suspended)	$3 \times$ (20 s on, 10 s off)
C5	Hold; bridges still stopped	30 s
C6	Heal: Bitcoin profile cleaned up, bridges restarted	30 s
C7	Wait for ANCHORED (limit 600 s)	until recovery
Seven-scenario sweep		
S1	Healthy baseline	15 s
S2	Checkpoint submission suspended on all validators	180 s
S3	All seven Celestia bridges stopped	60 s
S4	300 ± 50 ms delay on the six validator links of validator 1	120 s
S5	100% packet loss on validator 1	180 s
S6	One `bitcoind` and all seven bridges stopped	90 s
S7	Faults removed; wait for ANCHORED (limit 600 s)	until recovery

6.7. FSM Parameters

FSM parameters are fixed in the genesis file, so every validator holds identical values. All live experiments use the defaults of Table 13 unless a section states an override. Sensors are sampled once per block, when a validator signs its vote extension; the measurement scripts poll each validator's RPC every 3 s (2 s during the packet-loss cycles of C4). Blocks arrive about once per second.

6.8. Synthetic Workloads

Noise profile (in-process). The hysteresis sweeps drive the real state machine with sensor readings generated per block. Each block is disturbed independently with probability 0.2 and healthy otherwise, for 100 blocks, from a fixed pseudo-random seed (7 for the RECOVERING→ANCHORED sweep, 11 for the ANCHORED→SUSPICIOUS sweep), so every threshold value sees the same noise sequence. A disturbance sets sensors to a failing reading for one block. Critical disturbances set the Bitcoin gap to `SovereignThreshold`. Warning-level disturbances set the Bitcoin gap to `SuspiciousThreshold`, mark DA unavailable, or give the P2P sensor a snapshot with no clean

Table 13: FSM parameters used in the live experiments

Parameter	Value	Unit and role
`SuspiciousThreshold`	5	Bitcoin gap, blocks
`SovereignThreshold`	8	Bitcoin gap, blocks
`DAThreshold`	30	DA gap, blocks
`KDeepFinality`	2	Reorg depth, Bitcoin blocks
`MinPeers`	3	Clean peers
`MinSubnetDiversity`	2	Distinct subnets
`MinAnchorPeers`	2	Active anchor peers
`MaxChurnRate`	5	Peer churn events
`MinAvgTenure`	300	Mean peer tenure, s
`MaxPeerLatency`	200	Round-trip time, ms
`RecoveryHysteresisWait`	2	RECOVERING→ANCHORED, blocks
`DownHysteresisThreshold`	2	ANCHORED→SUSPICIOUS, blocks
`MaxDownHysteresisThreshold`	8	Cap on the backed-off threshold
`MaxUnprovenTailBlocks`	128	Tail after the proved checkpoint that still counts (default 8)
`ProofOnlyRecovery`	true	Recovery requires a verified proof (default false)
`StableWindow`	16	Healthy ANCHORED blocks that lower the flap level by one
`SuspiciousHysteresisWait`	2	SUSPICIOUS→ANCHORED, blocks
`MaxSuspiciousTime`	24	SUSPICIOUS escalation bound, blocks

Overrides: the Docker testbed runs the table's `DownHysteresisThreshold` of 2 with the backoff cap of 8, so the optional backoff of the down threshold is enabled, and sets `SuspiciousHysteresisWait` to 8, which is not adaptive. The hysteresis sweeps of Section 7.3 vary the down threshold, a static value being `MaxDownHysteresisThreshold` equal to `DownHysteresisThreshold`; the Monte Carlo study of Section 7.1 uses the P2P thresholds of Section 6.8.

peers, diversity, or tenure and only the minimum anchor count. The combined environments apply several at once.

Monte Carlo P2P study. A *snapshot* is one synthetic tuple of the six P2P sensor fields (subnet diversity, active anchors, clean peers, churn rate, mean tenure, latency). A benign snapshot draws each field inside its healthy range with a uniform random margin. An attack of *intensity* $x \in [0, 1]$ perturbs the fields it targets in proportion to x (for example, the relay attack adds x (`MaxPeerLatency`+150) ms of latency and leaves the other fields benign); $x = 0$ is benign and $x = 1$ is the full attack. Detectors are the six-bound profiler and the peer-count-only rule CleanPeers $\geq$ `MinPeers`. The study uses production-scale P2P thresholds: 8 subnets, 4 anchors, 20 clean peers, churn 5, tenure 300 s, latency 200 ms. Four measurements per cell follow, all from fixed seeds.

- *False-positive rate:* the share of 2,000 benign snapshots flagged unhealthy.
- *False-negative rate:* the share of 2,000 full-intensity snapshots flagged healthy.
- *Detection delay:* over 500 ramps, the number of snapshots until the first flag, with intensity rising $0.1, 0.2, \ldots, 1.0$ over 10 snapshots. The mean is taken over ramps that were detected.
- *Evasion threshold:* the largest intensity (on the 0–1 scale) at which a majority of 20 probes still read healthy, found by a 12-step binary search that queries the detector, so it bounds an adversary with oracle access.

The ramp-shape study uses 300 ramps per shape and the three shapes x (linear), $\sqrt{x}$ (front-loaded), and x^2 (slow). The upper 95% confidence bound on a rate observed as 0 in n trials is about $3/n$ (rule of three): 0.15% for $n = 2{,}000$ and 1% for $n = 300$. The detection delay is reported as the mean and the sample standard deviation over the 500 ramps of each cell. The study has three limits. The benign distribution, the attack directions and the intensity

scale are our choices. Each attack perturbs only the fields it targets, along one direction. The evasion threshold varies one scalar, the intensity, by binary search against the detector, so the six dimensions are not searched jointly. The results therefore characterize the detectors' sensitivity within this generator.

6.9. Metrics

Terms. n is the number of validators (n=7 on the main testbed). A *run* is one execution of a scenario; unless stated otherwise it starts from ANCHORED on a freshly deployed chain, so runs are independent deployments, and intervals over runs are intervals over deployments. A *repeat* is a further measurement on the chain of an earlier one (the repeats of Table 24 share a deployment, so their interval describes one chain). An *arm* is one system configuration that a table compares with the others. The number of runs is written out wherever it matters.

Let a run record the FSM state s_b of every block b in a window W.

- *Anchored uptime* $= |\{b \in W : s_b = \text{ANCHORED}\}|/|W|$. In-process, W is the whole recorded timeline: for the noisy environments, the 100 noisy blocks plus the setup blocks that precede them (two for the RECOVERING→ANCHORED sweep, one for the others). The no-noise control of that sweep ends at the first ANCHORED block, so its window is only the setup blocks plus the wait, and its uptime falls as the wait grows. Live, W is the sequence of RPC samples of one validator over a 300 s window.
- *Flapping count* is the number of transitions that immediately reverse the preceding transition, for example ANCHORED→SUSPICIOUS followed by SUSPICIOUS→ANCHORED, over the same W. Live runs detect reversals between consecutive samples, so a reversal faster than the polling interval is not seen.
- *Absorption rate* is $a/(a+d)$, where a is the number of disturbed blocks that begin in ANCHORED and leave the state unchanged, and d is the number that move it to SUSPICIOUS. It is defined only on the ANCHORED→SUSPICIOUS and SUSPICIOUS→ANCHORED sweeps.
- *Blocks/s* $= (h_{\max} - h_{\min})/T$ over a measurement window of length T (60 s after an 8 s warm-up in Section 7.4). *Steady blocks/s* $= 1/\bar{\imath}$, where $\bar{\imath}$ is the mean block interval. *Gap fraction* $= 1 - n\bar{\imath}/T$, where n is the number of observed intervals: the share of the window not accounted for by intervals at the steady pace, that is, time spent in heights that took longer than one interval.

6.10. Sim-to-Real Gap

Four properties of the testbed limit how its results transfer to a production deployment.

- *Single physical host.* All Testbed A containers share one machine's CPU and memory, so resource contention can affect every validator at once; wide-area conditions are emulated with `tc netem` rather than real geographic distribution.
- *Accelerated Bitcoin.* Regtest blocks arrive about 30× faster than on mainnet. Height-based thresholds are unaffected, but wall-clock durations scale accordingly.
- *DA payload.* The DA publisher submits the transactions of committed blocks in height-ordered batches of at most 1.5 MB and tracks the highest contiguous published height; the exit gate of Section 4.7 requires that height to cover the SOVEREIGN interval. The gate rests on a quorum-reported contiguous height, not on a cryptographic proof of retrievability. Section 7.3 reports the republication of the interval after a live DA outage.

- *Single-bridge trust.* Each validator trusts its own bridge node, which downloads and verifies full Celestia blocks, rather than sampling across independent peers; our DA-outage results measure bridge failures, not sampling-detected partial withholding.

7. Evaluation

Each research question is answered by the live experiments that bear on it most directly; the remaining attack scenarios, the prover-backend comparison, and the Bitcoin-reorg boundary test appear in Appendices A–D. Unless stated otherwise, all numbers are live measurements on Testbed A, the seven-validator testbed of Section 6 (Table 11); FSM parameters are those of Table 13, and metrics follow Section 6.9.

7.1. RQ1: Safety Under Byzantine Behavior

By safety we mean agreement among honest validators (identical FSM state and AppHash, aggregate within the honest range, forged or stale receipts rejected) and the lock invariant on committed blocks: no withdrawal-marked transaction commits while the state is SOVEREIGN or RECOVERING. We do not claim that cross-chain assets are protected; the lock acts only once the FSM has reached a locked state (Section 7.2), and the prototype has no bridge (Section 8).

Single Byzantine proposer. The most direct attack on the extended consensus object is a proposer that claims an FSM state not implied by the certificate. One of the seven validators (n=7, 14.3% of the voting power) was configured to do so. Every honest validator recomputed $\delta(s^{h-1}, \mathsf{Agg}(C^h))$, rejected the forged claim in `ProcessProposal`, and the chain committed only honest proposals, with zero AppHash divergence. The same outcome held for the nine further single-actor attacks of Appendix A (eclipse, Sybil, data withholding, forged Bitcoin receipt, withdrawal in SOVEREIGN, censorship, a combined attack, double-signing, and timeout flooding).

Colluding Byzantine validators (f=2). The threat model allows f=2 colluding validators at n=7. We therefore define a collusion suite in which two validators act jointly: (B1) both report coordinated false health observations; (B2) one reports false observations while the other, as proposer, builds its certificate to favor them; (B3) the proposer omits honest observations, both down to the smallest quorum (five of seven validators) and below it; (B4) both double-sign at the same height; (B5) both attempt to include a withdrawal while in SOVEREIGN; and (B6) the proposer attaches a stale or forged Bitcoin or DA receipt. AppHash agreement is necessary but not sufficient evidence of safety, since honest validators could agree on an illegal transition. For every committed height we therefore also replay $\delta(s^{h-1}, \mathsf{Agg}(C^h))$ offline from the committed certificate and check that it equals the committed s^h, that the aggregate lies within the honest range of Lemma 1, and that no withdrawal commits while $s^h \in \{\text{SOVEREIGN}, \text{RECOVERING}\}$. We repeated each case three times in succession on the 7-validator cluster, every repeat starting from ANCHORED (B5 starts from SOVEREIGN by design). The repeats of a case share one chain, so they are not independent deployments. Table 14 lists the cases. In all 33 repeats of cases A3–B7, the honest validators committed identical AppHashes and the offline replay found no block whose committed state differs from $\delta(s^{h-1}, \mathsf{Agg}(C^h))$, no aggregate outside the honest range, and no withdrawal committed in a locked state. Honest validators rejected the forged Bitcoin hash (A4: 216 rejections in total), the stale or forged receipts (B6), the withdrawals in a locked state (B5: 320), and the proposer that kept fewer than a quorum of certificates (B3b: 210); B3a, where the proposer keeps exactly a quorum, was accepted. In B4 the honest validators produced double-signing evidence for both validators, within two blocks of the second signature at the longest. B7 repeats B1 while honest validators are actually eclipsed, so that part of the honest observations are themselves degraded: with two eclipsed (B7a) the chain behaved as in the other cases, and with three (B7b) it reached SOVEREIGN in all three repeats,

with every aggregate still inside the honest range.

Not every case is rejected, and safety does not rest on rejection in the others. In B1, B2, B3a and B7 the blocks are well-formed and the reports they carry are opinions rather than checkable facts; there safety rests on the aggregation. With at most two colluding validators the weighted median cannot leave the range of the honest observations (Lemma 1), and the replay found no block outside that range. The sweep of Table 15 shows where this stops: with b=3 the aggregate leaves the range and the adversary forces SOVEREIGN. A3 is accepted because a DA attestation one height ahead lies inside the tolerance of the receipt check, a limit of the DA trust assumption rather than of the aggregation. B4 is not rejected either, since each vote is valid by itself; the honest validators record the double signature as evidence, and with two double-signers the repeats committed identical AppHashes and replayed cleanly.

State transitions cannot be attributed to the adversary in this setting. The control c0, in which no validator misbehaves and one is only restarted, also moved from ANCHORED to SUSPICIOUS in 3 of 3 repeats, because of ordinary sensor noise on the testbed. We therefore judge the cases by replay, AppHash agreement, and the honest-range check, not by whether a transition occurred.

Table 14: Collusion suite at n=7 (A3 and A4 are the single-actor cases it reruns; B8 is one run of an honest split view with no adversary; c0 is the control)

Case	Byz.	Behaviour	Rejected by honest validators	Repeats	Same AppHash	Replay clean
a3	1	claims a DA attestation one height ahead	none	3	3/3	3/3
a4	1	forged Bitcoin checkpoint hash	btc verify receipt 216	3	3/3	3/3
b1	2	coordinated false degraded observations	none	3	3/3	3/3
b2	2	false observations; proposer biases the certificate	none	3	3/3	3/3
b3a	1	proposer keeps exactly a quorum of certificates	none	3	3/3	3/3
b3b	1	proposer keeps fewer than a quorum	sub quorum certificate 210	3	3/3	3/3
b4	2	double-signing at the same height	none	3	3/3	3/3
b5	2	withdrawal in a block of a locked state	withdrawal locked 320	3	3/3	3/3
b6	2	stale DA receipt; forged Bitcoin hash	btc verify receipt 155, da verify receipt 165	3	3/3	3/3
b7a	2	false observations with two honest validators eclipsed	none	3	3/3	3/3
b7b	2	false observations with three honest validators eclipsed	none	3	3/3	3/3
b8	0	honest split view: 3 then 4 of 7 eclipsed	none	1	1/1	1/1
c0	0	control: node04 restarted, no behaviour	none	3	3/3	3/3

The suite fixes the number of Byzantine validators; the threshold itself is measured in a second sweep (Table 15) with $b \in \{0, 1, 2, 3\}$ validators and two adversary goals: forcing SOVEREIGN through false degraded observations, and hiding a real DA outage by reporting it healthy. Each cell is five runs, each on a fresh chain started from ANCHORED. For $b \leq 2$ no block lay outside the honest range and the adversary reached neither goal (30 of 30 runs). For b=3 the aggregate left the honest range in 40–42 blocks and the adversary forced SOVEREIGN in 5 of 5 runs, which is the boundary Lemma 1 predicts. Hiding an outage did not succeed at b=3 either: honest proposers' blocks still reported it (5–12 blocks outside the range). An outage can be hidden only when every run of consecutive honest proposers is shorter than the hysteresis threshold, and these placements did not produce that.

Table 15: Threshold sweep: Byzantine validators b at n=7 (f=2)

Goal of the adversary	Byz.	Runs	Blocks outside honest range	Same AppHash	Adversary's goal reached
force SOVEREIGN	0	5	0	5/5	0/5
force SOVEREIGN	1	5	0	5/5	0/5
force SOVEREIGN	2	5	0	5/5	0/5
force SOVEREIGN	3	5	40–42	5/5	5/5
hide a DA outage	0	5	0	5/5	0/5
hide a DA outage	1	5	0	5/5	0/5
hide a DA outage	2	5	0	5/5	0/5
hide a DA outage	3	5	5–12	5/5	0/5

Unequal voting power. The testbed and the TLA+ model give every validator the same power, where weight and count coincide, so a defect that confuses them would not appear in any run above. We therefore tested the aggregation function alone (`tests/weighted`) on three

weight vectors that sum to 100, $[30, 20, 15, 10, 10, 10, 5]$, $[26, 19, 17, 13, 11, 8, 6]$ and $[34, 33, 33]$, and on seven equal weights. Against a reference weighted median (the element at position $\lfloor T/2 \rfloor$ of the multiset in which each value is repeated by its power) the function agreed on 1,500 random quorum subsets with random values for every vector, and a verified power of exactly $\lfloor 2W/3 \rfloor + 1$ was accepted and one below it rejected for every subset. For every Byzantine coalition and every set of certificates the proposer may omit while keeping a quorum, with the adversary reporting the lowest value, the highest, or a split, the aggregate stayed within the honest values and honest booleans were not flipped whenever the coalition's power was below $W/3$ (up to 30 of 100 for the first vector and 33 for the other two). A coalition of 35 (20 and 15) in the first vector, and of 34 in the other two, escaped the honest range by having the proposer omit enough honest certificates, so the bound is tight. This checks the implementation of the weighted median and the quorum, not the integration with the genesis power assignment, which a live run with unequal power would add.

Proposer influence. For n from 4 to 10 equal-power validators and 30 random sets of distinct honest values each, we ran the aggregation on every subset the proposer may choose (all subsets of at least $\lfloor 2n/3 \rfloor + 1$ observations). The smallest and largest aggregate matched the order statistics of Section 4.3 in every case.

Peripheral P2P quorum gate. A P2P detector is useful for safety only if its signal cannot be abused by a minority. We first compare the six-bound profiler (Section 4.2) with a peer-count-only detector in a Monte Carlo sensitivity study on synthetic attacks (2,000 benign and 2,000 attack snapshots plus 500 ramps per cell; Section 6.8), then run the two most diagnostic attacks live.

Table 16: P2P detector accuracy: six-bound profiler against peer-count-only (synthetic sensitivity analysis; 2,000 benign and 2,000 attack snapshots, 500 ramps per cell)

	Six-bound profiler		**Peer-count-only**	
Attack	**Delay**	**Evasion**	**Delay**	**Evasion**
P1 Peer-slot exhaustion	2.23 ± 0.87	0.17	never	1.00
P2 Multi-subnet Sybil	1.00 ± 0.00	0.00	never	1.00
P3 Churn-based rotation	1.70 ± 0.61	0.12	never	1.00
P4 Relay-node latency	2.71 ± 1.19	0.25	never	1.00

Delay: mean ± sample standard deviation of the snapshots to first detection over the 500 ramps of 10 snapshots each ("never": no ramp was detected). Evasion: largest attack intensity on the 0–1 scale at which the detector still reads healthy. Both detectors have 0/2,000 false positives (95% upper bound 0.15%). The profiler has 0/2,000 false negatives at full intensity; the peer-count-only detector has 2,000/2,000, since each attack preserves the clean-peer count.

Table 16 is a sensitivity analysis of the two detectors against attack shapes that we define (Section 6.8); it does not measure robustness against arbitrary eclipse or Sybil strategies. Within these four attacks the profiler flagged every full-intensity snapshot (0 misses in 2,000) and every ramp (300 of 300 in each cell under three ramp shapes, linear, front-loaded and slow; 95% upper bound on the miss rate 1%), regardless of concurrent Bitcoin or DA degradation. It reads attacks of low intensity as healthy: up to 0.25 for P4, 0.17 for P1 and 0.12 for P3. An attacker who keeps every field within its bound is not detected by the six bounds by construction (Section 4.2). The peer-count-only detector detected none of the 300 ramps in any cell. The live runs show what happens after detection. In P3 (churn-based rotation: eight restart cycles on two of seven validators, 29% of voting power), the churn metric on both attacked validators crossed its threshold, yet no FSM transition was committed: two validators' signed observations cannot move a certified aggregate backed by a quorum. In P4 (a sustained 350±50 ms delay on all of one validator's P2P interfaces, observed by all seven validators), the chain moved ANCHORED→SUSPICIOUS→SOVEREIGN at +39 s and +72 s, held SOVEREIGN for the 244-second

attack, and returned to ANCHORED within 5 s of the delay clearing (Fig. 4).

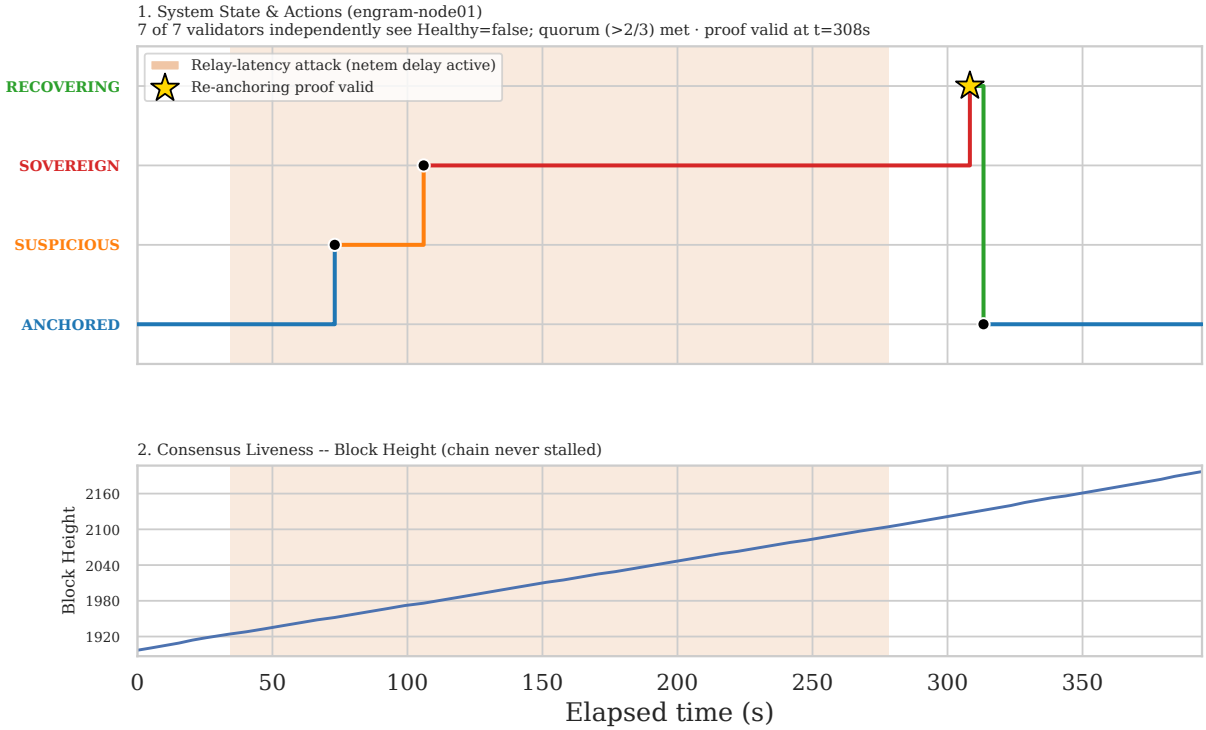


Figure 4: FSM cascade under a real, quorum-visible P2P relay-latency attack (P4): all 7 validators independently sense the degradation, the transition commits once quorum is met, and block production (bottom panel) never stalls through the cascade or the recovery.

P3 and P4 are two outcomes of one mechanism. The same kind of sensor-confirmed degradation is ignored when a minority observes it and drives a bounded cascade when all validators do, as Lemma 1 predicts: a minority cannot move the certified aggregate, whereas a degradation seen by every honest validator must move it.

7.2. RQ2: Liveness Under Peripheral Degradation

Engram versus the strict receipt-gated baseline. The baseline (Section 6.4) enforces Bitcoin and DA receipts at every height; the soft baseline keeps Engram's FSM but never locks withdrawals. We ran each system through three faults: Bitcoin checkpoint submission suspended, all seven Celestia bridges stopped, and both. A trial is 60 s healthy, 180 s of fault and 120 s of recovery, with a withdrawal-marked transaction sent every 2 s; each cell of Table 17 has three trials. The baseline is redeployed for every trial because, in our implementation, it does not recover on its own (below). Recovery uses proof-only re-anchoring with an unproven-tail bound of 128 blocks (Section 4.7).

Two observations should be kept apart. *During the fault*, the baseline commits a fraction of its blocks, which its validity predicate predicts and the experiment measures. Under a DA outage it commits 23 blocks in 180 s, 16% of its healthy rate, with proposals reaching round 21; the combined fault is the same. Under checkpoint suspension it slows to 56% (74 blocks, round 12): the receipt tolerance widens with the round up to a cap, so stale checkpoints are admitted for a while. *After the fault*, it commits nothing in the 120 s after the layer returns in all three DA trials and the combined trials, and in the checkpoint trials it stops resuming as well. In the one trial observed beyond the recovery window, the chain was still stuck after twelve minutes, with proposals rejected at round 57 and above. This does not follow from the predicate, which is

Table 17: Liveness and withdrawals under peripheral degradation (180 s fault, three runs per cell, mean and range). Rate is the fault-window block rate as a share of the same trial's healthy rate; "No resume" counts trials in which the chain committed no block in the 120 s after the fault ended ("—" in the gap column: it never resumed). The last two columns count withdrawals committed after and before the moment Engram's circuit breaker locks, 117 s (checkpoint pause), 41 s (DA outage) and 43 s (both) after the fault begins; the same window is applied to every system. Recovery is proof-only with an unproven-tail bound of 128 blocks.

			Liveness in the fault					Withdrawals committed	
Fault	**System**	n	Blocks	Rate	Longest gap (s)	Max round	No resume	After lock	Before lock
Checkpoint pause	Receipt-gated	3	74 (73–74)	56 (53–57)%	—	12	3/3	8 (6–9)	55 (54–56)
	Engram	3	147	102%	2.4 (2.4–2.5)	1 (0–1)	0/3	0	52 (45–56)
	Soft	3	144 (143–145)	99 (98–100)%	2.5 (2.4–2.5)	1 (0–1)	0/3	31 (30–31)	56 (56–57)
DA outage	Receipt-gated	3	23 (22–24)	16 (14–17)%	—	21 (21–22)	3/3	3	19 (18–20)
	Engram	3	150 (147–153)	96 (93–99)%	1.4 (1.3–1.4)	0	0/3	0	18 (18–19)
	Soft	3	151 (149–152)	99 (98–100)%	1.5 (1.4–1.7)	0	0/3	72 (72–73)	19 (19–20)
Both	Receipt-gated	3	24 (23–24)	17 (16–19)%	—	21 (21–22)	3/3	3	19 (19–20)
	Engram	3	153 (150–155)	99 (97–104)%	1.4 (1.3–1.4)	0	0/3	0	19 (18–20)
	Soft	3	151 (150–152)	98 (97–99)%	1.3	0	0/3	71 (70–71)	20 (20–21)

satisfiable again once the layer returns. It follows from two dependencies on block commits. The first is a choice of this implementation: the checkpoint submitter, the DA publisher and the sensor reads all run inside `ExtendVote`, which ABCI++ calls only when a validator is about to precommit a block, so once proposals are rejected no validator precommits, no new checkpoint is submitted and no DA height is published. The second follows from how observations travel: `ProcessProposal` receives no vote extensions, so the certificate in each proposal is built from the extended commit of the last committed block and does not change until another block commits. A design that delivers receipts as vote extensions and requires them for validity would share the second dependency; a design that observes the layers outside the vote extensions would not. The two together make the recovery circular, and in this implementation it takes a manual restart. We have not separated the two experimentally: a baseline whose checkpoint producer runs independently of block commits would remove the first only. Engram commits 147–153 blocks (96–102% of its healthy rate) under every fault, with a longest gap of at most 2.5 s and rounds 0–1, and resumes within two seconds of the fault ending.

The withdrawal columns separate the systems on withdrawal exclusion. After the circuit breaker has locked, Engram committed none; this count is a consistency check of the breaker, because the window opens when Engram's own FSM reaches Sovereign. The baseline, which has no breaker, committed 3 withdrawals after that moment under DA faults and 6–9 under checkpoint suspension, few only because it commits few blocks. Engram's cost shows in the last column: it committed 18–19 withdrawals before locking under DA faults and 45–56 under checkpoint suspension, because it needs about 41 s and 117 s to leave Anchored for Sovereign. These counts follow from the workload (one withdrawal-marked transaction every 2 s) and from Bitcoin running thirty times faster than on mainnet; they are not a production exposure. The workload-independent measures and a bound are in Section 8. The soft baseline shows the other side of the trade-off. It runs Engram's FSM and keeps Engram's liveness, 98–99% of the healthy block rate with a longest gap of at most 2.5 s, but it never locks, so it committed 31 withdrawals after the lock moment under checkpoint suspension and 71–72 under DA faults, against none for Engram. Counted by the FSM state at each height, 87–97 of its 175 withdrawals under checkpoint suspension and 132–137 of about 180 under DA faults were committed while the state was Sovereign or Recovering. Only Engram combines the liveness of the soft chain with the zero count of a chain that locks.

Composite fault. Engram was then run through a continuous, scripted composite-fault

sequence under a standing wide-area latency profile (Table 12): a Bitcoin fault, then a full DA outage (all seven bridges stopped), then three cycles of packet loss on the validator P2P network, healed in turn. The packet loss degrades peer quality without removing quorum connectivity. Fig. 5 shows the resulting timeline.

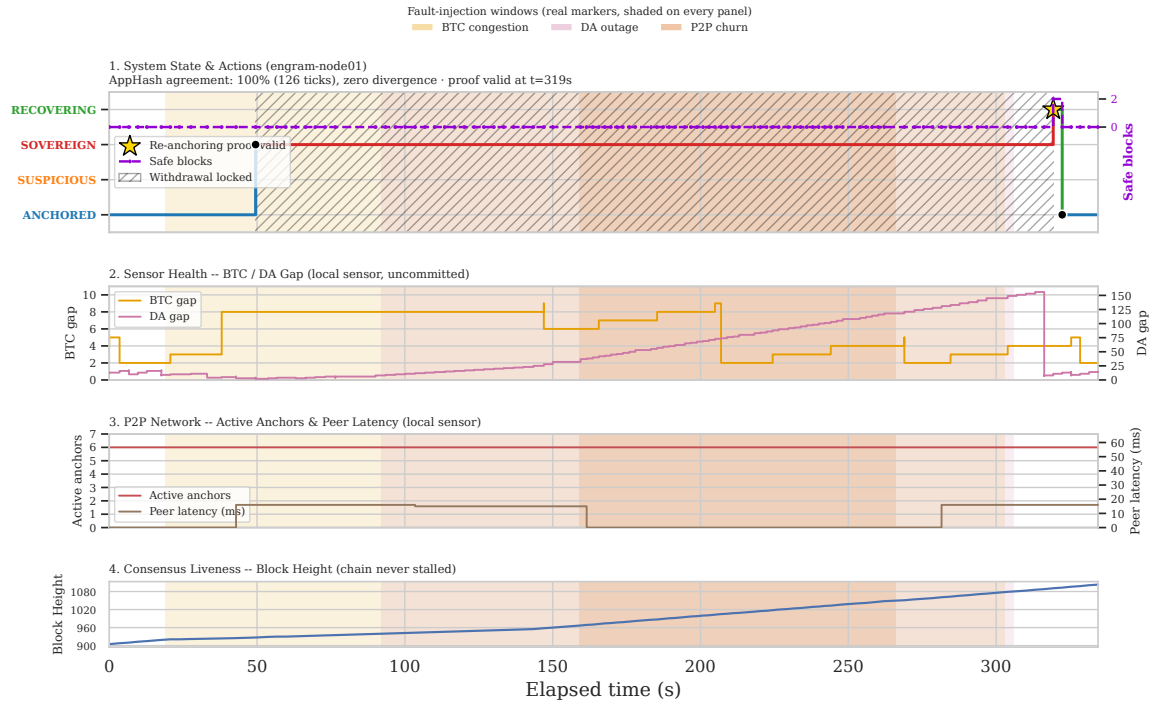


Figure 5: FSM state timeline, sensor readings, P2P network state, and block height under a layered composite fault sequence (Bitcoin congestion + DA outage + P2P churn, overlapping at their peak). First composite run: one trial, heuristic fallback enabled.

In the first run, made with the heuristic fallback enabled, over 341 seconds (742 samples, 126 polling ticks), the height advanced continuously from 905 to 1093 with no stalled height, and all seven validators agreed on the AppHash at every tick (zero divergent heights, zero RPC errors). The chain entered SOVEREIGN under the combined fault and, after the faults were healed, the recovery proof was verified at t=319 s, the chain entered RECOVERING at t=321 s and ANCHORED at t=324 s. Withdrawals stayed locked throughout SOVEREIGN and RECOVERING, as the circuit-breaker invariant requires. These timings come from a single trial.

We then repeated the sequence five times, each on a freshly deployed chain started from ANCHORED, with proof-only recovery and an unproven-tail bound of 128 blocks, so that recovery needed a verified proof. In all five runs the height advanced throughout (from 373–387 to 747–760), the validators agreed on the AppHash at every sampled height (zero divergent heights; one RPC error in one run), and the chain ended in ANCHORED. It reached SOVEREIGN 39–52 s after the start (mean 44.2 s, 95% CI ±6.5 s over five runs), four of the five passing through SUSPICIOUS first (16–22 s). RECOVERING began 312–325 s in, after the faults were healed, and lasted 233–257 s (mean 240.6 s, 95% CI ±12.0 s), against 3 s in the first run; most of that time is the cost of the recovery proof. In one run two of the seven validators fell back from RECOVERING to SOVEREIGN for 2 s before returning. [PENDING: repeat the seven-scenario run ($\geq$ 5 trials) and report its recovery time as mean ± 95% CI.]

Seven-scenario sweep. The composite trace does not isolate which failure mode drives which behavior. A second run therefore walks the chain through seven sequential scenarios in one 864-second run (one trial; Table 12): S1 healthy baseline, S2 Bitcoin congestion (checkpoint

submission suspended), S3 DA outage (bridges stopped), S4 partial P2P eclipse, S5 anchor-peer isolation of one validator, S6 combined Bitcoin and DA failure, and S7 recovery. Each scenario starts from the state the previous one left; 153 FSM transitions were logged across the seven validators (Fig. 6).

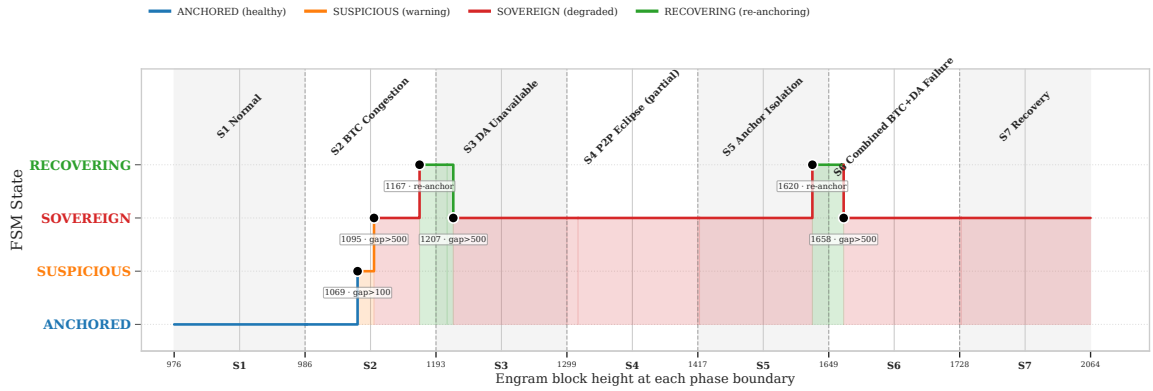


Figure 6: Continuous FSM-state timeline across seven sequential fault-injection scenarios (S1–S7), true block-height ranges, live 7-validator testbed. Each scenario begins where the previous scenario's recovery ended. One trial, heuristic fallback enabled.

Each scenario produced the expected response. S2 and S3 each escalated through Suspicious to Sovereign and recovered. S4 escalated to Sovereign, whereas S5 produced only a transient Suspicious excursion absorbed by hysteresis: isolating one of seven validators from its anchor peers leaves the certified active-anchor-peer count above zero, so the critical condition never holds. S4 and S5 are therefore distinct failure classes rather than two intensities of one signal. S6 escalated directly to Sovereign, and S7 returned to Anchored through a submitted and verified recovery proof. Two further observations stand out. First, S2 contains two brief Suspicious excursions (19–22 s and 77–80 s) that return to Anchored before the escalation at 141 s, showing hysteresis absorbing live rather than synthetic noise. Second, block production stayed close to baseline: block-interval means ranged 1.30–1.93 s across all seven scenarios (vs. 1.34 s at the S1 healthy baseline; all intervals in this paragraph are Engram's), and the largest p95 intervals occurred in S4 (4.27 s) and S7 (3.10 s). Engram keeps committing blocks under overlapping faults and under each failure mode in isolation, and returns to Anchored in every case.

7.3. RQ3: Recovery and Flapping

Hysteresis and flapping. Hysteresis should suppress oscillation ("flapping") without unduly delaying legitimate transitions. We swept the threshold on each of the three gated edges – Recovering→Anchored, Anchored→Suspicious, and Suspicious→Anchored – in-process under fixed-seed synthetic noise and live on the seven-validator cluster under real network noise (Fig. 7).

Table 18: Down-hysteresis sweep (Anchored→Suspicious), in-process (101 blocks, noise probability 0.2, seed 11, one run per threshold)

Threshold	1	2	4	6	8
Flapping (count)	31	1	1	0	0
Anchored uptime (%)	61.39	96.04	98.02	100.00	100.00
Absorption rate (%)	0.00	94.74	95.24	100.00	100.00

On the Anchored→Suspicious edge (Table 18), raising the down-hysteresis threshold from 1

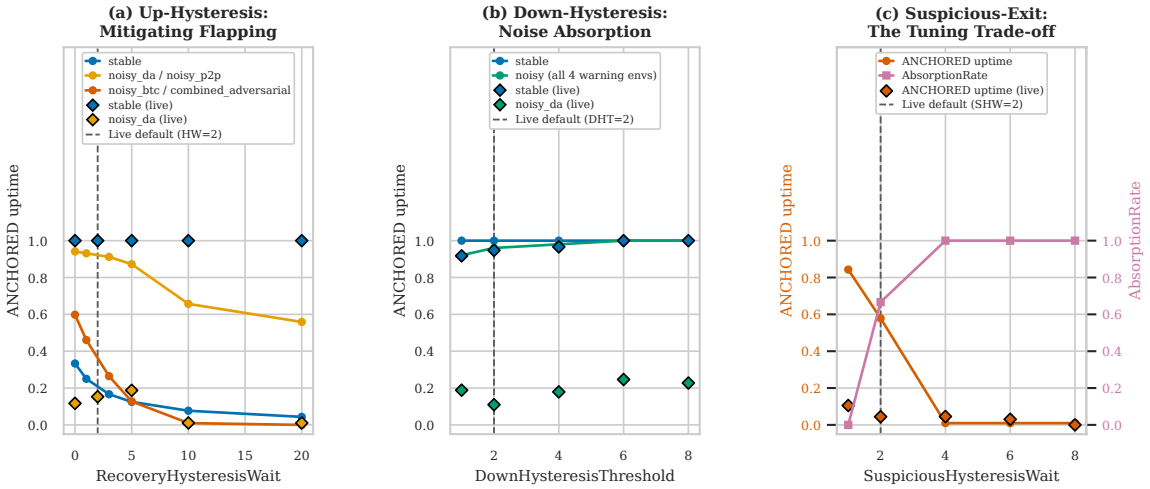


Figure 7: Hysteresis parameter sweep across all three absorb edges. Lines: in-process sweep under different noise profiles; diamonds: live spot-checks on the real 7-validator cluster (divergence from the in-process line is real network behavior, not measurement noise); dashed line: the protocol's live production default.

to 4 collapses flapping from 31 events to 1 and raises anchored uptime from 61.4% to 98.0%, and the absorption rate saturates near 100% by threshold 4–6. The live spot-check changes the conclusion. Under wide-area jitter alone, with no injected fault, thresholds 1, 2, and 4 still flapped, and only thresholds ≥ 6 were clean (0 flaps, 100% uptime). The default of 2 used in the other experiments is therefore too low for WAN conditions; the other experiments did not count flaps and cannot be taken as evidence to the contrary. A static default must trade reaction time (low k) against noise immunity (high k), and the right point depends on network conditions that change over time; the extension evaluated at the end of this section tries to adapt to them.

The other two edges show the opposite side of the trade-off. On the RECOVERING→ANCHORED edge, a larger hysteresis wait strictly reduces anchored uptime under sustained critical-level noise (59.8%→0.0% as the threshold goes from 0 to 20), under the critical-level noise of Section 6.8. This follows from the design. In RECOVERING, a critical reading returns the chain directly to SOVEREIGN (the RECOVERING branch of `CalculateNextState`), so the run of healthy blocks must start again, and a longer wait is less likely to complete between two critical readings. On the SUSPICIOUS→ANCHORED edge, absorption rate rises from 0% to 100% as the threshold goes from 1 to 4, at a cost: the SUSPICIOUS timeout keeps running while the counter accumulates, so only thresholds 1–2 reached ANCHORED before the timeout forced SOVEREIGN. Any adaptive threshold must therefore stay below the SUSPICIOUS timeout, which bounds $k_{\max}$.

Recovery: what ends Recovering. The state exposes whether the proof condition holds, not which path set it, so we ran one 120 s DA outage per setting and read the prover's log. With the heuristic fallback enabled, the chain left RECOVERING after 36 s while the prover's single proof was rejected as invalid (the interval had grown while it was built): the fallback, not the proof, ended recovery. With proof-only recovery and the default bound of 8 unproven tail blocks, the chain never left RECOVERING, not even after genesis (observed for over thirty minutes in two deployments), although the chain accepted the prover's proofs (seven in one deployment). A 256-header proof takes about two minutes to build and submit, more than 90 blocks at 1.2 s, so the tail was always longer than the bound. With the bound raised to 128 blocks the chain left RECOVERING 341–356 s after entering it in each of five runs on fresh chains (mean 347.2 s, 95% CI ±8.1 s), the first valid proof being accepted 341–353 s in (recorded in three of the five). The time is mostly the wait for the prover's batch of 256 headers and its build time, not the 68–73

blocks of the outage interval. The bound must therefore exceed the proving latency measured in blocks, and the blocks of that tail are protected by consensus alone. No block of these intervals was committed in a round other than 0. The RQ2 trials above and the repeated composite runs use proof-only recovery with the bound of 128; the first composite run, the seven-scenario run and the data-availability runs used the fallback.

Data availability of the Sovereign interval. The exit condition of Section 4.7 requires the interval's block data to be republished before RECOVERING→ANCHORED. We stopped all seven Celestia bridges for 120, 300 and 600 s, restarted them, and after the chain returned to ANCHORED re-fetched every height of the interval from Celestia through `blob.GetAll`, comparing the bytes with the Engram block. Table 19 shows that all 2,291 heights over nine runs were found and identical. The chain entered SOVEREIGN 46–49 s after the bridges stopped, and the time in RECOVERING did not grow with the interval: 37–61 s for 75–77 blocks and 49–61 s for 465–475 blocks. The check is external to the chain: the gate itself rests on the quorum-reported contiguous height, and these runs show that height was truthful when all validators were honest. They cover a total DA outage on one host with three runs per length. Runs under the earlier 512 MiB bridge limit are excluded; at that limit the bridges crash-looped and the 600 s outage never returned to RECOVERING. These runs kept the heuristic fallback of Section 4.7 enabled (proof-only recovery off), so the time in RECOVERING does not isolate the cost of the recovery proof; the data-availability results are independent of that choice. A gate that always opens would pass the runs above, so we also withheld data. A test hook makes a validator publish only head markers: its DA liveness stays healthy, so the chain can reach RECOVERING, but its contiguous height never advances. The certified height is a weighted median over the certificates, so the gate should hold when four or more of the seven validators withhold data and open when three or fewer do. Table 20 shows that it did in all seven runs. In a third control, two validators signed a contiguous height of 10^{12} while the five honest ones withheld data; the median stayed at zero and the gate held, as it should against $f{=}2$ misreporting validators. These controls use two to three runs per setting, each on a freshly deployed cluster, and the withholding validators are a test configuration, not a modelled adversary. A first attempt at the third control ran with only two of its five withholding validators because the override file omitted three nodes; it is excluded.

Table 19: Republication of the SOVEREIGN interval after a total DA outage (three runs per outage length; range over runs; heuristic fallback enabled)

Outage (s)	Interval (blocks)	Time in Recovering (s)	Heights found, identical
120	75–77	36.6–61.1	229 / 229
300	217–219	60.9–67.3	655 / 655
600	465–475	49.0–61.2	1407 / 1407

Table 20: Backfill-gate controls after a 120 s DA outage (validators withholding block data; the gate must hold for 300 s or open; heuristic fallback enabled)

Setting	Runs	Interval (blocks)	Outcome
4 of 7 withhold data	3	201–208	held in RECOVERING 3 / 3
3 of 7 withhold data	2	67–72	left RECOVERING after 34 and 43 s; data identical
2 inflated claims + 5 withhold	2	202–208	held in RECOVERING 2 / 2

Exploratory extension: an adaptive down threshold. A static threshold trades reaction time against noise immunity, and the right point depends on network conditions that change over time. The rest of this section evaluates an extension that tries to follow them. We report it as exploratory and do not claim it as a contribution; the summary at its end gives the reason.

We extend the backoff already used on RECOVERING to the down threshold $k_{\downarrow}$ of both demotion edges (Section 4.5). The effective threshold is $k_{\downarrow} = \min(k_{\max}, k_0 \cdot 2^c)$, where the flap level c rises by one at each such demotion and falls by one after T_{stable} healthy ANCHORED blocks. $k_{\downarrow}$ depends only on committed state, so it remains deterministic across validators. It starts at the responsive value k_0 and rises only when the network proves noisy, which also raises the cost of the repeated-trigger attack discussed in Section 8. The mechanism is implemented in the specification and in the prototype. We swept the down threshold live on the seven-validator cluster (Table 21): static $k \in \{2, 4, 6, 8\}$, each its own cap, and the adaptive threshold (k_0=2, $k_{\max}$=8), under wide-area latency jitter with no fault, under DA blips (all seven bridges stopped and restored in cycles of tens of seconds), and under a sustained DA outage (all bridges stopped) to measure detection. All 33 runs completed. A static value keeps no hidden state, so one genesis served all its trials and environments; the adaptive configuration was redeployed for every run because its flap level lives on the chain. Static k=2, k=6 and the adaptive threshold have three trials each; k=4 and k=8 have one.

Table 21: Live down-edge sweep on seven validators (300 s runs, mean ± 95% Student-t interval over the runs, none for a single run; flaps and uptime averaged over the seven validators of a trial). "Uptime" is the share of the run in ANCHORED; "Detection" is the delay from stopping the bridges of the sustained outage to the first non-ANCHORED state on one validator, and "To SOVEREIGN" the delay to SOVEREIGN.

Configuration	Runs	Flaps (WAN)	Uptime (WAN, %)	Uptime (DA blips, %)	Detection (s)	To Sovereign (s)
static k=2	3	9.0 ± 0.0	82 ± 3	1 ± 0	9.4 ± 0.4	38 ± 2
static k=4	1	0.0	100	1	11.1	41
static k=6	3	0.0 ± 0.0	100 ± 0	2 ± 2	12.8 ± 3.2	44 ± 5
static k=8	1	0.0	100	3	14.6	43
adaptive (2 → 8, T_{stable}=16)	3	8.3 ± 2.9	85 ± 7	1 ± 0	11.7 ± 0.9	42 ± 3
adaptive (2 → 8, T_{stable}=64)	3	3.0 ± 0.0	93 ± 2	–	11.1 ± 1.9	41 ± 3

Under jitter alone the static default of 2 flapped nine times per validator and spent 82% of the run in ANCHORED, and every static value from 4 up held ANCHORED without a flap. The adaptive threshold did not behave like the larger values: 8.3 flaps and 85% uptime, within the interval of k=2. Under DA blips no configuration stayed in ANCHORED (1–3% uptime), so the threshold did not matter there. Detection delay grew with the threshold, from 9.4 s at k=2 to 14.6 s at k=8 (one run), and the adaptive configuration sat between k=2 and k=6 at 11.7 s. In this sweep the adaptive threshold therefore bought no flap suppression over a static k=2, whereas a static k of 4 or more suppressed all flapping at a cost of under 2 s of detection delay for k=4; with one trial at k=4 and k=8 this is an observation, not an estimate. We did not measure the cost the adaptive threshold imposes on a repeated-trigger attacker, which is its stated purpose. A longer stable window is a candidate explanation for the weak suppression, since the flap level decays after 16 healthy blocks, about twenty seconds. We therefore repeated the adaptive configuration with a window of 64 blocks, under jitter and under the sustained outage only, three runs each, every run on a fresh genesis (last row of Table 21). Flaps fell from 8.3 to 3.0 per validator (all three runs gave 3), uptime rose from 85 to 93%, and the detection delay did not change (11.1 against 11.7 s). A window of 64 blocks recovers part of the gap, but not the full suppression of a static k of 4 or more; we did not test larger windows or the DA-blip environment with it.

Changing noise and a repeated-trigger adversary (simulation). The sweep above holds the noise constant, the case in which a well-chosen static threshold needs no adaptation. The case for the adaptive threshold is noise that changes, and a party that triggers demotions on purpose. We tested both in-process with the keeper's own transition code. Noise is a sequence of bursts of one to three consecutive warning-level blocks, each starting with probability 0.03 per block; this reproduces the live demotion rate of a static threshold of 2 (4.6 demotions per 250 blocks, against 5.0 per 300 s live), and the same 200 noise schedules drive every configuration, so

differences are paired. A run has 150 quiet blocks, 250 noisy blocks and a quiet phase, after which a sustained DA outage starts and we count the blocks until the first non-ANCHORED state. The adversary sends warning-level bursts for 1200 blocks and waits for ANCHORED between them: a naive one always sends k_0 blocks with one healthy block in between, a patient one waits for the flap level to return to zero first. The last column of Table 22 is the concealment simulation of Section 8.

Table 22: Static and adaptive down thresholds in the in-process simulation (blocks; mean ± 95% interval over 200 paired noise schedules). "Detect after noise" starts the outage as soon as the chain is ANCHORED again. Repeated trigger: demotions in 1200 blocks. Outage hidden: placements out of 35 of three Byzantine proposers that keep a real DA outage from being detected, with the chain quiet and with the flap level at its highest.

	Changing noise			**Repeated trigger**		**Outage hidden**
Threshold	**Detect, quiet**	**Flaps**	**Detect after noise**	**Naive**	**Patient**	**quiet / raised**
static k=2	2.0 ± 0.0	8.2 ± 0.5	2.0 ± 0.0	300	300	0 / 0
static k=4	4.0 ± 0.0	0.2 ± 0.1	3.9 ± 0.1	200	200	28 / 28
adaptive 2 → 8, T_{stable}=64	2.0 ± 0.0	3.4 ± 0.2	3.2 ± 0.2	1	18	0 / 35
adaptive 2 → 4, T_{stable}=64	2.0 ± 0.0	3.4 ± 0.2	3.1 ± 0.1	1	18	0 / 28

Against a static threshold of 2, the adaptive one detects the outage as fast when the network is quiet (2 blocks), cuts the flaps of the noisy phase from 8.2 to 3.4, and absorbs the naive adversary (1 demotion against 300); a patient adversary obtains 18. Against a static threshold of 4 it does not win on flaps (3.4 against 0.2). What it keeps is the 2-block detection when quiet, about two seconds at the block rate here, and the resistance to concealment of the smallest threshold (Section 8). Its responsiveness returns slowly after noise: right after the noise the detection takes 3.1–3.2 blocks, and 2.0 after 96 quiet blocks. An adversary that reads the committed flap level pays 7.9 bad blocks per demotion with $k_{\max}$=8 and 4 with $k_{\max}$=4 (2 for a static threshold of 2). These are simulation results, calibrated to the live sweep above but not measured live.

Changing noise (live). We ran the same three phases on the seven-validator cluster: a quiet phase, a phase under the wide-area latency and jitter profile of the stationary sweep, and a quiet phase after which, once the chain was ANCHORED again and had stayed so for an offset of 0 or 60 s, all Celestia bridges were stopped. Every run used a fresh genesis, with three runs per configuration and offset (Table 23 and Fig. 8; the noisy-phase columns pool the six runs of a configuration).

Table 23: Live three-phase runs: mean (range) over three runs per offset (the noisy phase pools six). Detection is the time from the bridge stop to the first non-ANCHORED state.

	Flaps	**Anchored share** noisy phase	**Detection, offset 0** s	**To Sovereign, offset 0** s	**Detection, offset 60** s
Static k=2	7.5 (7.0–9.0)	0.82 (0.80–0.84)	10.9 (10.7–11.4)	39.8 (38.7–41.3)	none: left ANCHORED in 3/3
Static k=4	0.0 (0.0–0.0)	1.00 (1.00–1.00)	13.7 (13.1–14.1)	43.7 (42.6–44.5)	11.5 (11.2–11.7)
Adaptive, window 16	6.8 (5.0–8.0)	0.84 (0.83–0.88)	12.7 (12.3–12.9)	41.3 (40.3–41.9)	none: left ANCHORED in 3/3
Adaptive, window 64	3.0 (3.0–3.0)	0.92 (0.91–0.92)	14.6 (13.2–15.8)	44.3 (43.1–45.4)	9.8 (9.7–10.0)
Adaptive, window 128	1.0 (1.0–1.0)	0.96 (0.95–0.96)	16.2 (13.1–20.6)	46.7 (43.0–51.2)	none: left ANCHORED in 3/3
Adaptive, cap 4, window 64	3.0 (3.0–3.0)	0.89 (0.80–0.92)	15.6 (14.1–17.5)	45.9 (44.8–46.6)	10.6 (10.3–10.8) [1/3 left ANCHORED]

A static threshold of 4 never flapped and stayed ANCHORED through the noisy phase of all six runs, at a detection delay of 13.7 s against 10.9 s for k=2, which flapped 7.5 times and spent 82% of the phase ANCHORED. The adaptive threshold suppressed more as its window grew, 6.8 flaps for a window of 16 blocks, 3.0 for 64 and 1.0 for 128, which is close to the simulation for windows of 16 and 64 (6.1 and 3.4 flaps). It paid for that in detection at offset 0, 12.7, 14.6 and 16.2 s. No adaptive configuration reached the flap count of the static threshold of 4 at a shorter delay; the only one faster than it, with a window of 16, flapped 6.8 times. After an offset of 60 s the adaptive level had decayed: with a window of 64 the detection took 9.8 s against 11.5 s

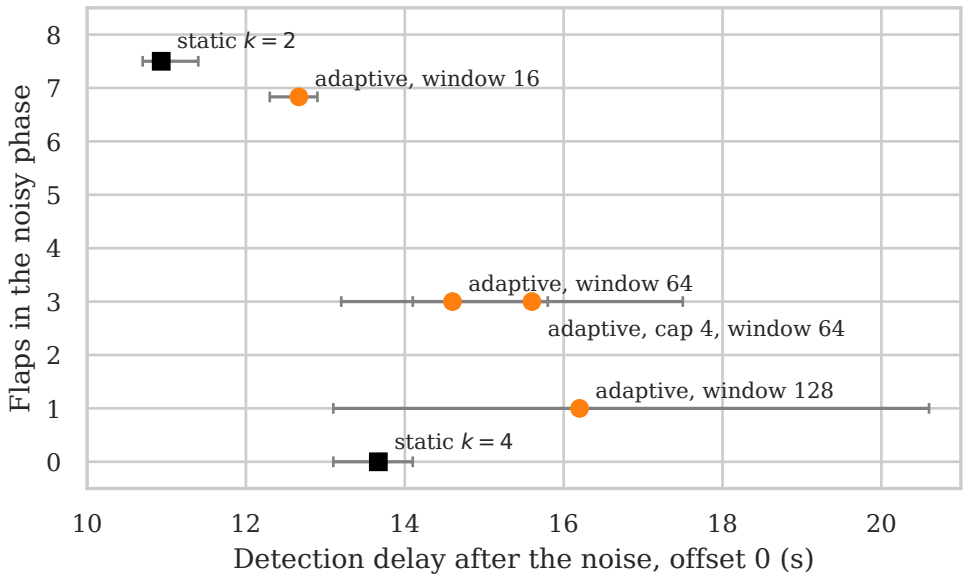


Figure 8: Flaps against detection delay in the live three-phase runs. Squares are static thresholds, circles the adaptive ones; the bars span the three runs at offset 0. Lower and to the left is better.

for the static threshold of 4, the ranges not overlapping, though with three runs each this is an observation, not an estimate. For k=2, a window of 16 and a window of 128 the chain left ANCHORED before the outage in all three runs at that offset, so no detection delay was measured; we did not identify the cause.

Repeated trigger (live). The live adversary stops every Celestia bridge for a nominal pulse of 3 s and restarts them, 21 to 23 times in about ten minutes (15 runs, three per configuration, each on a fresh genesis). Stopping and restarting take time: the bridges were down for 15–17 s per pulse on average, far longer than the nominal pulse and than any threshold, so a pulse is a full DA outage and not the short warning-level burst of the simulation. In all 15 runs, whatever the configuration, the chain left ANCHORED at the first pulse (one demotion), reached SOVEREIGN, and spent 0.2–2.2% of the run in ANCHORED. This experiment therefore does not separate the thresholds, and it gives no live evidence that the adaptive threshold absorbs short pulses: the simulation remains the only evidence for that case, and a live adversary needs a fault faster than stopping a bridge to produce pulses shorter than the threshold.

Summary of the extension. In the stationary live sweep the adaptive threshold behaved like k=2 (8.3 flaps, 85% uptime), and with a window of 64 blocks it flapped 3.0 times. In the live three-phase runs the flaps fell with the window (6.8, 3.0 and 1.0 for 16, 64 and 128 blocks) at detection delays of 12.7, 14.6 and 16.2 s, against no flap at 13.7 s for a static threshold of 4. The live repeated-trigger runs did not separate the thresholds, and the case the extension is meant for, absorbing short repeated pulses, rests on the simulation alone. We therefore treat a static threshold of 4 or more as the setting to use (Section 8) and the backoff as a result that did not hold up.

Appendix D complements these results by measuring how deep a real Bitcoin reorganization must be before the FSM reacts.

7.4. RQ4: Cost

Where the cost is not. Each proposal carries the health certificate as its first transaction: one signed vote extension per validator in the quorum plus the leader's claimed aggregation. At seven validators it is 894 bytes on average (live, ANCHORED; 889–926 across states, 300

Table 24: Overhead of the extended consensus object against vanilla CometBFT (20 tx/s, 60 s per repeat, mean ± 95% CI)

	Vanilla			Engram
Metric	1 process	7 val., no ext.	7 val., ext. on	7 val.
Repeats (runs)	5	5	3	5
Rounds per height	1.000	1.000	1.000	1.000
Proposals rejected / height	0	0	0	0
Nil precommits / height	0	0	0	0
Block interval, mean (s)	1.034 ± 0.001	1.233 ± 0.015	1.229 ± 0.023	1.262 ± 0.020
Block interval p50 / p95 (s)	1.032 / 1.042	1.220 / 1.313	1.217 / 1.300	1.241 / 1.335
Blocks/s	0.967	0.811	0.814	0.793
Committed tx/s	20.00	20.00	20.00	20.00
Tx commit latency p50 / p95 (s)	0.67 / 1.16	0.99 / 1.57	0.99 / 1.56	1.03 / 1.63
Mempool backlog, mean / max	8.2 / 24	14.5 / 31	14.6 / 31	15.5 / 44
Tx per block (mean)	17.6	21.1	21.1	21.5

Per-height values come from scanning *every* committed height (commit round, nil precommits, block time). Nil prevotes are not stored on chain; rejected proposals (distinct height/round/reason in the validators' logs) are the proxy for why a validator would prevote nil. "ext. on" keeps vote extensions enabled in the genesis with empty extensions; "no ext." is a fresh genesis without them. The Engram arm is a fresh deployment; a first deployment gave 1.242 ± 0.005 s (three runs). Transaction counts exclude the health certificate in `Txs[0]`. Latency runs from submission to the first 0.25 s poll that sees the transaction in a block. The 1-process arm is a local single node.

consecutive blocks), which splits into a fixed 70 bytes and 118 per validator (22 for the address, 28 for the extension, 66 for the signature). The signatures are a copy of the vote extensions, kept in the proposal because `ProcessProposal` receives none under ABCI 2.0; they are separate from the signatures in the block's last commit, so the cost is paid twice. The one proof-carrying block observed was 14.9 KB, consistent with the 14.7 KB proof. Validating an accepted proposal, including the ed25519 verification of one signature per validator, costs 263 μs per block at seven validators (mean of ten benchmark runs, standard deviation 6 μs). Size and validation time are linear in the number of validators (Table 25); we measured the block interval only at seven validators, so the larger rows are a microbenchmark of the certificate, not a measurement of a larger network. Verifying a recovery proof, a one-off event, takes 26 ms and is not included. `ExtendVote`, which reads the live Bitcoin and Celestia sensors before a validator can precommit, takes 16.6 ms on average (p95 51 ms, five repeats) and does not depend on the number of validators, since each validator reads only its own sensors. None of these is large enough to explain a lower block rate at a ∼1 s block interval at seven validators.

Table 25: Certificate size and `ProcessProposal` time against the number of validators (microbenchmark of the shipped code with real ed25519 signatures, one core, ten runs; sizes are from the benchmark fixture, whose extensions are about 15% smaller than the live ones, so the live size at n=7 is 894 against 776 bytes).

Validators	**Certificate, bytes**	**`ProcessProposal`, μs (sd)**
4	473	159 (10)
7	776	263 (6)
16	1,685	625 (62)
32	3,301	1,177 (29)
64	6,533	2,367 (50)

Where the cost is. Against a single local process, Engram's blocks arrive 0.23 s later (1.262 against 1.034 s). Table 24 shows that this gap is not Engram's. Seven vanilla validators on the same hosts, network, genesis and load produce a block every 1.233 s with vote extensions disabled and every 1.229 s with empty extensions, within 0.03 s of Engram (95% CIs of 0.015–0.023 s). Hence the 0.2 s is the price of running a seven-validator BFT network on one host, and vote

extensions as such add no measurable time. Engram's own point estimate is about 0.03 s per height (2%), the order of the 16.6 ms `ExtendVote` time, but the data do not establish it. The repeats of one arm share a chain, so the unit of independence is the deployment: the three Engram deployments average 1.2707, 1.2423 and 1.2620 s and the two seven-validator vanilla deployments 1.2292 and 1.2331 s. The difference of means is 0.027 s, and its 95% interval from the pooled between-deployment spread (0.012 s, three degrees of freedom) is −0.008 to 0.062 s (−0.6 to 5.0%), which includes zero. The data show an addition of a few percent at most, not a more precise value. [PENDING: report the paired estimate, which alternates the two arms on fresh deployments.] The arms ran on separate deployments except the first Engram and the extensions-on arm, which shared one chain (Engram first); the Engram arm was repeated on a fresh deployment to check for an order effect and moved by 0.02 s.

Load sensitivity. Table 24 is measured at 20 tx/s. Both systems accepted every submitted transaction into the mempool across an earlier 5–100 tx/s ramp, but mempool acceptance is not throughput. To see how the two systems respond to load we offered 20, 50, 100, 200 and 500 tx/s for 60 s with five repeats per rate and drained the mempool after each (Table 26). We did not look for a throughput ceiling, which one host cannot give: vanilla CometBFT and Engram committed every offered transaction at every rate, so neither saturated, and the limit we hit first was CPU, with the busiest validator at 64–83% at 500 tx/s on a host that runs all seven. What separates the systems is latency: the p95 stays at 1.5–1.9 s for vanilla, while Engram's rises from 2.2 s at 100 tx/s to 6.5 s at 500 tx/s.

Table 26: Load sensitivity under open-loop offered load, seven validators on one host (60 s per repeat, five repeats; mean ± 95% Student-t interval). The ANCHORED share is the fraction of two-second samples of one validator's FSM state spent in ANCHORED and was recorded for the third arm only.

Offered (tx/s)	Arm	Committed (tx/s)	p95 latency (s)	Peak CPU (%)	Anchored share
20	Engram	20 ± 0	1.6 ± 0.1	10 ± 2	–
20	Vanilla	20 ± 0	1.6 ± 0.1	8 ± 3	–
50	Engram	50 ± 0	1.7 ± 0.0	15 ± 8	–
50	Vanilla	50 ± 0	1.6 ± 0.0	12 ± 2	–
100	Engram	100 ± 0	2.2 ± 0.4	29 ± 6	–
100	Engram, peer-latency bound 5000 ms	100 ± 0	1.7 ± 0.1	21 ± 5	0.83 ± 0.09
100	Vanilla	100 ± 0	1.6 ± 0.1	19 ± 11	–
200	Engram	200 ± 0	3.8 ± 0.8	81 ± 11	–
200	Engram, peer-latency bound 5000 ms	200 ± 0	1.9 ± 0.5	39 ± 23	0.13 ± 0.37
200	Vanilla	200 ± 0	1.5 ± 0.1	23 ± 5	–
500	Engram	500 ± 0	6.5 ± 0.8	83 ± 12	–
500	Engram, peer-latency bound 5000 ms	500 ± 0	4.6 ± 1.1	73 ± 11	0.00 ± 0.00
500	Vanilla	500 ± 0	1.9 ± 0.4	64 ± 30	–

At low and medium load the addition of Engram is a few percent of the block interval (Table 24); at high load latency grows and the FSM can leave ANCHORED, as follows. The Engram figures above 100 tx/s are not a steady ANCHORED state. The peer-latency sensor read round-trip times of up to 1.2 s between validators under this load in the first Engram arm, above the 200 ms bound, so we repeated the sweep with the bound raised to 5000 ms. The chain still left ANCHORED: the share was 0.83 at 100 tx/s, 0.13 at 200 tx/s and 0 at 500 tx/s, so the peer-latency bound is not the only reason. A second candidate is the DA check: under load its RPC may exceed its timeout and read as a failed layer. We repeated the runs at 200 and 500 tx/s on fresh deployments, three each, with the DA RPC timeout raised to 90 s and the DA check timeout to 2.5 s, against three runs with the defaults. The share of samples in ANCHORED rose from 0.68 to 0.75 at 200 tx/s and from 0.36 to 0.80 at 500 tx/s, where the default runs also reached SOVEREIGN and the relaxed ones did not; the relaxed runs still left ANCHORED, so the DA timeouts explain part of the departure and not all of it. In the relaxed runs at 500 tx/s the p95 latency was 3.1–4.6 s and the peak validator CPU was 42–46% (three runs). That arm had

lower latency than the default one (p95 1.9 against 3.8 s at 200 tx/s) and was still above vanilla. We therefore cannot separate the cost of the consensus extension from the effect of a demoted FSM above 100 tx/s on this host, and report only that Engram's latency grows with load where vanilla's does not. All seven validators share the host's cores, so absolute values do not transfer to separate machines, and one host does not support a claim about production throughput or scalability. The 5000 ms bound was set through the environment of that deployment, whose genesis we did not keep; the first Engram arm did not record the FSM state.

8. Discussion and Limitations

8.1. Security Envelope and Trade-offs

The fallback trades cross-chain withdrawals for in-chain liveness. We discuss the trade along three axes: liveness inside the chain, what the lock establishes, and the cost of griefing.

Liveness inside the chain. In SOVEREIGN, block production, in-chain transactions, application logic, and the forced-transaction queue continue. Cross-chain withdrawals stop. A strict receipt-gated chain under the same degradation commits only a fraction of its blocks and, in our implementation, does not resume after the layer returns (Section 7.2). The two cases differ in the sensors. Observations are produced when validators precommit, so the loop that reads the periphery closes only while blocks commit. Relaxing the receipt of the failed layer in SOVEREIGN keeps blocks committing, which keeps the sensors reading, so the chain sees the layer return; the baseline rejects proposals for lack of fresh receipts and thereby stops the reads that would show them back.

What the lock establishes.

- *Shown:* the decision to admit or reject a withdrawal-marked transaction is agreed by all honest validators and applied identically once the FSM is SOVEREIGN or RECOVERING (Section 7.1).
- *Not shown:* that cross-chain assets are protected. The prototype has no bridge and no real assets: it mounts neither `x/auth` nor `x/bank`, transactions are unsigned envelopes, and a withdrawal is recognized by the substring `TX_WITHDRAWAL` in the transaction bytes.
- *Before the lock:* nothing is excluded until the FSM reaches SOVEREIGN; the exposure in that window is measured in Table 27.
- *What asset safety would need:* a bridge that releases funds only against evidence that the Engram height is committed in an anchored state, and a history that a coalition above the fault bound cannot rewrite while the chain is in SOVEREIGN mode (Section 4).

Cost of griefing. An adversary that degrades the peripheral view of *all* honest validators – for example by delaying their Bitcoin connectivity – can drive the chain into SOVEREIGN and freeze withdrawals for as long as the degradation lasts (P4, Section 7.1). This is a new attack surface relative to a chain that ignores its periphery, but not relative to a strict receipt-gated chain: the same attacker stalls the strict receipt-gated chain, whereas Engram freezes only withdrawals. Lemma 1 prevents Byzantine validators from causing the freeze alone: honest validators whose peripheral view is degraded must supply the rest of the majority. Together with the Byzantine validators they must hold more than half of the certificate weight, and the proposer's choice of quorum subset sets that weight. At n=7 with two colluders, an honest proposer that includes all seven observations requires two degraded honest validators, and a Byzantine proposer that selects a minimal quorum of five requires one. In a design without a quorum-backed health input, one node's view suffices. The freeze is the intended response to an unverifiable periphery,

not a safety violation. Its economic cost to users and the attacker's cost of sustaining it are not quantified.

Repeated triggering of Suspicious. Backoff on RECOVERING already raises the price of forcing repeated failed recoveries. Without the adaptive threshold of Section 7.3, the ANCHORED↔SUSPICIOUS boundary has no such protection beyond the bounded forced-transaction queue; the adaptive threshold is meant to close this asymmetry. In the simulation of Section 7.3 it absorbs a naive repeated adversary (1 demotion in 1200 blocks against 300 for a static threshold of 2) but gains little over a static threshold of 4. Live, the adaptive threshold suppressed flapping under changing noise only with a long stable window and never better than a static threshold of 4 at a shorter detection delay, and the live repeated-trigger runs did not separate the thresholds (every configuration escalated to SOVEREIGN at the first pulse), so the absorption of short pulses has no live evidence.

Exposure before the circuit breaker locks. Withdrawals lock when the FSM reaches SOVEREIGN, so the exposure is the time from the start of a fault to that point, and the number of withdrawals that can commit in it is at most the number of Engram blocks in the window times what one block can hold. The window is set by the thresholds, which count Bitcoin blocks for a Bitcoin fault and Engram blocks for a DA fault. On the testbed Bitcoin mines thirty times faster than on mainnet and Engram does not, so the measured times convert as in Table 27: a checkpoint pause locks after 117 s, 5.9 Bitcoin blocks and about 97 Engram blocks, which at mainnet cadence is about 58 minutes and 2,900 Engram blocks; the DA outage locks after 41 s on both. The 45–56 withdrawals of Section 7.2 follow from the workload and the acceleration: the same one withdrawal every 2 s over the mainnet window would be about 1,750, and nothing but block capacity limits a faster sender.

Table 27: Time from the start of a fault to the lock of withdrawals, as measured on the testbed (Bitcoin block every 20 s) and converted to mainnet cadence (every 600 s). Engram blocks are 1.2 s on both.

Fault	Testbed	Bitcoin blocks	Engram blocks	Mainnet
Checkpoint pause	117 s	5.9	97	58 min, 2,900 blocks
DA outage	41 s	—	34	41 s, 34 blocks

To check the conversion we drove the real FSM with Bitcoin blocks and a model of the checkpoint cycle of the anchor tracker: one checkpoint is pending at a time, it is included in the next Bitcoin block, it counts as confirmed with `KDeepFinality`+1 confirmations, the next one is submitted at once, and the committed anchored height follows the tracker after a lag of two to three Engram blocks, the one free parameter. At testbed cadence the model reproduces what we measured: the checkpoint pause reaches SOVEREIGN after 129 s (5th to 95th percentile 91 to 149 s; measured 117 s in RQ2 and 141.6 ± 13.8 s in S2), and the quiet phase of the live sweeps is 0.80–0.84 in ANCHORED at k=2 and 1.00 at k=4 (measured 0.79–0.87 and 1.00). At mainnet cadence the same fault reaches SOVEREIGN after 49.6 minutes with steady 600 s blocks (41 to 58) and 46.7 minutes with exponential block intervals (18 to 79), which is about five Bitcoin blocks and 2,300–2,500 Engram blocks.

The healthy gap is not flat. It follows the checkpoint cycle, 2, 3, 4, 5 and back to 2 every three Bitcoin blocks, and in the first 18 s of the five composite runs, before any fault, it sat at 5, the `SuspiciousThreshold`, in 3–16% of the samples, for about two Engram blocks each time. This has two consequences. The threshold cannot simply be lowered, since the healthy cycle already touches it. And part of the flapping at k=2 in the live sweeps is the cycle, not the wide-area noise: the model gives one demotion per cycle, 1,440 per day at the testbed cadence and 48 per day at mainnet cadence, so the accelerated testbed exaggerates the baseline flapping thirty-fold.

The model also predicts a limit of the current parameters at mainnet cadence. When two Bitcoin blocks arrive within one polling interval, the next checkpoint skips a block and the gap stays at 5 for a full Bitcoin interval, much longer than `MaxSuspiciousTime` (24 Engram blocks, about 29 s, chosen for the 20 s cadence), so the FSM escalates to SOVEREIGN without a fault. With exponential block intervals a healthy chain does so about once in eight days (10 events in 77 days of exposure, the same for k=2 and k=4) and never with steady blocks or at the testbed cadence. This is a prediction of the model, not an observation. The timeout must therefore follow the Bitcoin cadence of the deployment: the live value of 24 blocks is 1.4 Bitcoin intervals at the testbed's 20 s, and a value of about 1.5 intervals (750 blocks, 15 min, at 600 s) removes most false escalations at the price of about 14 more minutes to reach SOVEREIGN in a real fault (Table 28). We keep 24 as the default, which the testbed experiments use, and treat the timeout as a deployment parameter. [PENDING: confirm at a 600 s mining interval on the testbed.]

Table 28: `MaxSuspiciousTime` at mainnet cadence (simulation; exponential Bitcoin block intervals of mean 600 s, k=4): false escalations of a healthy chain per day (30 runs of up to 72 h; the counts behind the small rates are 10, 3 and 1 events, so they indicate order of magnitude only) and the mean time from a checkpoint pause to SOVEREIGN (40 runs).

Blocks	Time	False escalations per day	Mean to Sovereign, min
24	29 s	0.13	53
500	10 min	0.04	63
750	15 min	0.01	67
1500	30 min	0.01	76

Table 29 lists ways to bound the exposure and what each costs. We implement none of them; the first two bound the number of withdrawals without a lower threshold, and the last one shortens the window.

Table 29: Ways to bound the exposure before the lock.

Mechanism	Effect	Cost
Withdrawal quota per Bitcoin block, graded by the gap	$N \leq$ (Bitcoin blocks in the window) $\times$ quota, independent of the Engram block rate and of the sender	Lower withdrawal throughput also when healthy
Release funds only after the block is anchored K deep	Exposure close to zero	Every withdrawal waits about three Bitcoin blocks; needs a real bridge
Sensor on the age of the pending checkpoint against the expected cycle	Window shortens from about six to three or four Bitcoin blocks (our estimate)	Changes the FSM input, so every experiment on the Bitcoin gap would be rerun

Hysteresis: noise against concealment. A higher down threshold k tolerates longer bursts of noise, and it also tolerates a longer run of reports that a Byzantine proposer interleaves with honest ones. Within the assumption the choice of k does not matter: for each of the 21 placements of two Byzantine proposers in the seven-slot rotation, a real DA outage was detected after exactly k blocks ($k \in \{2, 3, 4\}$, simulation). With three Byzantine validators, outside the assumption, a Byzantine proposer can keep a certificate that reports a healthy DA layer, and an honest proposer then reports the outage in the blocks it proposes; the outage is detected only if enough honest proposers follow one another to reach k. In the simulation it was detected for all 35 placements at k=2, missed for 14 at k=3 and for 28 at k=4, and with four Byzantine validators it was missed for every placement. The live check at k=2 agrees: with three Byzantine validators the outage was detected in 5 of 5 runs. The adaptive threshold exposes the same weakness only while its flap level is high (0 of 35 placements hidden at level 0, 28 at level 1,

all 35 at level 2), whereas a static threshold of 4 or more is exposed permanently. A rule that demotes when at least m of the last w blocks are bad, instead of after a consecutive run, is not cut by interleaved reports. In a reduced model of the ANCHORED↔SUSPICIOUS edge, validated against the keeper-based simulation above (static thresholds and the adaptive $2 \to 4$ reproduce its flaps, detection delays and hidden placements), 4 of 8 hid the outage in 0 of 35 placements, against 28 for a static threshold of 4, at 0.8 flaps against 0.2 and the same 4-block detection. It does not remove the cost of being fast: its detection is as slow as the static threshold, and an adversary that sends bursts of m blocks still obtains 200 demotions in 1200 blocks, where the adaptive threshold allows 1. Combining the two (an adaptive window) gave 4.6 flaps and 172 demotions, so no single rule won on all four criteria. We did not implement the window rule. The live check at k=4 (three Byzantine validators with biased certificates, all seven bridges stopped for 120 s, a fresh chain per run) agrees with the simulation for adjacent placements and not for interleaved ones. With three consecutive Byzantine slots the outage was detected in 3 of 3 runs, as predicted. With slots 0, 2 and 4 of the rotation, which the simulation predicts would hide it, the chain also left ANCHORED and reached SOVEREIGN in 3 of 3 runs, with 6 to 9 blocks outside the honest range per run (6 to 12 over all six). We have not isolated why the prediction failed for the interleaved placement, so the concealment at k=4 rests on the simulation alone.

Choosing the down threshold. The experiments ran with `DownHysteresisThreshold` of 2, and the live runs show that value to be too low for a wide-area network: under the three-phase noise a static k=2 flapped 7.5 times and spent 82% of the noisy phase in ANCHORED, a static k=4 did not flap and stayed ANCHORED throughout, at a detection delay of 13.7 s against 10.9 s. At the testbed's accelerated Bitcoin cadence the healthy checkpoint cycle reaches the suspicious threshold in 3–16% of the samples, and k=2 spent only 0.79–0.87 of the quiet phase in ANCHORED, against 1.00 for k=4; the model of the cycle accounts for this, though we did not isolate it live. We therefore recommend $k_{\downarrow} \geq 4$ as a deployment setting, set from the noise of the environment like `MaxSuspiciousTime` is set from the Bitcoin cadence, and keep 2 as the value of the reported experiments. The price is the concealment of the preceding paragraph: an outage is detected only if k consecutive proposers are honest, so the larger the threshold, the easier it is to hide one.

Proposer bias within the honest range. When honest observations straddle a threshold, a proposer can choose which honest value the certificate selects (Section 4.3). This cannot produce a transition that no honest view supports, but a sequence of Byzantine proposers can bias the timing of transitions. A persistent split straddling an FSM threshold (e.g. three versus four validators on each side) has not been tested live; the RQ4 scan shows no extra rounds in the unfaulted case, and the split case itself remains untested.

Restart after a long chain. The publisher's contiguous height is kept in memory. A validator that restarts therefore republishes the chain from height 0 and reports DA as failed until it has caught up. In one run, four validators restarted on a chain of about 7,400 heights; the certified DA health stayed failed and the chain remained in SOVEREIGN for more than half an hour. Persisting the contiguous height would remove the dependence on chain length; the prototype does not.

History divergence at re-anchoring. During SOVEREIGN, the ordering of blocks rests on the validator quorum alone. A coalition exceeding f could produce a SOVEREIGN history that differs from one it would have produced under Bitcoin timestamping; the recovery proof certifies the history's continuity, its recorded mode and lock labels, and the boundary quorum signatures, not its uniqueness beyond the f bound. The withdrawal lock limits what such a history can extract.

Data availability of the Sovereign interval. During SOVEREIGN, the interval's block data is held by the validators' own block stores, so its recoverability depends on those validators until it is published. The backfill gate of Section 4.7 makes publication a precondition for returning

to ANCHORED.

The unproven tail: liveness against trust. A recovery proof is built while the chain keeps producing blocks, so by the time it is submitted the tip stands beyond the last header it covers. The gate accepts the proof only if that unproven tail is at most a bound, `MaxUnprovenTailBlocks`, so the bound must be at least the proving latency times the block rate. The bound sets a trade-off. A small bound keeps almost all of the SOVEREIGN interval under proof when withdrawals reopen, but a proof that takes longer than the bound allows to build never satisfies the gate, and the chain stays in RECOVERING. A larger bound lets slower proofs through, at the price that the last blocks before the chain returns to ANCHORED are certified by the validator quorum alone; a coalition above f could rewrite that stretch without the proof detecting it, which is the same residual exposure as in the history-divergence paragraph above, now bounded by the tail. At about 0.8 blocks per second, the bound of 128 blocks used in our experiments (the code default is 8) is about 2.7 minutes and a bound of 512 about 10.7 minutes.

Measured proving latency. In the first live deployment with chunk aggregation, the prover built a 512-header aggregate from two chunks in about 4.3 minutes and a 514-header aggregate from three in about 6.6 minutes; the tip then stood 211 and 321 blocks beyond the last covered header, close to the product of latency and block rate. Both proofs were accepted by `SubmitRecoveryProof`, but with the bound at 128 neither made the recovery flag hold, and the chain stayed in RECOVERING. A single 256-header proof takes about two minutes, about 110 blocks, and fits, but with little margin: at a block rate about 17% higher, or a slower prover, it would not, so the recovery times of the proof-only runs hold for this testbed's block rate and proving speed. With the bound raised to 512 blocks and everything else unchanged, the chain left RECOVERING through an aggregate proof in both of two deployments. In the first the prover built a 514-header proof from three chunks in 6.5 minutes, and the tip stood 309 blocks (about 6.4 minutes) beyond the last covered header when it was submitted; in the second a 512-header proof from two chunks took 4.6 minutes, with the tip 224 blocks (about 4.7 minutes) beyond. All seven validators were then in ANCHORED, 1,023 and 888 seconds after the chain started. Proof-only recovery was on, so only the proof could have restored it. The interval proved in these runs is the one since genesis, not one following an outage injected into a healthy chain, and the runs with the bound at 128 are a single deployment.

Aggregation bounds the verifier's cost, not the prover's. The aggregate is one more proof on top of the chunk proofs, so building it takes longer than building the chunks alone (49% longer at 512 headers and 22% at 1024 in Appendix C), and our prover builds the chunk proofs one after another, so latency grows with their number. The chunk proofs are independent given the chunk boundary roots, so they could be built on separate machines and joined by the aggregator circuit, and the latency would then be about that of one chunk plus one aggregation. On the offline workstation of Appendix C that is about one minute (a leaf proof takes 31.5 s and the aggregation 25.7–29.5 s), roughly 46 blocks at 0.8 blocks per second. On the shared Docker host a single chunk took about two minutes, so one chunk plus one aggregation would take about three minutes by our estimate from the single-chunk and aggregation timings, which exceeds the bound of 128 blocks used in our experiments. Going below it needs faster proving of each chunk, for example on dedicated multi-core machines or with hardware acceleration such as GPUs. The aggregation step still needs all chunk proofs on one machine. We measured neither hardware acceleration nor concurrent proving: the host had 8.7 GiB free once the testbed was stopped, below the 13 GiB that two concurrent provers need. What we did measure, with the testbed stopped, is the sequential cost on this ten-core host: a 512-header aggregate, two leaf proofs and the aggregation one after the other on all cores, took 52.7 and 52.2 s in two runs (the leaf proofs 38 and 36 s together, the aggregation 14 and 16 s), about 42 blocks at 0.8 blocks per second, inside the bound of 128. The 4.3–6.6 minutes of the live deployment were measured while the validators ran on the same cores, and the 31.5 s per leaf proof of Appendix C on the

same workstation at another time, so these latencies depend on the load of the host. Whether the prover keeps up with block production therefore depends on whether it has cores of its own: aggregation makes the proof verifiable at a cost that does not depend on the interval, and keeping the unproven tail within the bound requires a prover that is not competing with the validators. The validators verify the submitted proof during block execution, so the verification cost does not change with how the proof was built.

8.2. Interpretation of Results

The formal and empirical results cover complementary ground. The TLA+ results establish, per layer and within bounded configurations, that the FSM's safety invariants hold and that removing any of four defensive mechanisms reproduces the violation it targets; Lemma 1 bounds the certified health input by honest observations. The live evaluation shows that the composed implementation behaves as specified under real peripheral failures. Because the cross-layer refinement is open, the whole-system claim rests on the empirical side. Single-trial live results (composite fault, seven-scenario sweep, P4, reorg depth) show that correct behavior occurs without characterizing its distribution; the repeated experiments (hysteresis sweeps, the five-repeat overhead benchmark, the 2,000-trial detector study) do characterize distributions.

Unexpected results. Three results need a causal account.

- *Block rate.* Against a single local vanilla process, Engram's block interval is longer, which at first suggested a cost of the extended object. Seven vanilla validators on the same hosts give the same interval, with vote extensions on or off, so the gap is the cost of the seven-validator network and Engram's own addition is a few percent at most (point estimate 2%, 95% interval -0.6 to 5.0% over five deployments; Section 7.4).
- *Hysteresis threshold.* A static threshold of 2 flapped under wide-area jitter (9.0 flaps per validator), and thresholds of 4, 6 and 8 did not (Section 7.3; three runs for 2 and 6, one for 4 and 8). We did not isolate which sensor crosses its threshold; a run with the jitter profile removed, and per-predicate logging, would show it.
- *Reorg depth.* Reorgs of depth 3–5 orphaned confirmed blocks without a reaction, while depth 15 escalated through the suspicious-duration bound (Appendix D). Whether the anchor-reverification flag was ever set is not established, because the per-validator sensor value was not captured; logging it and sweeping depths 6–14 would locate the boundary.

8.3. Limitations

- *Equal power.* The live runs and the TLA+ model use equal voting power. The weighted aggregation is tested on unequal power only as a function (Section 7.1), not on a live network.
- *Testbed.* All results come from one host with accelerated Bitcoin (thirty times mainnet; conversions and a model in the exposure paragraph above) and single-bridge DA trust; the backfill gate rests on a quorum-reported contiguous height, not a proof of retrievability (Section 6.10).
- *Proof gate.* The recovery-validity flag holds if a verified proof was submitted within the permitted unproven tail of the tracked tip, or if a heuristic holds: the last submitted Bitcoin checkpoint has reached the anchored depth. A protocol parameter disables the heuristic, so that only a verified proof satisfies the gate. The RQ2 trials and the five repeated composite runs used proof-only recovery with an unproven-tail bound of 128 blocks; the first composite run, the seven-scenario run and the data-availability runs of Section 7.3 were made with the heuristic enabled, so their time in RECOVERING does

not isolate the cost of the recovery proof. Recovery through a proof also depends on the unproven-tail bound and on the testbed's block rate and proving speed, as discussed under the unproven tail above.

- *Byzantine scale.* The single-actor attacks involve one Byzantine actor at a time; the $f=2$ collusion suite (Section 7.1) addresses colluding validators.
- *Fixed validator set.* The prototype assumes one validator set throughout a degraded interval (Section 3.1).
- *Formal scope.* The cross-layer refinement is open, and the health certificate is modelled with Boolean observations in the server layer; numeric aggregation is covered by Lemma 1 only (Section 5).
- *Load range.* Load was offered at up to 500 tx/s on one host (Section 7.4). Above 100 tx/s Engram's latency grows and its FSM can leave ANCHORED; neither system saturated, and no throughput ceiling or production scalability is claimed.

8.4. Threats to Validity

Internal validity. Host sharing can confound cost measurements. Repeated measurements with confidence intervals, and a vanilla comparison on the same host under the same load, mitigate this; the host still affects absolute values.

External validity. Results at seven validators on one host need not extrapolate to larger or geographically distributed validator sets; they are evidence of correct behavior and of the source of the overhead, not production performance figures.

Construct validity. AppHash agreement and height progression are the conventional operational proxies for safety and liveness in CometBFT-family systems. AppHash agreement alone cannot detect an illegal transition agreed on by all validators; the semantic replay check of Section 7.1 addresses this gap.

8.5. Future Work

Three directions remain beyond the items marked in this paper. First, closing the cross-layer refinement obligation, and extending the certificate model from Boolean to numeric observations, would extend the formal results to the composed system; deductive methods [44, 71] are a natural complement where bounded model checking does not scale. Second, replacing the fixed-arity aggregator with incremental recursion would make the recovery artifact constant-size for intervals of any length, and proving the chunks in parallel or on dedicated hardware would shorten the recovery whose latency the unproven-tail discussion measures. Third, replacing single-bridge DA trust with sampling across independent peers would let the DA sensor detect partial withholding rather than only outages. A further step is the external verifier named in Section 4: a light client or bridge on another chain that accepts a re-anchoring checkpoint only with a valid proof. We have not built one or measured its verification cost.

Reducing the proposer's influence on the certificate. Two steps would narrow the choice of which honest observations enter the aggregate, and neither is implemented. The first requires one observation in the certificate for every commit vote of `LastCommit`. It is a single added condition in `ProcessProposal`, it does not affect an honest proposer, and it makes an omission visible and attributable, but the proposer still chooses which precommits `LastCommit` records, because CometBFT accepts any `LastCommit` with more than two thirds of the voting power. Removing that choice is the second step: changing how the consensus core selects the precommits it records. That alters the agreement protocol, needs a new safety and liveness analysis of the

changed core, and would require every experiment to be run again. The quantified influence of Section 4.3 stays the guarantee of this paper.

Asset-level safety. The lock is a necessary condition for protecting bridged assets, not a sufficient one. Establishing asset safety needs (i) a bridge on the counterparty chain that releases funds only against a proof that the Engram height is committed in an anchored state, with escrow accounting and replay protection, so that the sum of external and escrowed supply stays constant; (ii) a treatment of history rewriting by a coalition above the fault bound while the chain is in SOVEREIGN mode, which Section 4 leaves open; and (iii) a bound on the exposure before the lock, for example by a withdrawal quota per Bitcoin block. We evaluate none of these.

9. Conclusion

A chain whose block validity requires settlement and DA receipts halts when those layers fail, although its validators are connected and correct: an external dependency can cause a liveness failure without a BFT failure. Engram makes suspending such a dependency an agreed validity and safety mode. A quorum-backed health certificate drives a four-state FSM above an unchanged CometBFT agreement core. In the degraded modes the corresponding receipt requirement is relaxed and withdrawal-marked transactions are excluded, and the return to ANCHORED requires recovery hysteresis, DA backfill of the SOVEREIGN interval, and a validator-verified recovery proof.

The evidence supports the mechanism within the stated assumptions. It does not show that cross-chain assets are protected: that needs a bridge that honors the lock, which the prototype does not implement. Bounded model checking found no violations in each layer (the largest run explored 4.54 million distinct states); the cross-layer refinement remains open. A seven-validator testbed with live Bitcoin and Celestia networks kept committing blocks under a composite fault and completed a full recovery cycle with complete AppHash agreement in each of five runs. Against seven vanilla validators on the same hosts, Engram lengthens the block interval by a few percent at most (point estimate 2%, interval −0.6 to 5.0%) (1.262 s against 1.233 s).

Artifact Availability

The implementation, TLA+ specifications, and experiment scripts are in three repositories:

- `engram-sovereign-fsm`: the Cosmos SDK application, the TLA+ specifications, the recovery circuits, the experiment scripts, and this manuscript. `https://github.com/cuongct924/engram-sovereign-fsm`
- `engram-consensus-core`: the CometBFT fork used as the agreement core (Section 6.1 lists its four differences from upstream). `https://github.com/cuongct924/engram-consensus-core`
- `engram-barretenberg-wasm`: the build of the UltraHonk verifier embedded in the application; it is provenance only, not a build dependency. `https://github.com/cuongct924/engram-barretenberg-wasm`

The first two repositories are released under the Apache License 2.0. [PENDING: release tag or commit of each repository that produced the reported results.]

References

[1] M. Al-Bassam, "LazyLedger: A Distributed Data Availability Ledger With Client-Side Smart Contracts," arXiv:1905.09274, 2019, https://arxiv.org/abs/1905.09274.

[2] E. Buchman, "Tendermint: Byzantine Fault Tolerance in the Age of Blockchains," Ph.D. dissertation, University of Guelph, 2016, https://atrium.lib.uoguelph.ca/items/6c1ad7d4-7e5c-4f7f-b3d4-3f60d4fda1f5.

[3] E. Buchman, J. Kwon, and Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938, 2018, https://arxiv.org/abs/1807.04938.

[4] J. Neu, E. N. Tas, and D. Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma," in *Proceedings of the 2021 IEEE Symposium on Security and Privacy (S&P)*, 2021, pp. 446–465.

[5] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, "Combining GHOST and Casper," arXiv:2003.03052, 2020.

[6] F. Gomes Figueira, M. Derka, C. L. Chiu, and J. Gorzny, "A Practical Rollup Escape Hatch Design," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2025)*, 2025, https://arxiv.org/abs/2503.23986.

[7] Offchain Labs, "Inside AnyTrust," Arbitrum documentation, https://docs.arbitrum.io/inside-anytrust, accessed 2026-10-02.

[8] Optimism contributors, "Alt-DA Mode Specification," OP Stack experimental specification, https://github.com/ethereum-optimism/specs/blob/main/specs/experimental/alt-da.md, accessed 2026-10-03.

[9] Cosmos SDK contributors, "ADR 064: ABCI 2.0 Integration (Phase II)," Cosmos SDK architecture decision record, https://github.com/cosmos/cosmos-sdk/blob/main/docs/architecture/adr-064-abci-2.0.md, accessed 2026-10-03.

[10] Skip Protocol and dYdX contributors, "Slinky: A General-Purpose Oracle Leveraging ABCI++," Software repository (continued as skip-mev/connect), https://github.com/dydxprotocol/slinky, accessed 2026-10-03.

[11] N. C. Mendonça, C. M. Aderaldo, J. Cámara, and D. Garlan, "Model-Based Analysis of Microservice Resiliency Patterns," in *Proceedings of the 2020 IEEE International Conference on Software Architecture (ICSA)*, 2020, pp. 114–124.

[12] R. Pass and E. Shi, "Thunderella: Blockchains with Optimistic Instant Confirmation," in *Advances in Cryptology – EUROCRYPT 2018*. Springer, 2018.

[13] P.-L. Aublin, R. Guerraoui, N. Knežević, V. Quéma, and M. Vukolić, "The Next 700 BFT Protocols," *ACM Transactions on Computer Systems*, vol. 32, no. 4, pp. 1–45, 2015.

[14] N. Giridharan, F. Suri-Payer, I. Abraham, L. Alvisi, and N. Crooks, "Autobahn: Seamless High Speed BFT," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP 2024)*, 2024, pp. 1–23.

[15] R. Pass and E. Shi, "The Sleepy Model of Consensus," in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science, vol. 10625, 2017, pp. 380–409.

[16] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437, 2017.

[17] T. D. Chandra and S. Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[18] W. Chen, S. Toueg, and M. K. Aguilera, "On the Quality of Service of Failure Detectors," *IEEE Transactions on Computers*, vol. 51, no. 5, pp. 561–580, 2002.

[19] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ Accrual Failure Detector," in *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems (SRDS)*, 2004, pp. 66–78.

[20] A. Doudou, B. Garbinato, R. Guerraoui, and A. Schiper, "Muteness Failure Detectors: Specification and Implementation," in *Dependable Computing – EDCC-3*, 1999, pp. 71–87.

[21] D. Dolev, N. A. Lynch, S. S. Pinter, E. W. Stark, and W. E. Weihl, "Reaching Approximate Agreement in the Presence of Faults," *Journal of the ACM*, vol. 33, no. 3, pp. 499–516, 1986.

[22] D. Stolz and R. Wattenhofer, "Byzantine Agreement with Median Validity," in *19th International Conference on Principles of Distributed Systems (OPODIS 2015)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 46, 2015, pp. 22:1–22:14.

[23] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town Crier: An Authenticated Data Feed for Smart Contracts," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 270–282.

[24] S. Eskandari, M. Salehi, W. C. Gu, and J. Clark, "SoK: Oracles from the Ground Truth to Market Manipulation," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '21)*, 2021, pp. 127–141.

[25] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A Decentralized Truth Discovery Approach to the Blockchain Oracle Problem," in *Proceedings of IEEE INFOCOM 2023*, 2023, pp. 1–10.

[26] Y. Yao, Y. Shi, G. Tian, M. Miao, and W. Susilo, "PSCBO: A Provably Secure Consensus-Based Blockchain Oracle," *Computer Standards & Interfaces*, vol. 91, 2025.

[27] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi, "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients," in *Financial Cryptography and Data Security (FC 2021)*, ser. Lecture Notes in Computer Science, vol. 12675, 2021, pp. 279–298.

[28] D. Pham Minh, H. Vuong Huu, and D. A. Tran, "Efficient Data Availability Sampling via Coded Distributed Arrays," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2026)*, 2026, https://arxiv.org/abs/2606.16200.

[29] D. Karakostas and A. Kiayias, "Securing Proof-of-Work Ledgers via Checkpointing," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2021)*, 2021.

[30] E. N. Tas, D. Tse, F. Yu, and S. Kannan, "Babylon: Reusing Bitcoin Mining to Enhance Proof-of-Stake Security," arXiv:2201.07946, 2022, https://arxiv.org/abs/2201.07946.

[31] S. Azouvi and M. Vukolić, "Pikachu: Securing PoS Blockchains from Long-Range Attacks by Checkpointing into Bitcoin PoW using Taproot," in *Proceedings of the 2022 ACM Workshop on Developments in Consensus (ConsensusDay '22)*, 2022, pp. 53–65.

[32] E. N. Tas, D. Tse, F. Gai, S. Kannan, M. A. Maddah-Ali, and F. Yu, "Bitcoin-Enhanced Proof-of-Stake Security: Possibilities and Impossibilities," in *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 126–145.

[33] M. Capretto, M. Ceresa, A. Fernández Anta, P. Moreno-Sanchez, and C. Sánchez, "A Secure Sequencer and Data Availability Committee for Rollups," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.

[34] ——, "A Decentralized Sequencer and Data Availability Committee for Rollups Using Set Consensus," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2026)*, 2026, pp. 1–9.

[35] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, "zkBridge: Trustless Cross-chain Bridges Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 3003–3017.

[36] P. Vesely, K. Gurkan, M. Straka, A. Gabizon, P. Jovanovic, G. Konstantopoulos, A. Oines, M. Olszewski, and E. Tromer, "Plumo: An Ultralight Blockchain Client," in *Financial Cryptography and Data Security (FC 2022)*, 2022, pp. 597–614.

[37] J. Bonneau, I. Meckler, V. Rao, and E. Shapiro, "Coda: Decentralized Cryptocurrency at Scale," Cryptology ePrint Archive, Paper 2020/352, 2020, later renamed Mina.

[38] B. Bünz, L. Kiffer, L. Luu, and M. Zamani, "FlyClient: Super-Light Clients for Cryptocurrencies," in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 928–946.

[39] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 111–120.

[40] S. Bowe, J. Grigg, and D. Hopwood, "Recursive Proof Composition without a Trusted Setup," Cryptology ePrint Archive, Paper 2019/1021, 2019.

[41] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes," in *Advances in Cryptology – CRYPTO 2022*, ser. Lecture Notes in Computer Science, vol. 13510, 2022, pp. 359–388.

[42] I. Konnov, J. Kukovec, and T.-H. Tran, "TLA+ Model Checking Made Symbolic," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, 2019.

[43] S. Braithwaite, E. Buchman, I. Konnov, Z. Milosevic, I. Stoilkovska, J. Widder, and A. Zamfir, "Formal Specification and Model Checking of the Tendermint Blockchain Synchronization Protocol (Short Paper)," in *2nd Workshop on Formal Methods for Blockchains (FMBC 2020)*, ser. OpenAccess Series in Informatics (OASIcs), vol. 84, 2020.

[44] O. Padon, K. L. McMillan, A. Panda, M. Sagiv, and S. Shoham, "Ivy: Safety Verification by Interactive Generalization," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2016, pp. 614–630.

[45] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill, "IronFleet: Proving Practical Distributed Systems Correct," in *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*, 2015, pp. 1–17.

[46] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson, "Verdi: A Framework for Implementing and Formally Verifying Distributed Systems," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2015, pp. 357–368.

[47] L. Qiu, Y. Kim, J.-Y. Shin, J. Kim, W. Honoré, and Z. Shao, "LiDO: Linearizable Byzantine Distributed Objects with Refinement-Based Liveness Proofs," *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, 2024.

[48] S. Bano, A. Sonnino, A. Chursin, D. Perelman, Z. Li, A. Ching, and D. Malkhi, "Twins: BFT Systems Made Robust," in *25th International Conference on Principles of Distributed Systems (OPODIS 2021)*, ser. LIPIcs, vol. 217, 2022, pp. 7:1–7:29.

[49] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[50] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, 1999, pp. 173–186.

[51] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony," *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.

[52] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, "HotStuff: BFT Consensus with Linearity and Responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.

[53] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus," in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*, 2022, pp. 34–50.

[54] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, "Bullshark: DAG BFT Protocols Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 2705–2718.

[55] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and Efficient Asynchronous Broadcast Protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. Lecture Notes in Computer Science, vol. 2139. Springer, 2001, pp. 524–541.

[56] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath, "BFT Protocol Forensics," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021, pp. 1722–1743.

[57] W. Honoré, L. Qiu, Y. Kim, J.-Y. Shin, J. Kim, and Z. Shao, "AdoB: Bridging Benign and Byzantine Consensus with Atomic Distributed Objects," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 419–448, 2024.

[58] W. Honoré, J.-Y. Shin, J. Kim, and Z. Shao, "Adore: Atomic Distributed Objects with Certified Reconfiguration," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '22)*, 2022, pp. 379–394.

[59] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[60] J. R. Douceur, "The Sybil Attack," in *Peer-to-Peer Systems (IPTPS 2002)*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 251–260.

[61] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg, "Eclipse Attacks on Bitcoin's Peer-to-Peer Network," in *Proceedings of the 24th USENIX Security Symposium*, 2015, pp. 129–144.

[62] Z. Rehman, M. A. Gregory, I. Gondal, H. Dong, and M. Ge, "Eclipse Attacks in Blockchain Networks: Detection, Prevention, and Future Directions," *IEEE Access*, vol. 13, pp. 25 918–25 933, 2025.

[63] A. Back, M. Corallo, L. Dashjr, M. Friedenbach, G. Maxwell, A. Miller, A. Poelstra, J. Timón, and P. Wuille, "Enabling Blockchain Innovations with Pegged Sidechains," Whitepaper, Blockstream, 2014.

[64] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, Private Smart Contracts," in *Proceedings of the 27th USENIX Security Symposium*, 2018, pp. 1353–1370.

[65] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais, "SoK: Layer-Two Blockchain Protocols," in *Financial Cryptography and Data Security (FC 2020)*, ser. Lecture Notes in Computer Science, vol. 12059, 2020, pp. 201–226.

[66] "Arbitrum Orbit AnyTrust Chains," Arbitrum blog, https://blog.arbitrum.io/arbitrum-orbit-anytrust-chains/, accessed 2026-10-02.

[67] S.-S. Lee, A. Murashkin, M. Derka, and J. Gorzny, "SoK: Not Quite Water Under the Bridge: Review of Cross-Chain Bridge Hacks," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2023)*, 2023, pp. 1–14, arXiv:2210.16209.

[68] E. Deirmentzoglou, G. Papakyriakopoulos, and C. Patsakis, "A Survey on Long-Range Attacks for Proof of Stake Protocols," *IEEE Access*, vol. 7, pp. 28 712–28 725, 2019.

[69] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019.

[70] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002, https://lamport.azurewebsites.net/tla/book.html.

[71] R. Lotan and S. Shoham, "Implicit Rankings for Verifying Liveness Properties in First-Order Logic," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, 2025.

[72] N. Bondarev, K. Ziborov, and Y. Yanovich, "Systematization of Knowledge: Formal Verification of Consensus Protocols," arXiv:2608.21935, 2026, https://arxiv.org/abs/2608.21935.

[73] J. Kwon and E. Buchman, "Cosmos: A Network of Distributed Ledgers," Whitepaper, 2016, https://v1.cosmos.network/resources/whitepaper.

[74] Aztec Labs, "Noir: A Domain-Specific Language for Zero-Knowledge Proofs, and the Barretenberg Proving Backend," Software documentation, 2024, noir: https://noir-lang.org; Barretenberg: https://github.com/AztecProtocol/barretenberg.

Appendix

A. Extended Attack-Resilience Results

Table 30 reports the ten-scenario single-actor attack suite, rerun for this paper on the seven-validator testbed (n=7) with three repeats per scenario. The repeats of a scenario share one chain, each starting from ANCHORED, so they are not independent deployments. A3 and A4 are the cases of the collusion suite (Section 7.1) with one Byzantine validator. The column *Byz. VP* gives the share of voting power held by the misbehaving validator: one of seven validators is 14.3%, and the attacker containers of A1 and A2 are non-validators (0%).

Table 30: Single-actor attack suite at n=7 (live; three repeats each; safety held and zero AppHash divergence in all 30 repeats)

#	Attack	Byz. VP	Attack-specific signal
A1	Eclipse (10 attackers, one subnet)	0%	Filter held in 3/3: peak 8 peers in the target subnet, equal to the limit of 8; nil-prevote ratio 0.0. The FSM left ANCHORED for SUSPICIOUS in 2/3 repeats
A2	Multi-subnet Sybil (12 attackers, 4 subnets)	0%	Filter held in 3/3 (peak 8 of 8). The FSM left ANCHORED for SUSPICIOUS in 2/3 repeats
A3	Data withholding	14.3%	Not rejected by honest validators; replay clean
A4	Forged Bitcoin receipt	14.3%	Rejected by honest validators (216 rejections over 3 repeats)
A5	Withdrawal during SOVEREIGN	—	Blocked in 3/3; the chain did not reach SOVEREIGN and the transaction never committed
A6	Malicious proposer (forged FSM state)	14.3%	Safety held in 3/3 (Section 7.1)
A7	Censorship	14.3%	Censoring observed in 3/3; liveness held; honest reject signals stayed at 0
A8	Combined (A6 + A1)	14.3%	Safety held under simultaneous overlap
A9	Double-signing	14.3%	6/6 honest validators logged real duplicate-vote evidence in 3/3 repeats, detected one block after the offense
A10	Timeout flood	14.3%	Cadence held: 0.754→0.785, 0.791→0.791 and 0.816→0.783 blocks/s (baseline→flood)

The SUSPICIOUS excursions in A1 and A2 occur in the attack phase and again in the recovery phase, after the attacker is gone, and the baseline phase was always ANCHORED. These two scenarios have no control run without an attacker, so we cannot separate an effect of the attack from the sensor noise that the collusion suite's control c0 shows on the same testbed. A3 and A4 were instrumented only for safety, divergence and rejections, without an attack-specific metric such as A5's blocked-withdrawal flag or A9's detection latency. No scenario drove block production toward zero.

B. Backend Trade-off: Noir+Honk vs. Plonky3

This appendix compares the deployed Noir/UltraHonk recovery circuit (Section 4.7) with a STARK-based alternative, Plonky3. The comparison is necessarily partial: boundary-signature verification, condition (e), dominates the deployed circuit's cost (1,097,274 opcodes with it versus 6,143 without, a factor of ∼180), and no Plonky3 configuration we built implements it.

Table 31 confirms that proving time, verification time, and proof size are independent of the number of real headers up to the compiled capacity $N_{\max}$=256. Verification by the validators'

Table 31: Deployed circuit: constant-cost scaling (Noir+Honk, Poseidon2)

Interval count	**Prove time**	**Verify time**	**Proof size**
1	38.3 s	25 ms	14,656 B
16	41.4 s	23 ms	14,656 B
256 ($N_{\max}$)	40.4 s	24 ms	14,656 B

embedded WebAssembly verifier costs ∼26 ms per proof. An interval of L blocks requires $\lceil L/256 \rceil$ leaf proofs when submitted one by one, or a single aggregate proof up to 1024 blocks (Appendix C).

Table 32: Backend comparison: statement coverage

Config.	(a)	(b)	(c)	(d)	(e)
Noir+Honk (deployed)	✓	✓	✓	✓	✓
Plonky3, primitive-only	–	–	–	–	–
Plonky3, fuller statement	✓	✓	✓	✓	–

(a) header-chain continuity, (b) mode-label consistency (every recorded state is SOVEREIGN or RECOVERING), (c) lock-flag consistency (every recorded lock flag is set), (d) root binding (the first header links to the last anchored root and the last header hashes to the new root), (e) boundary quorum certification (ed25519 signatures of the first and last block).

Table 32 makes the coverage gap explicit. Under a primitive-only comparison (256 chained Poseidon2 permutations, no FSM or signature logic, i.e. neither configuration implements condition (e)), Plonky3 produces a larger proof (1,278,939 B vs. 14,656 B, ∼87×) but proves faster (0.044 s vs. 0.684 s, ∼15.5×) with a transparent, post-quantum-secure setup versus the Noir/Honk backend's trusted KZG setup. Under a fuller-statement comparison that adds the mode-label, lock-flag and root-binding conditions (but still omits condition (e)), Plonky3 proves in 0.024–0.096 s with proof sizes of 84–124 KB across 2–256 headers. Neither comparison supports a general speed claim for Plonky3, because both omit the component that dominates the deployed circuit's cost. These numbers characterize primitive-level backend trade-offs only.

C. Chunk Aggregation

This appendix measures the aggregator of Section 4.7 against submitting the chunk proofs one by one. The measurements were taken offline on a 10-core, 32 GiB macOS workstation, not on Testbed A, with nargo 1.0.0-beta.22 and `bb` 5.0.0-nightly.20260522. The chain is synthetic: it has one deterministic header sequence with real ed25519 precommits and one validator set, so that consecutive chunks chain through their roots and heights. Aggregate proving and verification are averaged over five trials per interval length; each leaf proof was timed once. Absolute times are not comparable to Table 31, which was measured on a different host.

Table 33: Chunk aggregation against submitting chunk proofs one by one (K=4)

	One by one			**Aggregate**		
L	Prove	Bytes	Verify	Prove	Bytes	Verify
256	31.5 s	14,656	19.8 ms	57.2 s	14,656	20.2 ms
512	60.5 s	29,312	39.5 ms	89.9 s	14,656	20.2 ms
1024	120.1 s	58,624	79.1 ms	146.7 s	14,656	20.2 ms

Verify is the validators' embedded WebAssembly verifier, measured per proof (30 iterations, three runs); the one-by-one column is n times the leaf cost. Aggregate prove includes the leaf proofs. At L=256 a single leaf proof is submitted and the aggregate row is shown for the overhead only.

The aggregator verifies the chunk proofs in-circuit and checks that chunk i starts at the root and height where chunk $i-1$ ended, under the same validator-set hash. It exposes the same public inputs as a single chunk, so an aggregate proof stands in for a leaf proof, and its capacity $K=4$ is fixed at compile time. Unbounded incremental recursion would require the step circuit to receive its own verification-key hash as a public input, which we did not implement. Recursion is applied at the chunk level because in-circuit verification of one UltraHonk proof costs about 0.7 million gates, whereas hashing one header costs about 12 opcodes: recursion over per-header proofs measured 650–1,200× slower than direct hashing at matched sizes.

Table 33 shows that submitted bytes and verification time of the aggregate do not depend on L up to $K \cdot 256 = 1024$, while the one-by-one cost grows linearly. The price is one extra proof: the aggregator circuit has 3,002,299 gates and proves in 25.7–29.5 s using about 5.2 GB, against 3,234,226 gates, 30.0–31.5 s, and 5.7–5.9 GB for a leaf. Total proving time rises by 49% at $L=512$ and 22% at $L=1024$. Chunk proofs were produced sequentially; proving them in parallel was not measured. Every chunk boundary height needs a round-zero commit, so a round skip at an interior boundary prevents building that chunk's witness.

The aggregator rejects each of eleven malformed witnesses: reversed or repeated chunks, a wrong total count, final root, first height, or validator-set hash, a height gap between chunks, a different validator set in the second chunk, an extension through padding slots, and a chunk count outside $[1, K]$. The valid two-chunk witness is accepted, and a 512-header aggregate passes every `SubmitRecoveryProof` binding against fixture chain state. On the Docker testbed, aggregate proofs of three and two chunks (514 and 512 headers) were accepted by `SubmitRecoveryProof` on a live chain; whether the chain then leaves RECOVERING depends on the unproven-tail bound, as the Discussion reports. Aggregation bounds verification; the prover's time still grows with the interval.

D. Bitcoin Reorg Depth Reaction

Section 5.4 states a threat-model boundary – safety is modeled given Bitcoin reorganizations no deeper than a configurable finality depth $K_{\text{DEEP_FINALITY}}$ – as a precondition of the formal result. This appendix tests the implementation at and beyond that boundary with forced Bitcoin reorganizations, each confirmed from the resulting block hashes, at five depths in one seven-validator run: {1, 2, 3, 5, 15}, with $K_{\text{DEEP_FINALITY}}=2$ and a suspicious-state threshold of 5.

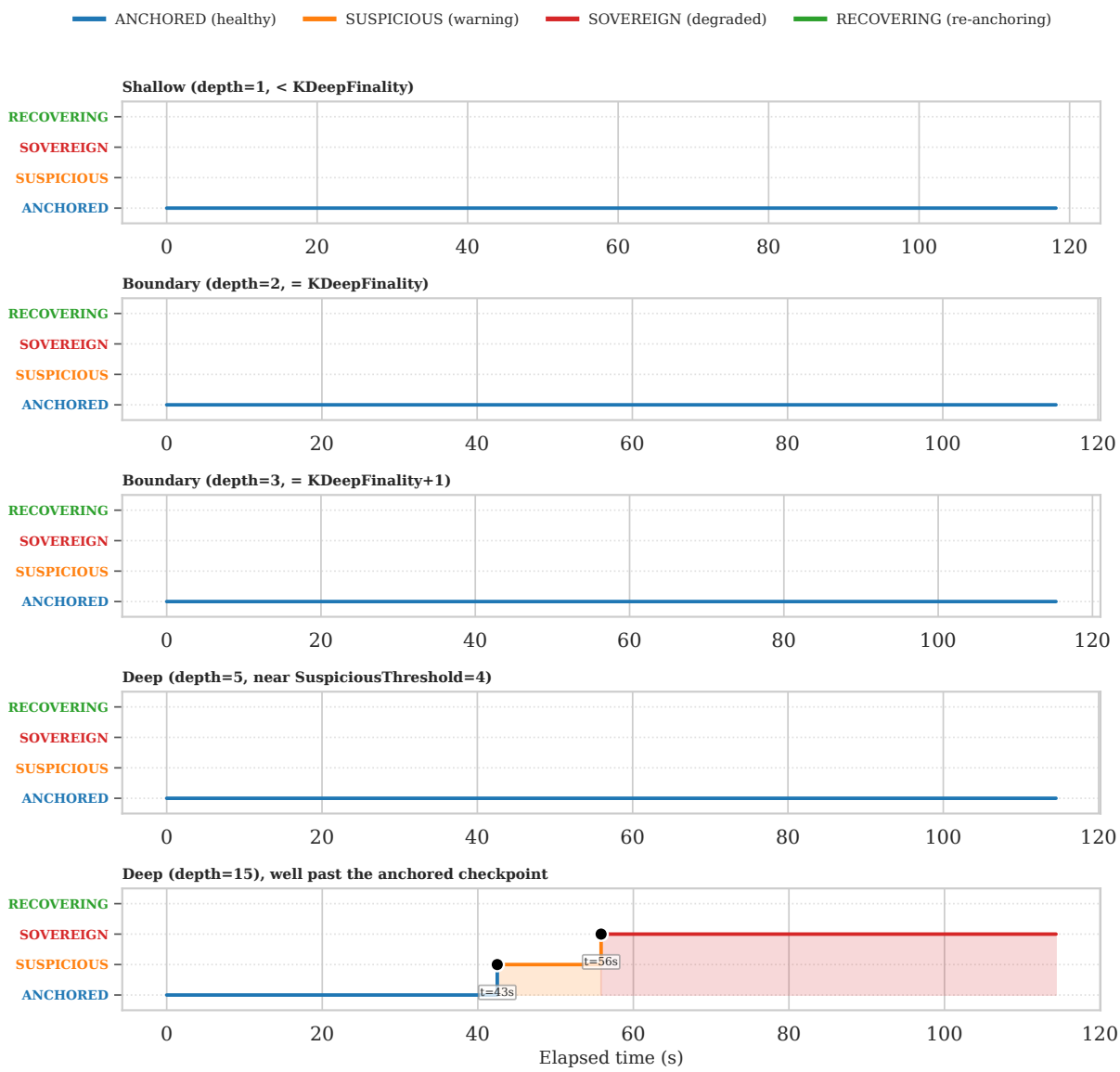


Figure 9: FSM reaction to Bitcoin reorgs at five depths, one trial per depth, seven-validator testbed. All 7 validators agree at every one of 181 polled rounds per trial (1,267 samples total, 0 RPC errors).

Table 34 summarizes the result. Depths 1–5 all orphan real, independently-confirmed blocks, yet none of them trip the FSM within the observation window: all 7 validators stay ANCHORED throughout. At depth 15, all 7 validators transition ANCHORED→SUSPICIOUS at t=43 s and →SOVEREIGN at t=56 s, in lockstep. This path rules out a direct critical-condition jump: the FSM's ANCHORED branch bypasses SUSPICIOUS entirely when the committed certificate's anchor-reverification flag is set, but the observed path instead passes through SUSPICIOUS, and the suspicious-duration counter reached 23 of the 24-block escalation bound (MaxSuspiciousTime) in the last sample before the SOVEREIGN transition. The depth-15 reaction is attributable to this duration-based escalation, not to anchor re-verification failing.

The crossover between depth 5 (no reaction) and depth 15 (reaction) was not swept at finer granularity. Whether anchor re-verification ever set the anchor-reverification flag during this run is not established by the available logs – the per-validator sensor value was not captured, only the committed FSM state and suspicious-duration counter. The data that is available shows the

Table 34: FSM reaction to real Bitcoin reorganizations by depth

Depth	Reorg confirmed?	FSM reaction
1 (below K_{DEEP})	N/A (too shallow)	Stays ANCHORED
2 (= K_{DEEP})	Yes	Stays ANCHORED
3 (= K_{DEEP}+1)	Yes	Stays ANCHORED
5	Yes	Stays ANCHORED (within window)
15 (past checkpoint)	Yes	SUSPICIOUS (43s) → SOVEREIGN (56s)

depth-15 escalation came from the suspicious-duration bound, not from a direct critical-condition jump.

E. Network Emulation Parameters

Table 35 lists every `tc netem` profile, applied through Pumba. No correlation or distribution option is set, so delay jitter is uniform and loss is independent per packet (the `netem` defaults). Profiles that name no interface use Pumba's default interface (`eth0`), which carries Bitcoin, Celestia, and RPC traffic rather than the dedicated pairwise validator links. Only the S4 scenario and the relay-latency attack (P4) of Section 7 target the validator links explicitly.

Table 35: Network emulation profiles (Testbed A)

Profile	Effect and target	Duration	Used by
`chaos-delay`	100 ms delay, 20 ms jitter, all seven validators	5 min	baseline
`chaos-loss`	5% loss, validators 1 and 2	2 min	composite
`chaos-eclipse`	100% loss, validator 1	3 min	S5
`chaos-btc-delay`	500 ms delay, 100 ms jitter, one `bitcoind`	2 min	composite
`chaos-wan-latency`	Per-validator delay and jitter, Table 36	10 min	WAN baseline
`chaos-wan-loss`	Per-validator loss, Table 36	10 min	WAN variant
S4 eclipse	300 ms delay, 50 ms jitter on the six validator links of validator 1	2 min	S4
P4 relay latency	350 ms delay, 50 ms jitter on all P2P interfaces of one validator	244 s	P4

Table 36: WAN baseline profiles per validator (one profile of seven emulated regions)

Validator	1	2	3	4	5	6	7
Delay (ms)	15	70	140	45	100	30	180
Jitter (ms)	3	15	25	10	20	6	30
Loss (%, loss variant)	1	3	6	2	4	1	5

Each validator receives one delay value on its egress, so the profile is a per-validator offset rather than a pairwise round-trip-time matrix. A pairwise matrix between regions is not measured on Testbed A.